Advanced Topics in Computer Graphics I

# Material Appearance - BTF-Acquisition


Prof. Dr. Reinhard Klein

Institut of Computer Science II – Computer Graphics
University of Bonn

June 30, 2020

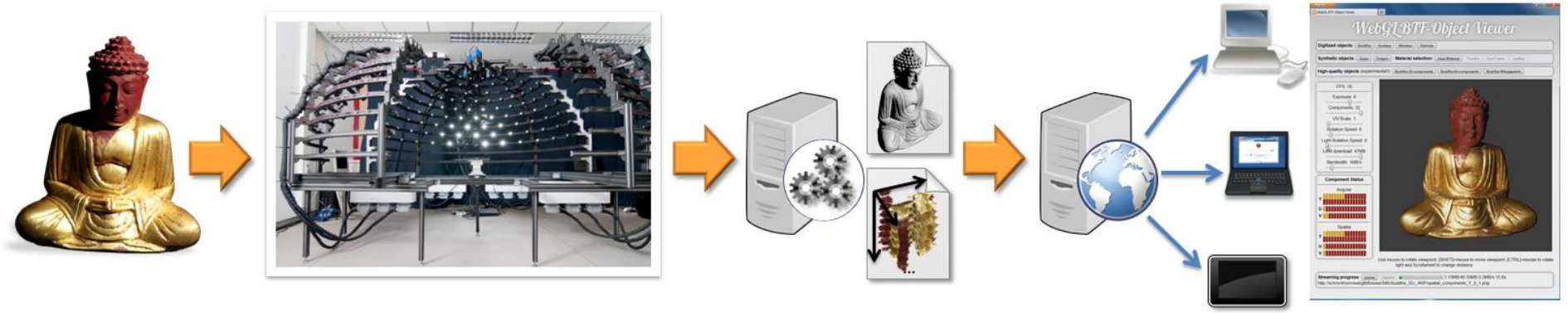
# Overview

Pipeline

Acquisition

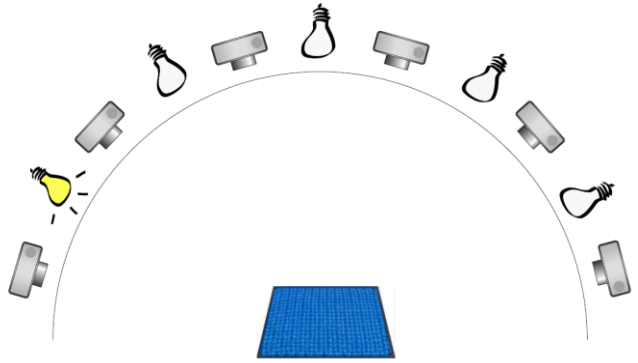
Modeling

Transmission and Rendering



# Reminder: capturing reflectance data

Sample the continous function $\mathcal{B}_V(\omega_i; \mathbf{x}_o, \omega_o)$:



Define

- a set of basis illuminations

$$\mathcal{L} = \{\mathbf{l}_i\}_{i \in I}$$

- a set of corresponding outgoing light-fields parametrized by an index set $K$ (one image for each direction)

$$\mathcal{R} = \{\mathbf{r}_i\}_{i \in I}$$

- Writing the incoming light-field as linear combination

$$\mathbf{l} = \sum_i l_i \mathbf{l}_i$$

- we get the discrete image-based relighting equation for BTFs

$$\mathbf{l}_o = \sum_{i \in I} l_i \mathbf{r}_i = \mathbf{B}^{(K \times I)} \mathbf{l}$$

discrete BTF

- the discrete BTF can be interpreted as Light Transport Matrix

# BTF-Compression

Preferable properties:

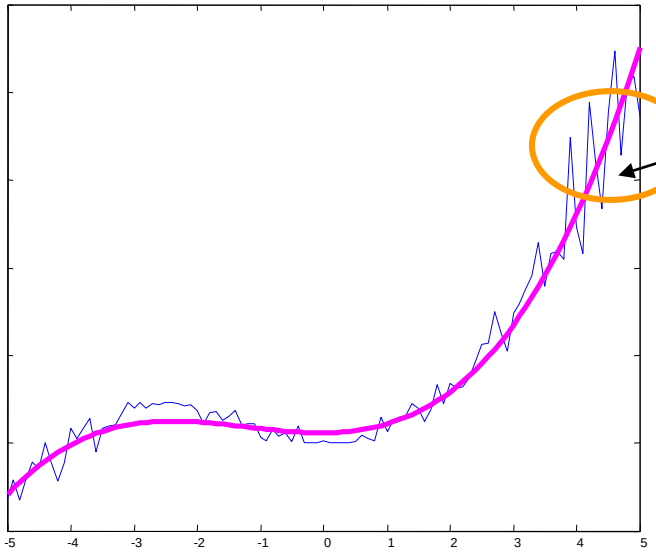
- asymmetric compression
  - fast (real-time), random access decompression
- preservation of visual important features
- maximum of a few MBs
  - depends on the current graphics card memory and material complexity of scene

# Fitting Analytical Functions

parameters

$F(\mathbf{x})$

$F(\mathbf{x},\mathbf{p})$




Feature or Noise?

$|\mathbf{p}|$ small, evaluation of $\tilde{F}(\mathbf{x},\mathbf{p})$ cheap

➔High compression and fast rendering

# Fitting Analytical Functions

Fixing spatial position

$$B_{\mathbf{x}}(\mathbf{v},\mathbf{l}) := BTF(\mathbf{x},\mathbf{v},\mathbf{l})$$

- 4D-function depending on incoming light direction $\mathbf{l}$ and outgoing view direction $\mathbf{v}$

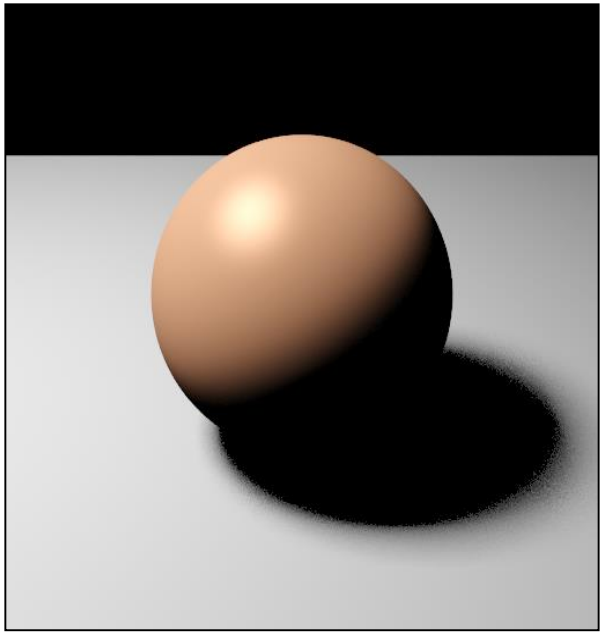
**Apply techniques from BRDF modeling!**

BTF VERSUS SVBRDF

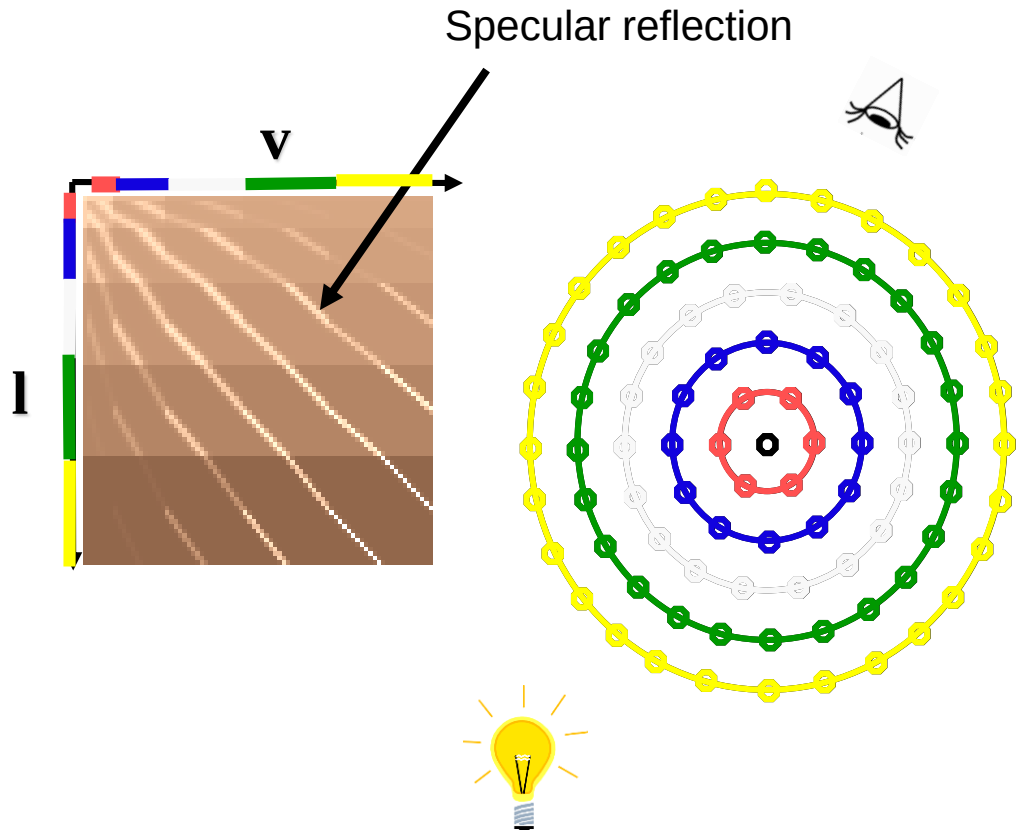
# Fitting analytical functions

How do these functions look like?

How do typical BRDFs look like?

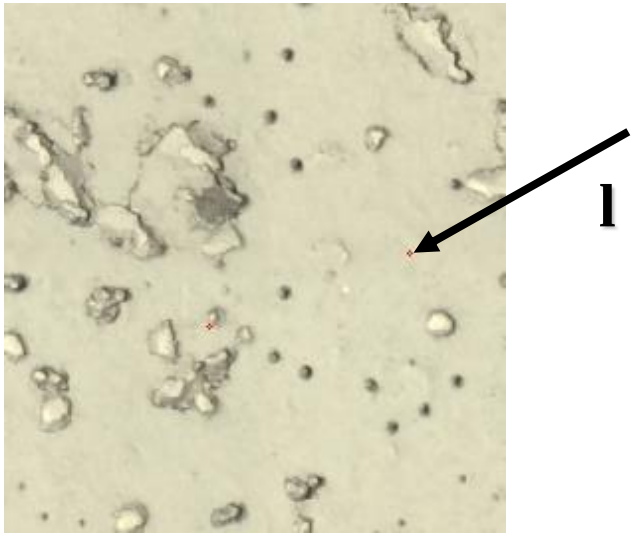


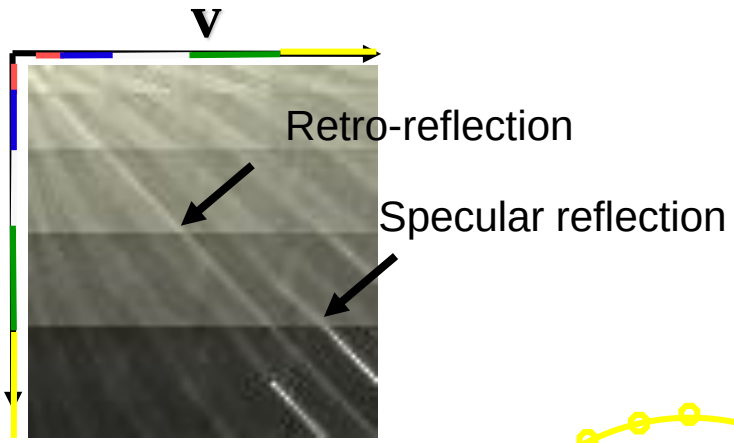
**shiny plastic**

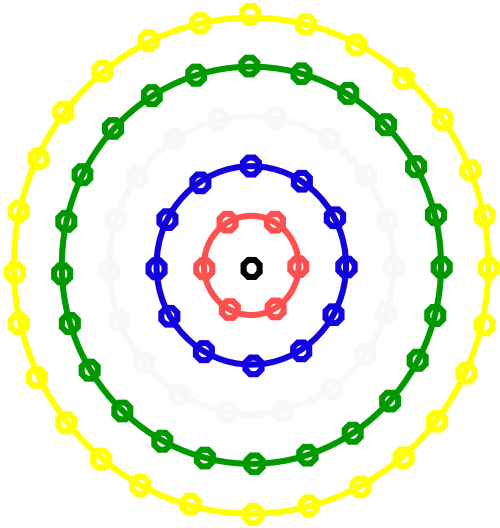


# Fitting Analytical Functions

Are these functions typical BRDFs?

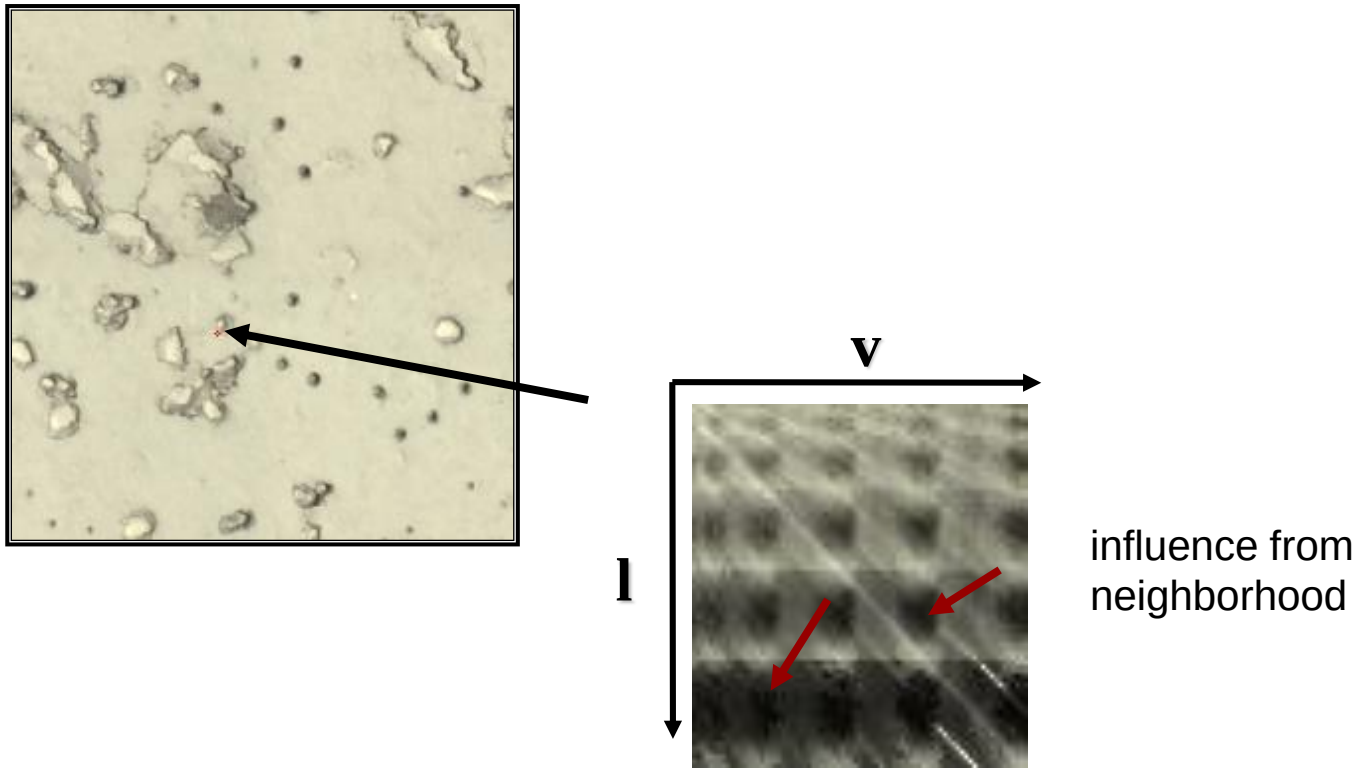




# Fitting Analytical Functions

Are these functions typical BRDFs?

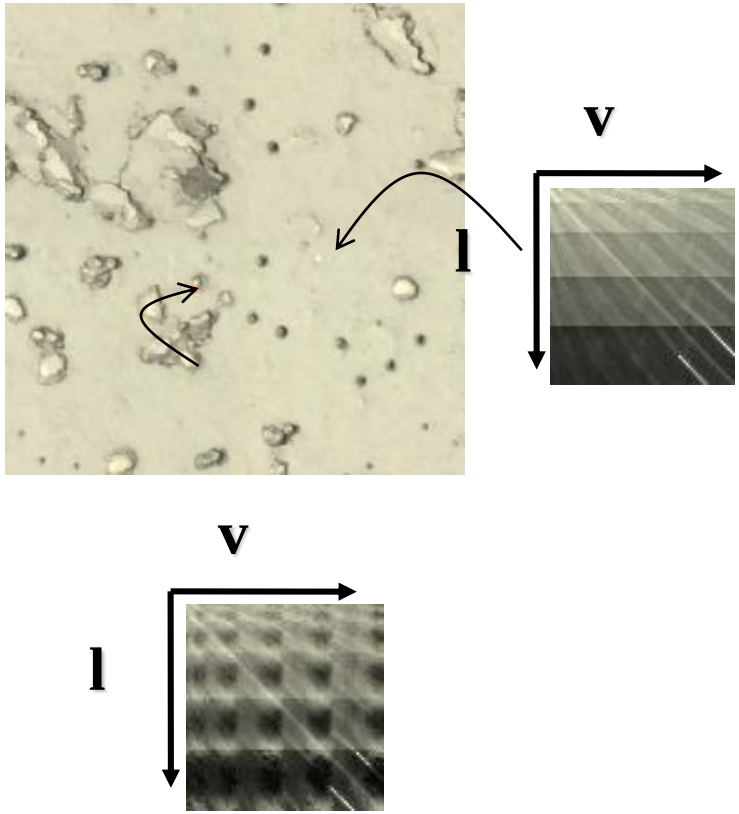



influence from neighborhood

# Fitting Analytical Functions

ABRDF = Apparent BRDF [Wong et al. 97]

- ►Contains also influence from neighborhood:
  - Self-Shadowing
  - Self-Occlusion
  - Sub-Surface Scattering
  - Resampling artefacts
  - …



# Fitting BRDF-Models

Generalized Cosine-Lobe Model [Lafortune et al. 97] :

$$\Rightarrow BTF(\mathbf{x},\mathbf{v},\mathbf{l}) \approx f_{\mathbf{x}}(\mathbf{v},\mathbf{l},\mathbf{p}) = \rho_{d,\mathbf{x}} + \sum_{i=1}^{k} \rho_{s,\mathbf{x}} \cdot \left(\mathbf{v}^T \mathbf{D}_{\mathbf{x},i} \mathbf{l}\right)^{n_{\mathbf{x},i}}$$

diffuse color

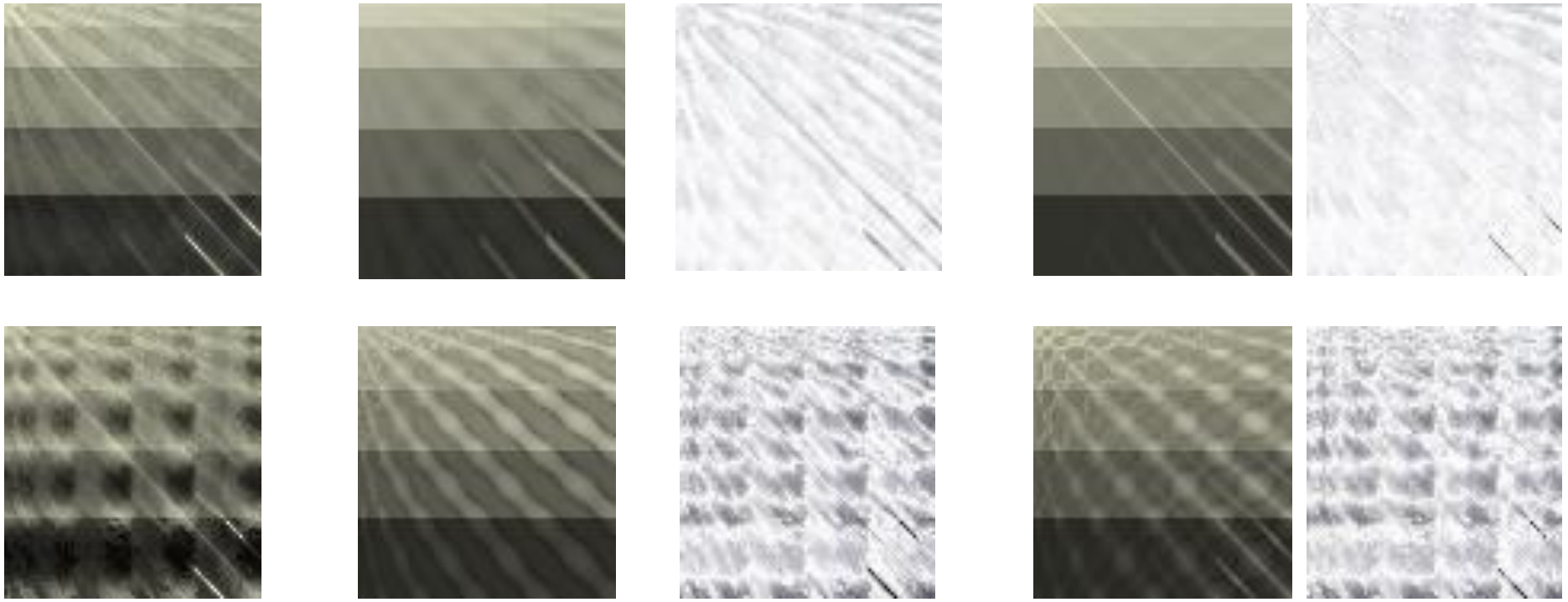
specular color

specular lobe

- Non-linear least-squares fitting (Levenberg-Marquardt)
- $\mathbf{k}$ typically around 2
- Improvement [Daubert et al. 2001]: view-dependent scale factor per texel to account for shadowing effects

# Fitting BRDF-Models

## Results (cosine lobe model)



Original | Fit (2 lobes) | Difference | Fit (10 lobes) | Difference

# Summary: BRDF-Models

Advantages

- ▶ High compression
- ▶ Simple evaluation

Problems

- ▶ Non-linear fitting
- ▶ Loss of depth impression!

- ▶ Possible solution:
  Use normal map!
  (later in the lecture)

Photography

Fit[1]

Examples: Normal map + BRDF Fitting[2]


1. David Kirk McAllister, *A generalized surface appearance representation for computer grap*hics, Dissertation University of North Carolina at Chapel Hill

2. Roland Ruiters, Christopher Schwartz und Reinhard Klein, *Example-based Interpolation and Synthesis of Bidirectional Texture Functions,* In: Computer Graphics Forum, 32:2(361-370)

# Statistical Data Analysis

Motivation

- ►Assuming general basis functions (polynomials, lobes, etc…) suboptimal for a given measured BTF-dataset as BRDFs do not model important features:
  - Self-Shadowing
  - Self-Occlusion
  - Sub-Surface Scattering
  - ABRDFs do not obey Helmholtz reciprocity

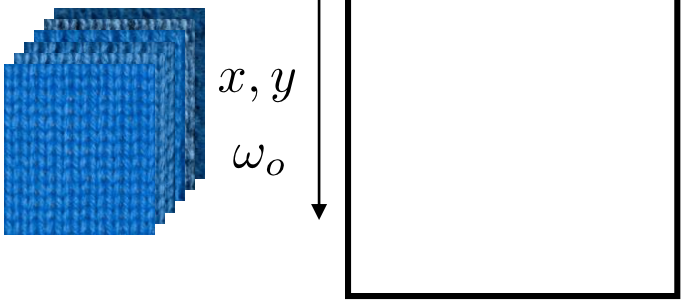
Idea

- ►Find customized basis functions adapted for the actual data set
- ⇔ Learn small set of “features” describing the data set best

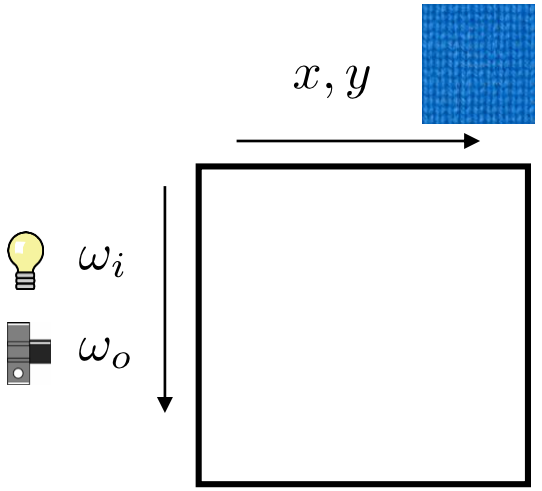
# Compression using statistical analysis

Organization of the discrete BTF

- As matrix

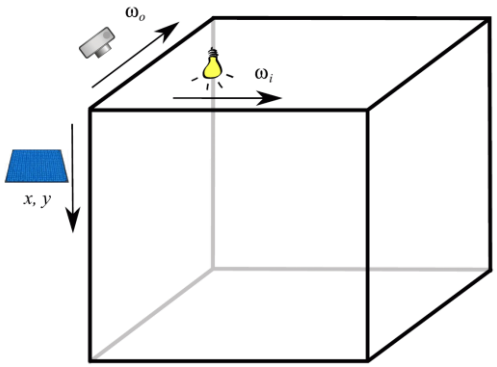



Light transport matrix

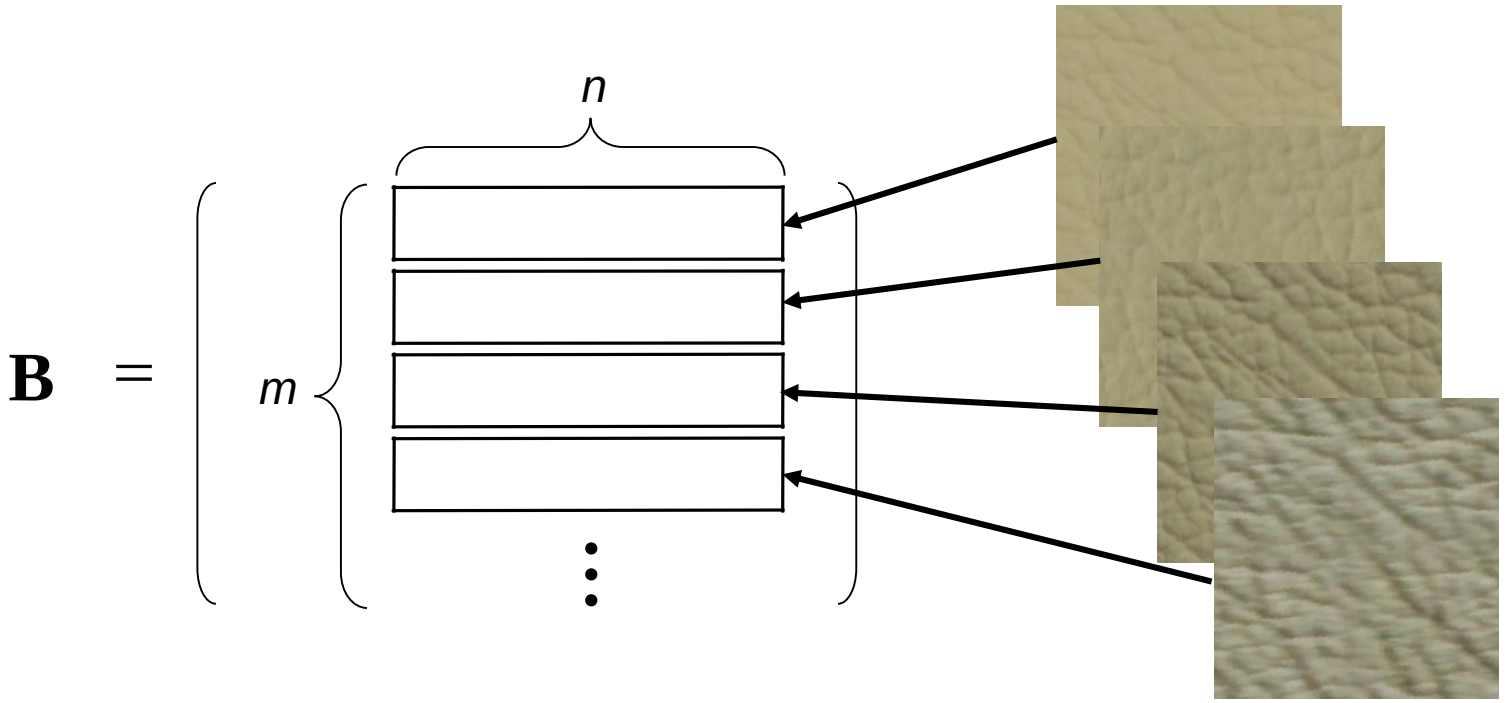



ABRDF/Image Matrix

- As Tensor



# Example ABRDF/Image Matrix

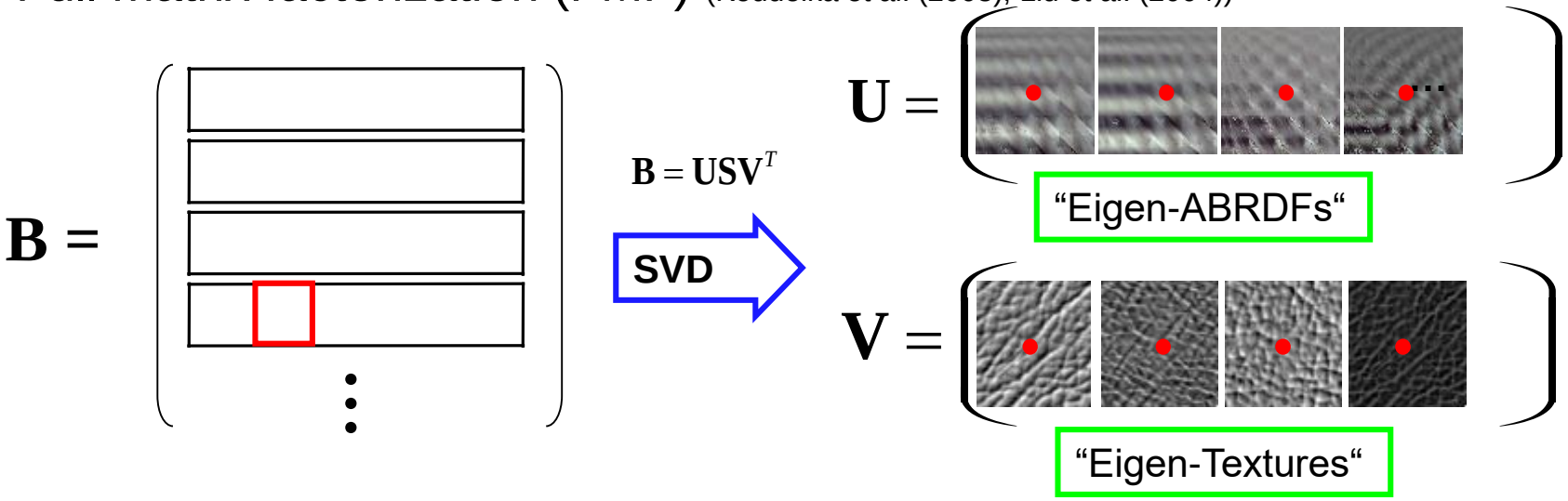



*n = x-resolution * y-resolution = 512 * 512 = 262.144*

*m = #views * #lights * #colors = 151 * 151 * 3 = 68.403*

# Matrix factorization

Full matrix factorization (FMF) (Koudelka et al. (2003), Liu et al. (2004))




ABRDF/Image Matrix

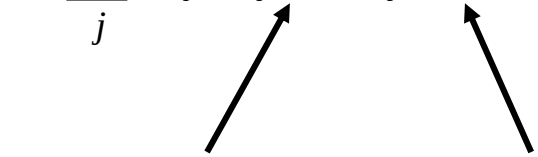
$$\mathbf{B} = \mathbf{U}\mathbf{S}\mathbf{V}^T = \sum_j^N \sigma_j \mathbf{u}_j \mathbf{v}_j^T$$

- Factorize the matrix $\mathbf{B} \approx \tilde{\mathbf{M}} = \sum_j^{r<<N} \sigma_j \mathbf{u}_j \mathbf{v}_j^T$
- Keep the first *r* columns of **U** and **V**
  - Usually about 100 sufficient for good quality
- Allows for random access (important for rendering!)

# Full BTF-Factorization

Write the BTF as sum of products of two functions

$$\Rightarrow BTF(\mathbf{x},\mathbf{v},\mathbf{l}) = \sum_{j}^{N} \sigma_j \cdot t_j(\mathbf{x}) b_j(\mathbf{v},\mathbf{l}) \approx \sum_{j}^{c} \sigma_j \cdot t_j(\mathbf{x}) b_j(\mathbf{v},\mathbf{l})$$



Eigen-Textures“

“Eigen-ABRDFs“

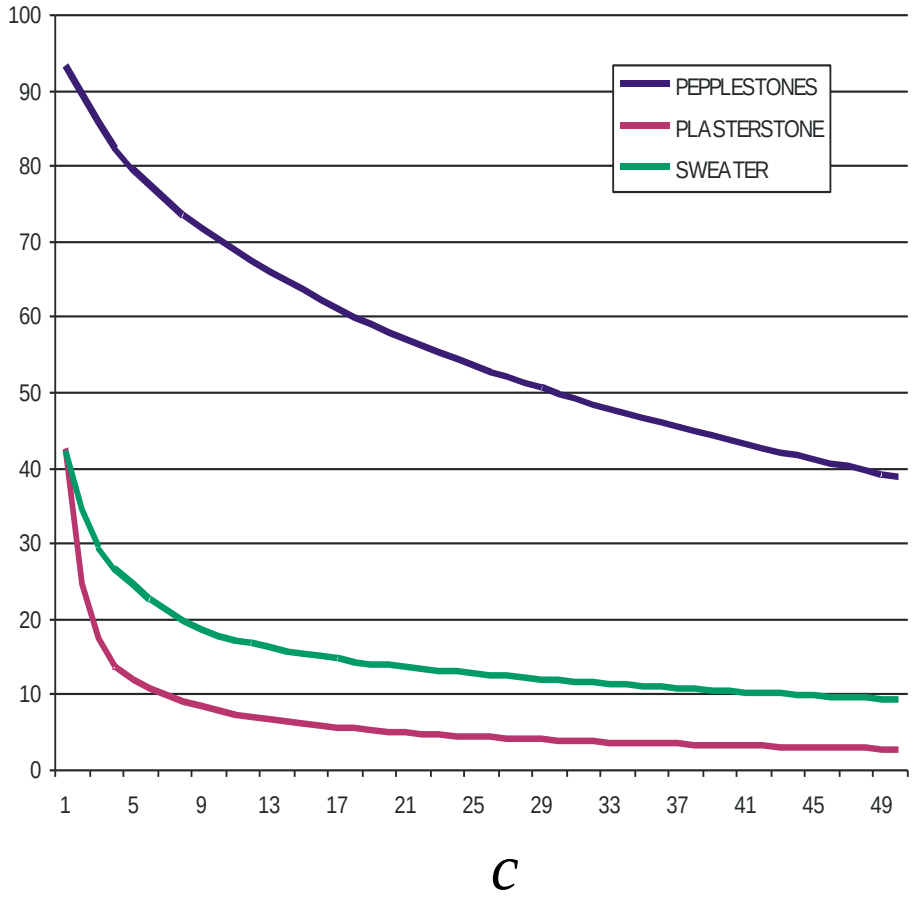
Note: smooth reconstruction requires interpolation

# Full BTF-Factorization

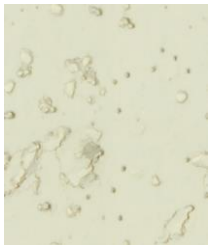
## Number of terms

RMS-Error

$$\varepsilon = \sqrt{\sum_{i=c+1}^{N} \sigma_i^2}$$




PEBBLESTONES



PLASTERSTONE

SWEATER

# Full matrix factorization (Example)

r=10

# Full matrix factorization (Example)

r=20

# Full matrix factorization (Example)

r=40

# Full matrix factorization (Example)

r=80

# Full matrix factorization (Example)

reference

# Full matrix factorization

Advantages

- Simple concept
- Good relationship between *Quality* level and *Compression Rate*
- Decompression with random access
- Reconstruction on GPU possible
  - E.g. by adding up the $\boldsymbol{r}$ weighted “Eigen-Textures”

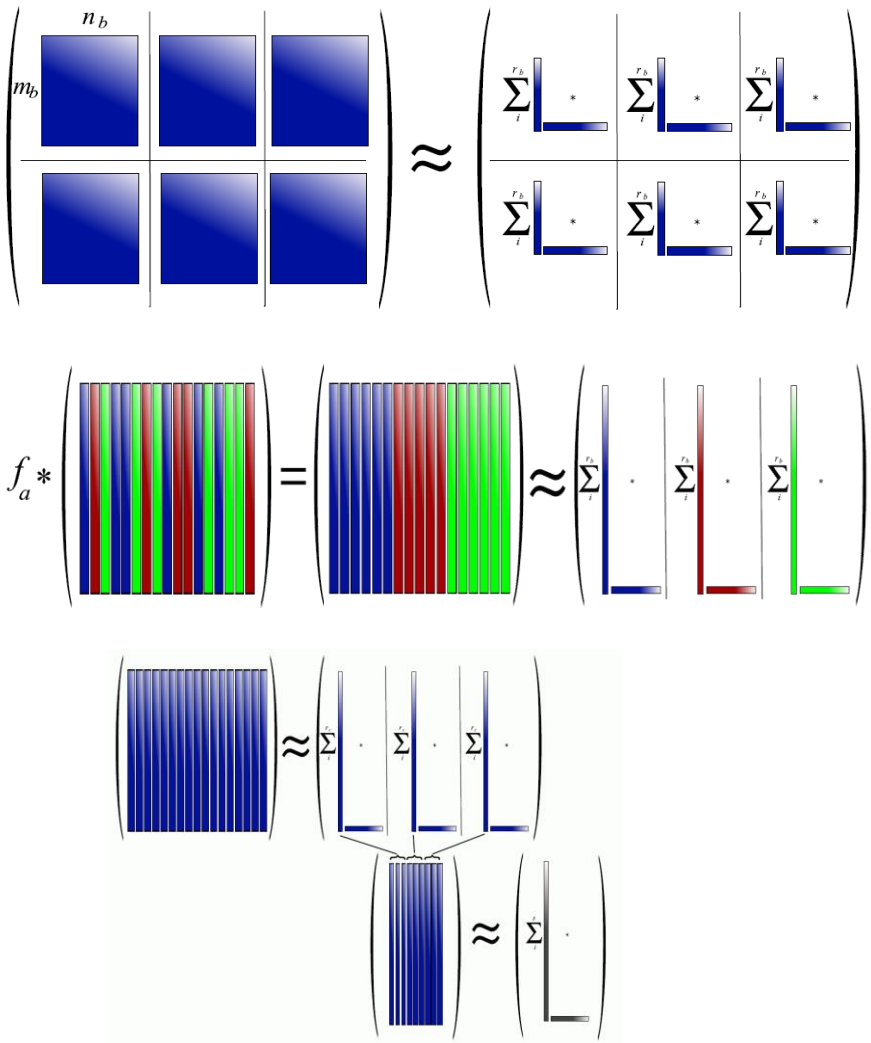
Drawbacks

- Data do not fit into main memory which requires out-of-core management during SVD computation
- Time consuming compression
- $\boldsymbol{r}$ often too large for interactive rendering
- Intra-image and intra-ABRDF correlations not fully considered

# Can we do better?

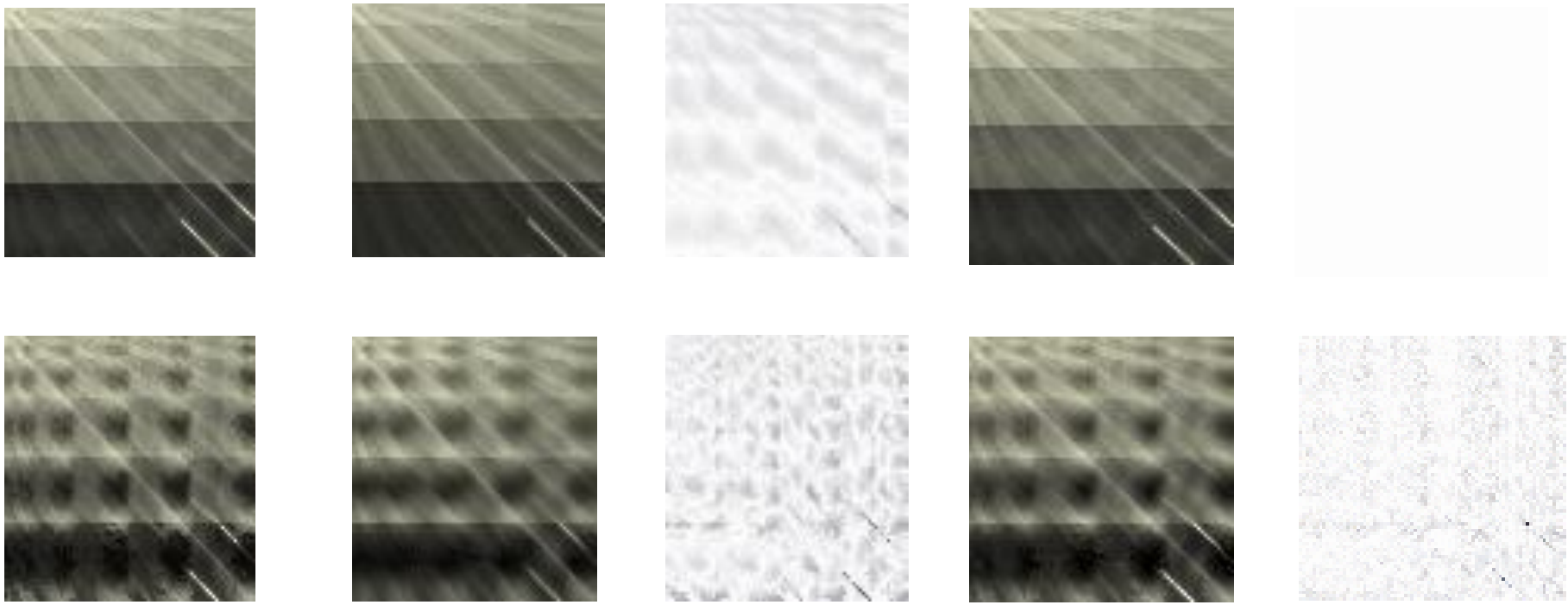
Block-wise factorization

- ► Regular partitioning
  - Per-View-Factorization, Sattler et al. (2003)
- ► Adaptive partitioning
  - Per Cluster Factorization, Müller et al. (2003)
- ► Two-pass Factorization
- ► Empirical evaluation (PhD Gero Müller, 2009)
  - Much faster compression
  - Much more efficient decompression
  - Similar compression ratios as FMF
  - At high compression rates cluster borders might become visible



# Full BTF-Factorization

## Analysis result

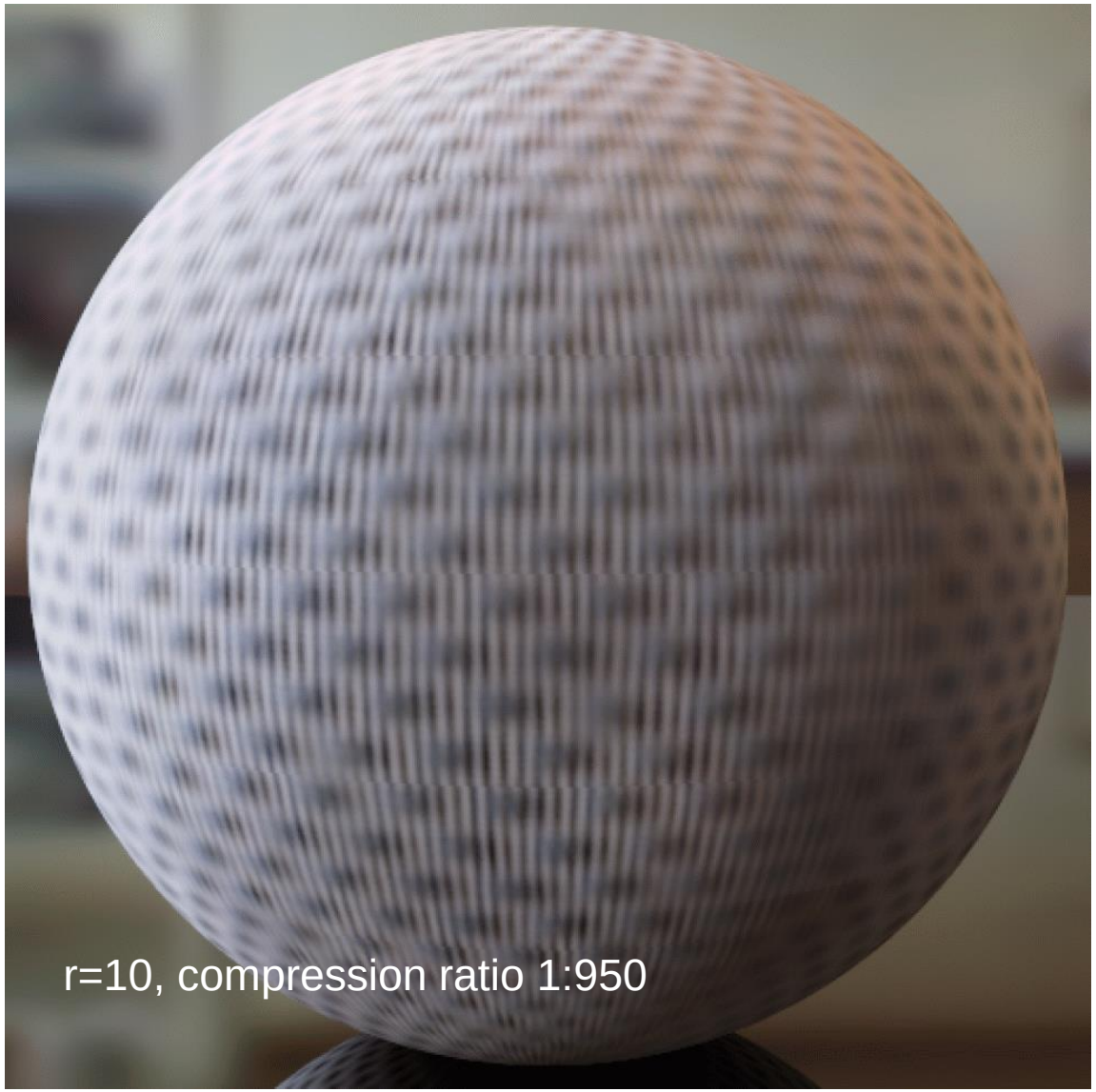


Original

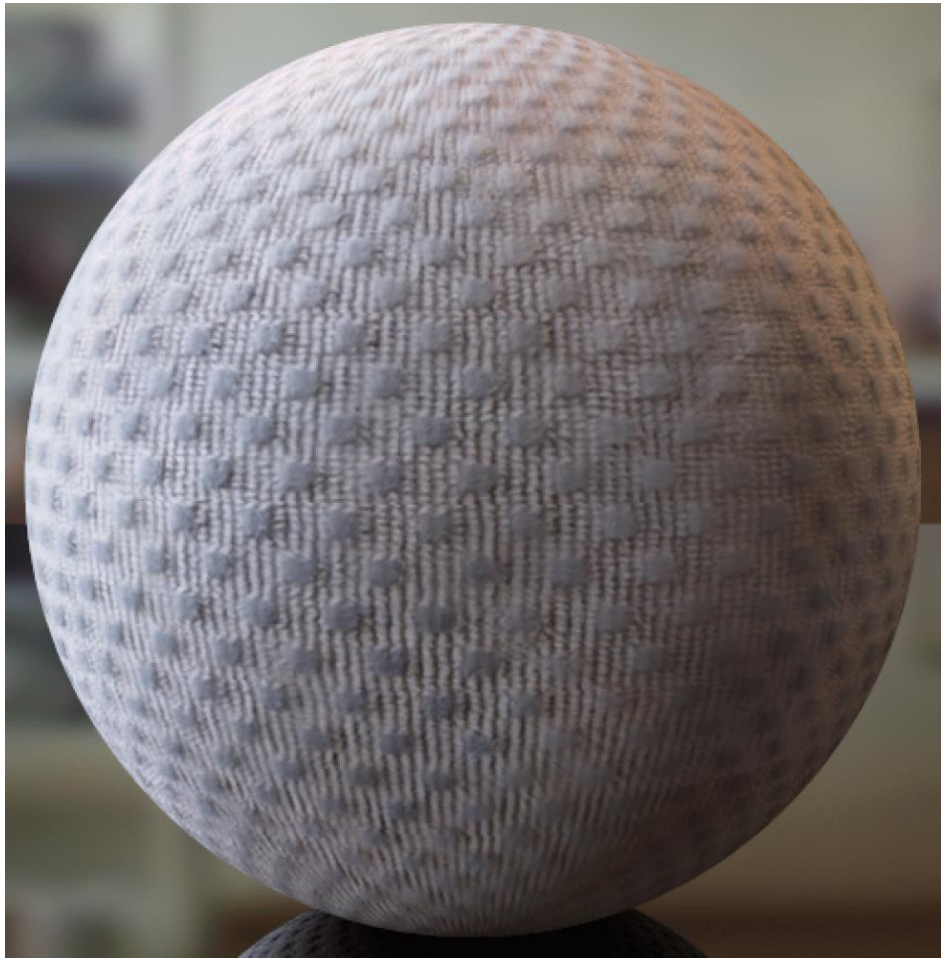
$c$=8

$c$=50

# Comparison (same reconstruction costs)

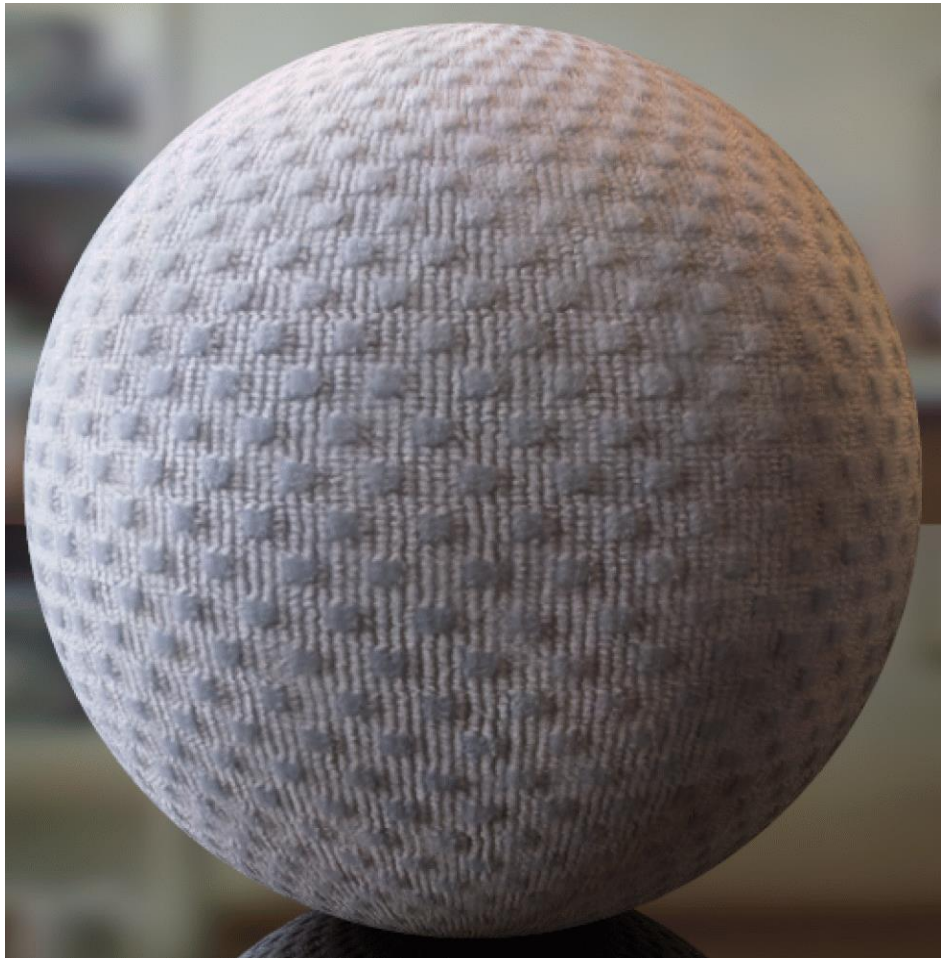


# Comparison (same reconstruction costs)



$r_b$=9, $p_{spatial}$=8, $p_{angular}$=12, compression ratio 1:110

# Comparison (same reconstruction costs)



reference

# Full Matrix Factorization

Good compression ratios

Efficient rendering

But: Compression is still slow

- Full-Matrix-Factorization of a BTF with $512 \times 512 \times 95 \times 95$ requires ~13h.
- Much  longer than 4h for the acquisition process

Factorization of very large matrices necessary

- Out-of-Core Algorithms required
- IO expensive, many passes should be avoided

# Fast SVD

Full Factorization of a $m \times n$ matrix requires $O(mn^2+m^2n+n^3)$

For compression only the largest $k$ singular values and corresponding vectors are needed.

Can be calculated in $O(mnk)$:

- **Incremental SVD**
  - Matthew Brand , „Incremental singular value decomposition of uncertain data with missing values“ (2002)
- **Fast PCA**
  - Liu et al. „ “Fast Principal Component Analysis using Eigenspace Merging” (2007)
- **EM-PCA**
  - Sam Roweis, „EM algorithms for PCA and SPCA“ (1998)

# Parallelized Matrix Factorization

*O(mnk)* still very big (e.g. 262.144 * 68.403 * 100)

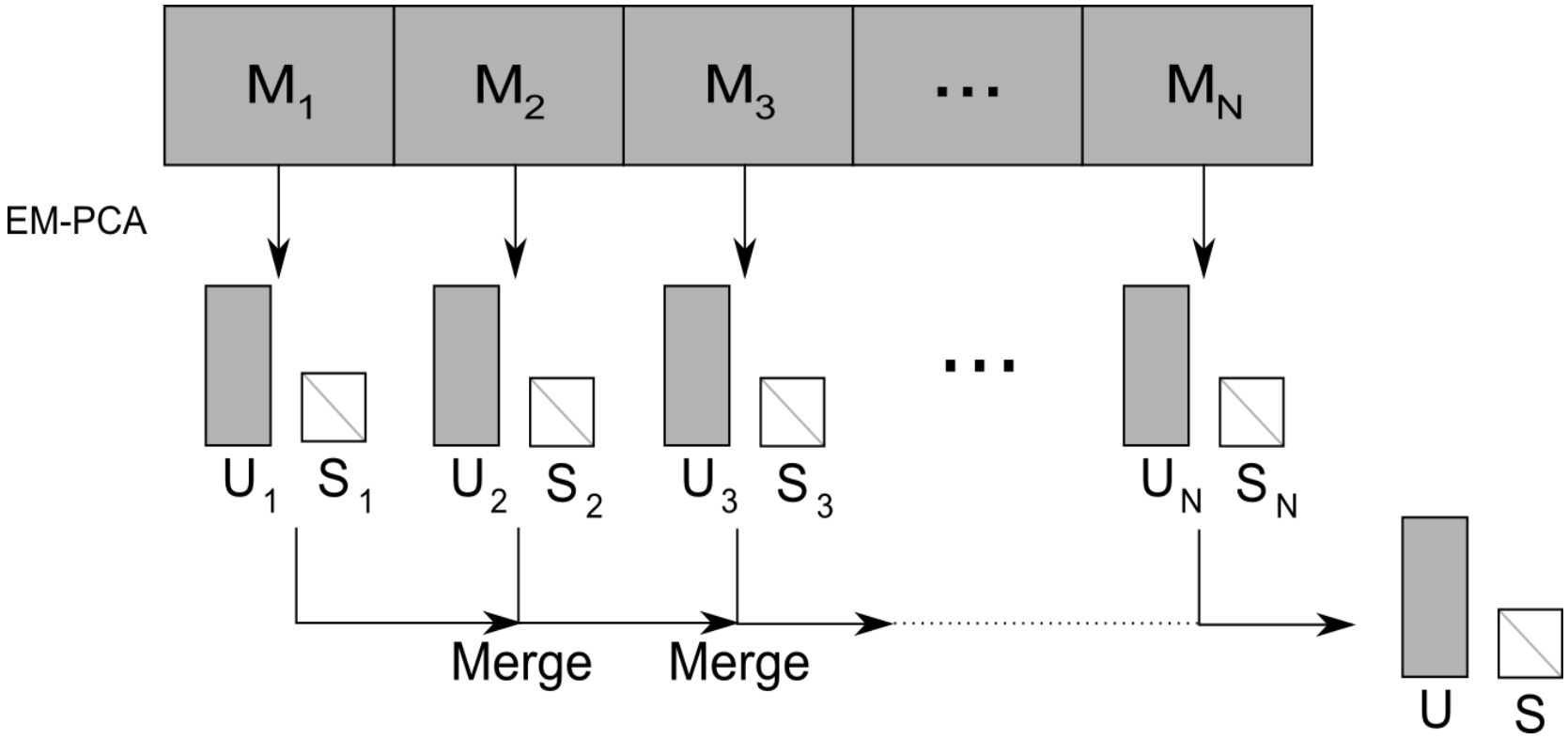
- ► GPU Implementation desirable*

**Idea**

- ► Perform as much processing as possible in-core
  - Reduces amount of IO
- ► Use algorithms which are mainly based on basic linear algebra operation
  - Very efficient GPU implementations available
- ► Perform a few more complex operations on the CPU, but only on small matrices

Ruiters, Roland, Martin Rump, and Reinhard Klein. "Parallelized Matrix Factorization for fast BTF Compression." In *EGPGV*, pp. 25-32. 2009.

# Algorithm Overview




Split Input Matrix into Blocks

Calculate EM-PCA independently for each block

Merge the Blocks to obtain the final result

# Algorithm Overview

**Input** matrix **M**

**Output** mean **m**

matrices **U**,**S**,**V**

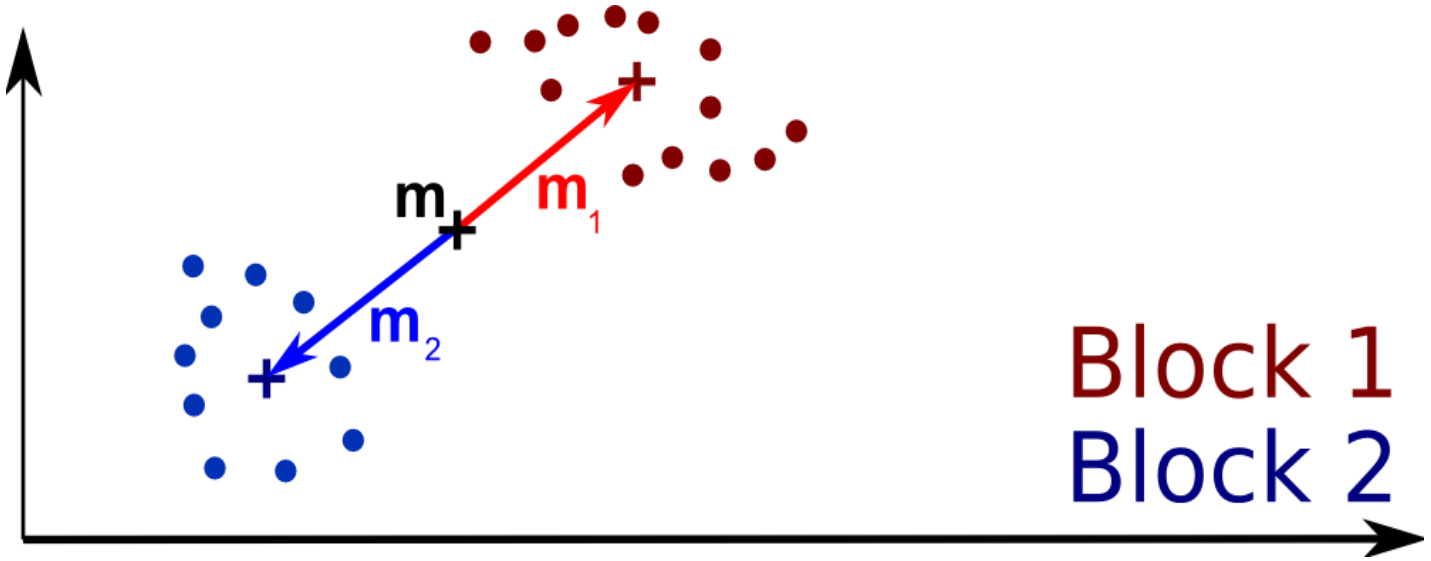
**Algorithm**

Subtract mean **m** from input matrix **M**
Calculate the first *k* columns of **U**
Project **M** onto subspace spanned by the columns of **U** to get **S** and **V**

We first calculate only **U**

- ► Avoids accumulation of errors during the merge steps
- ► Not necessary to update **V** in each step

# Block Mean




EM-PCA requires zero mean input

Individual $\mathbf{M}_i$ blocks can still have non-zero mean

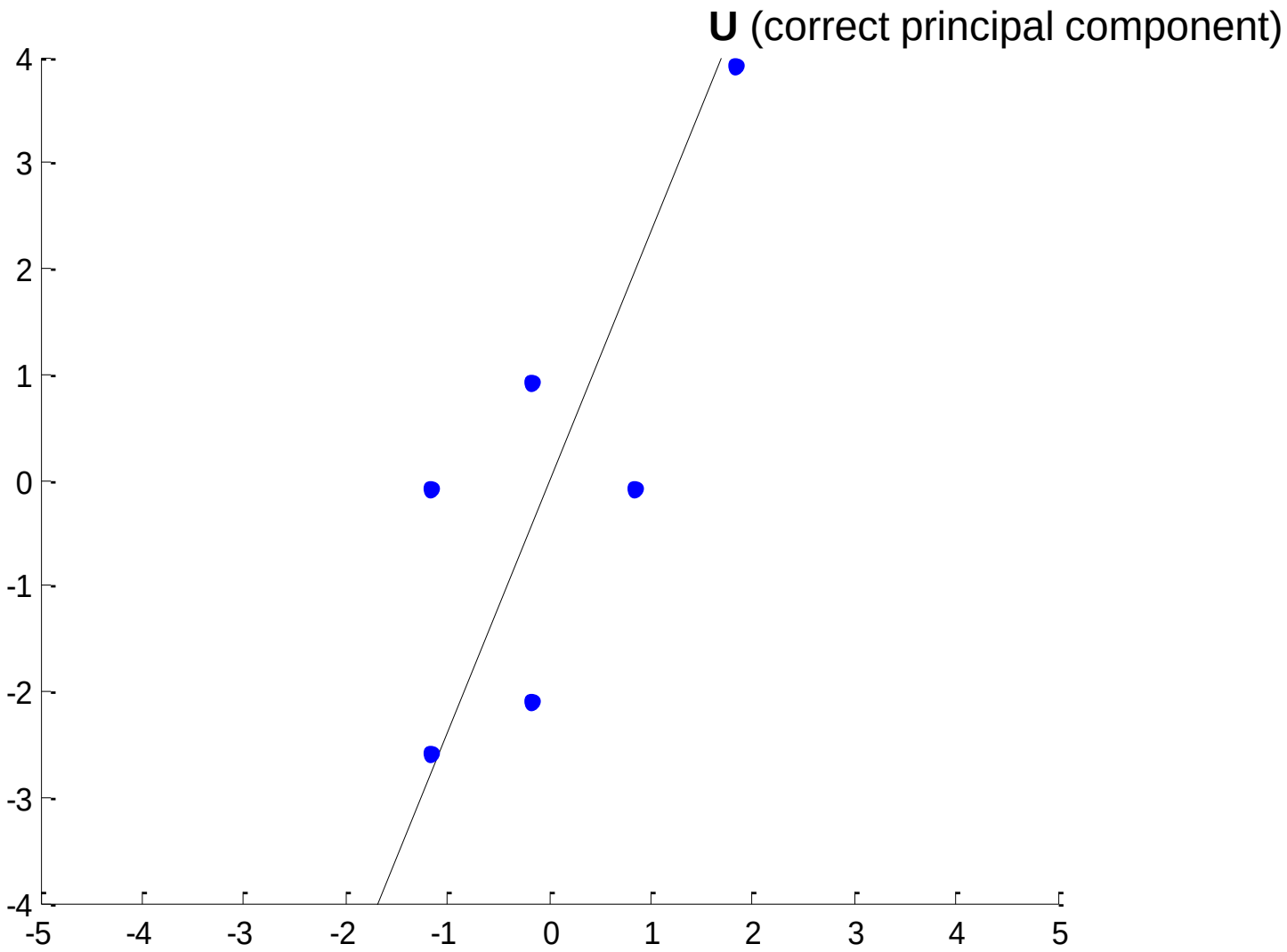
- ► Even though mean **m** has been subtracted from **M**

⟶Subtract $\mathbf{m}_i$ from individual blocks prior to EM-PCA

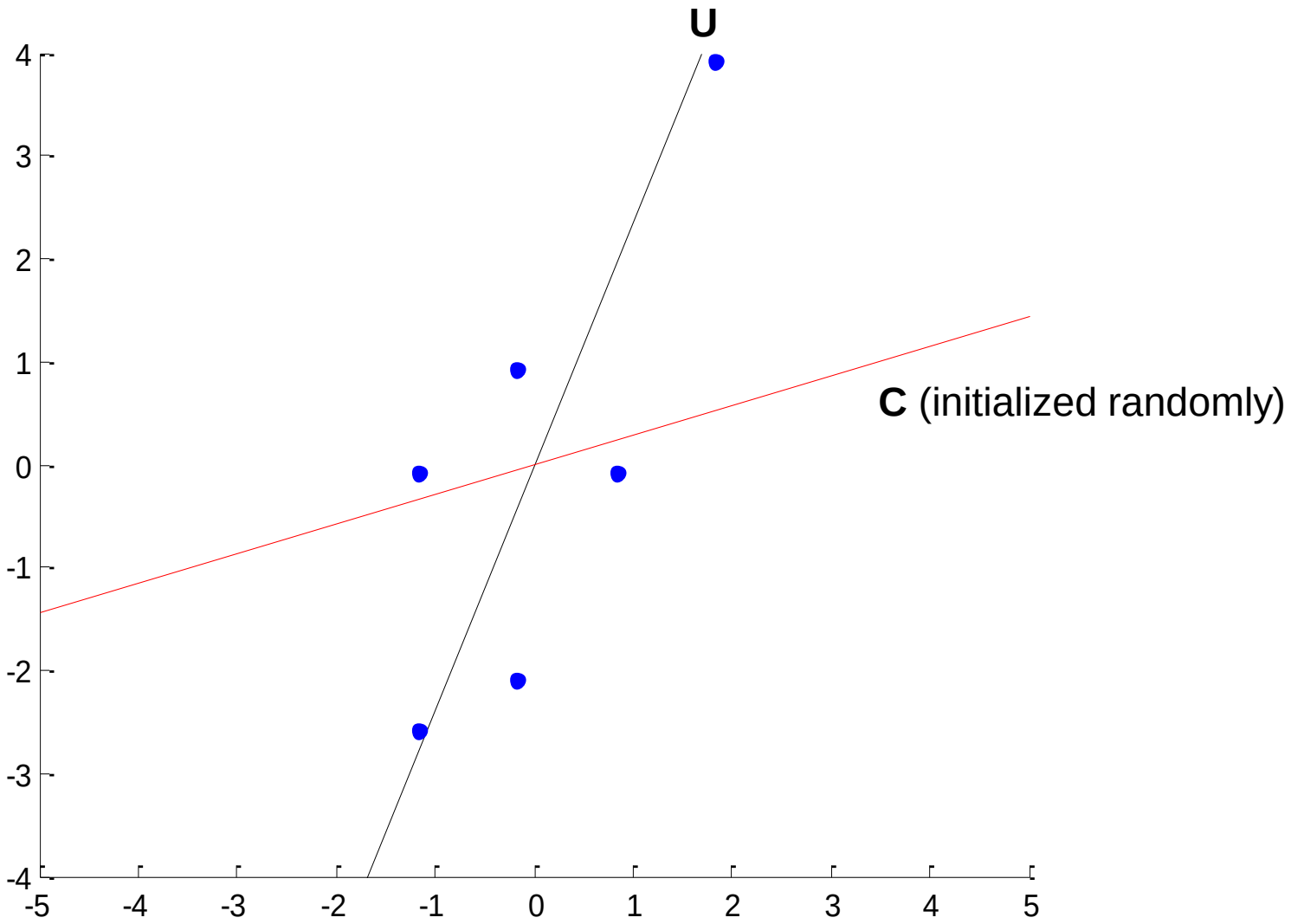
⟶Add coulmn $\mathbf{m}_i$ to **C** for final SVD calculation

→ Otherwise the component of the mean direction orthogonal to **C** would be lost

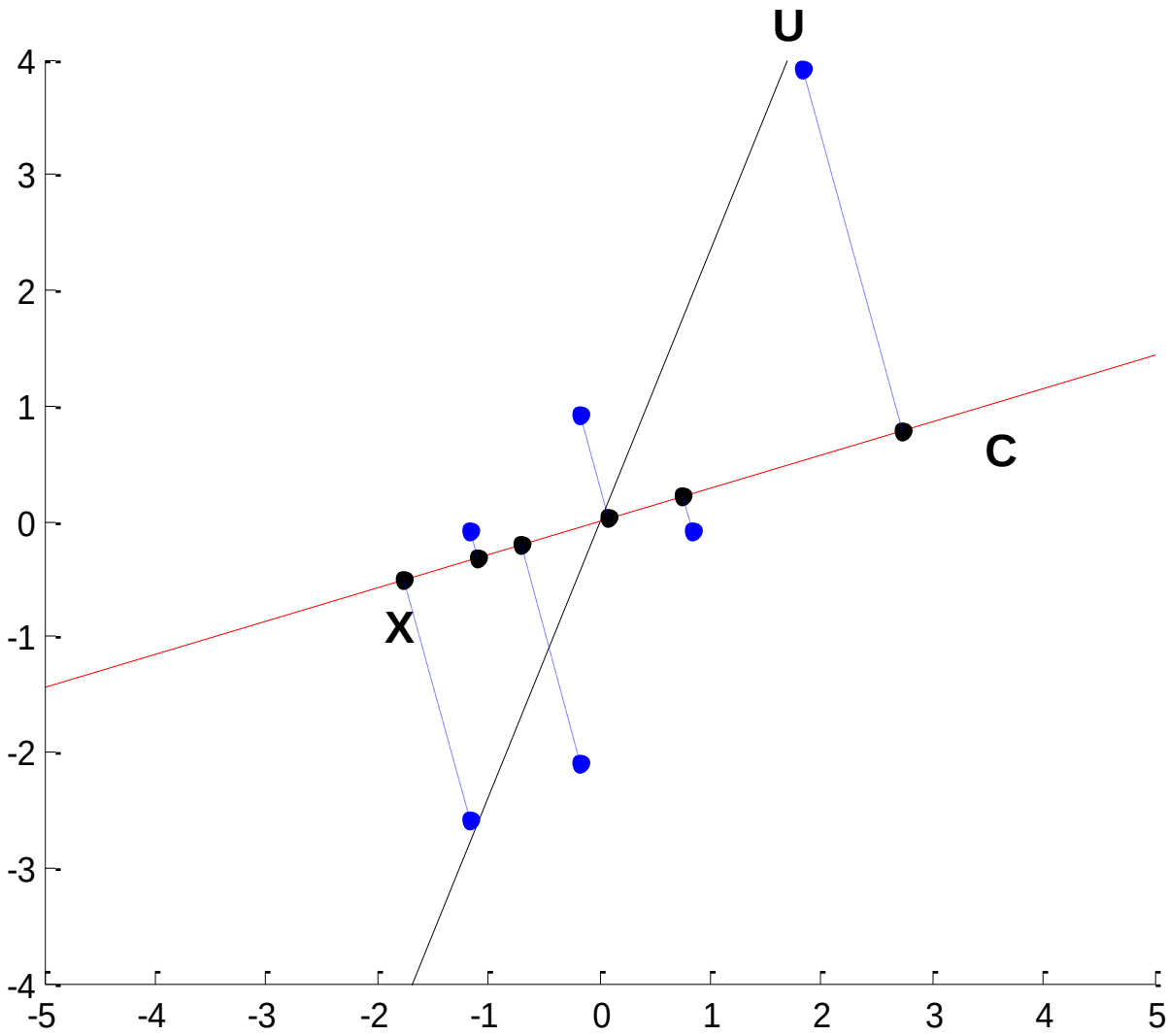
# Principle of EM-PCA



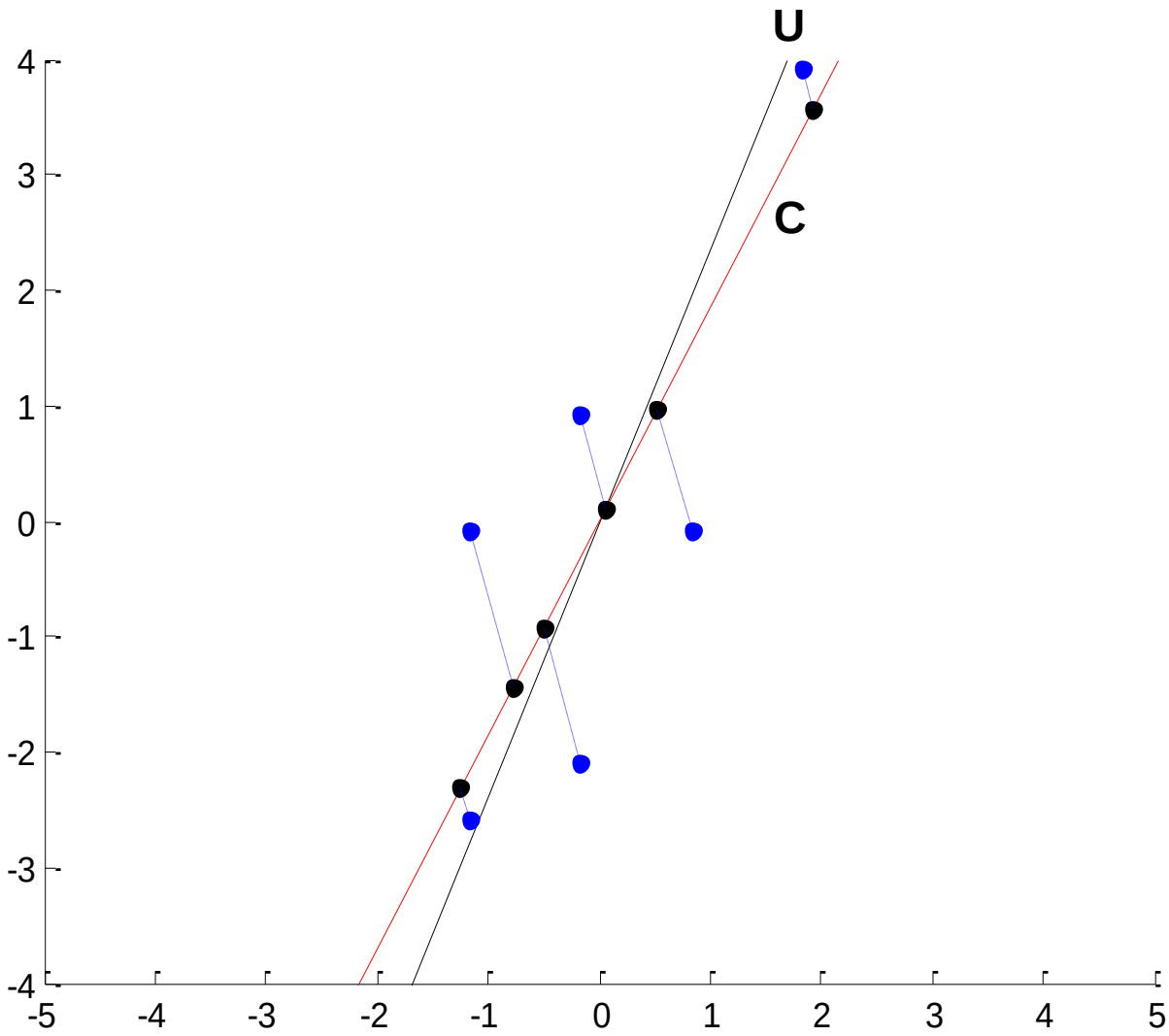
# Principle of EM-PCA



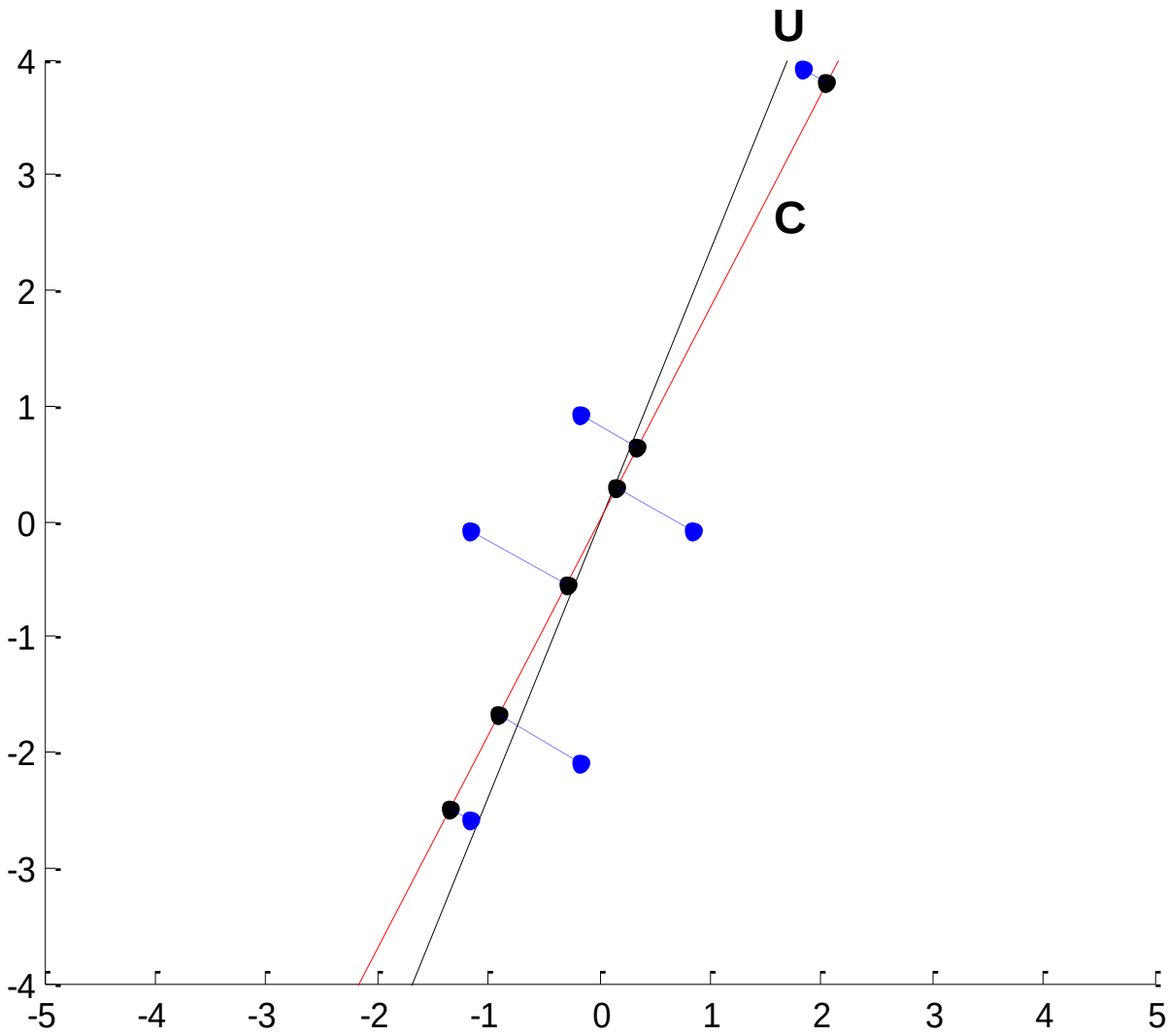
# Principle of EM-PCA



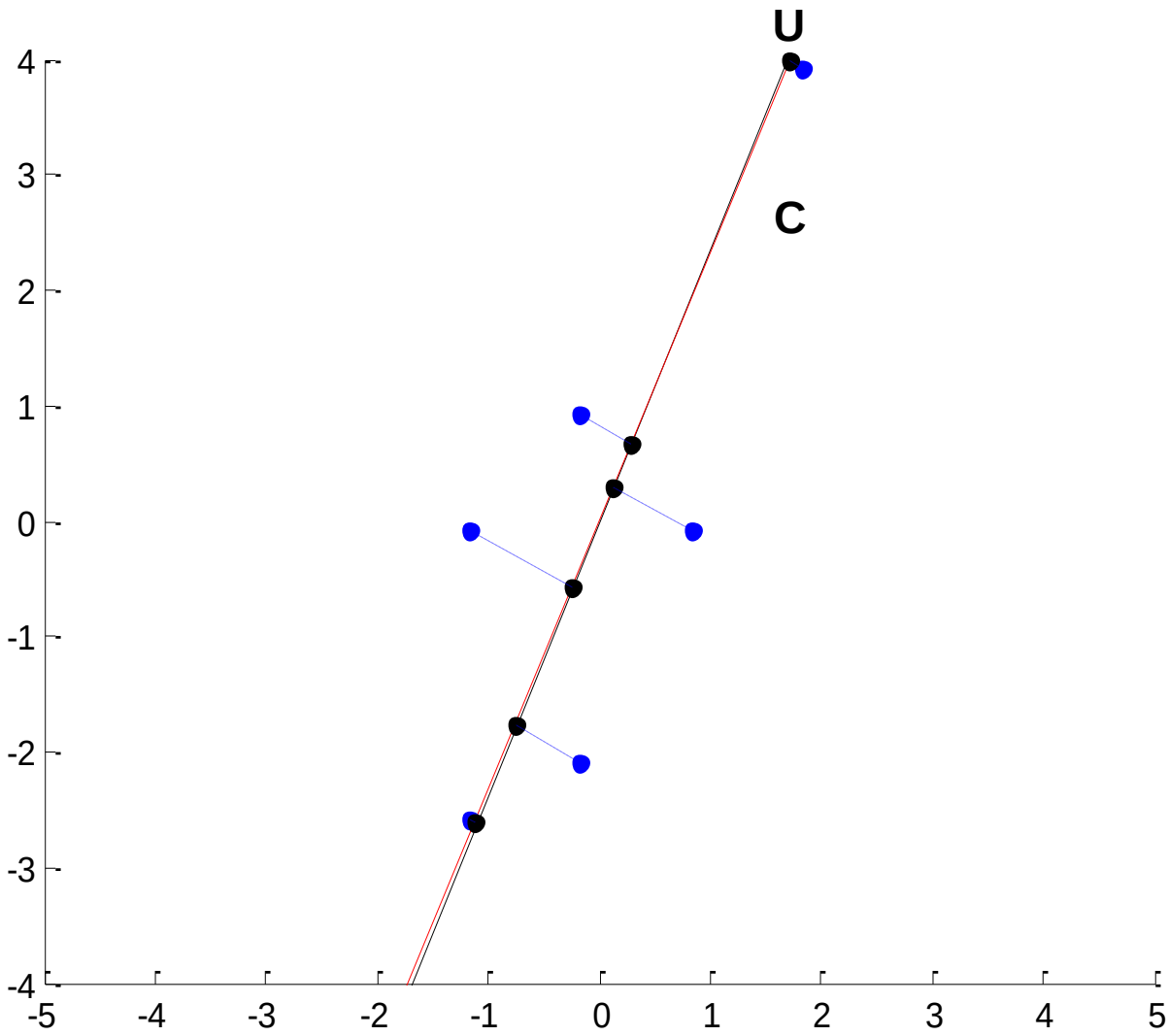
# Principle of EM-PCA



# Principle of EM-PCA



# Principle of EM-PCA



# EM-PCA

Expectation Maximization Algorithm to calculate the PCA of a matrix

Iterates between two steps:

- For a fixed subspace **C**:
  Find the best representation **X** of the input sample matrix $\mathbf{M}_i$

$$\mathbf{X} = (\mathbf{C}^T\mathbf{C})^{-1}\mathbf{C}^T\mathbf{M}_i$$

- For a fixed representation **X:**
  Find the **C** which gives the best reconstruction of $\mathbf{M}_i$

$$\mathbf{C}^{new} = \mathbf{M}_i\mathbf{X}^T(\mathbf{X}\mathbf{X}^T)^{-1}$$

# EM-PCA

**E-Step:** $\mathbf{X} = (\mathbf{C}^T\mathbf{C})^{-1}\mathbf{C}^T\overline{\mathbf{M}}_i$

**M-Step:** $\mathbf{C}^{new} = \overline{\mathbf{M}}_i\mathbf{X}^T(\mathbf{X}\mathbf{X}^T)^{-1}$

Converges after a small number of iterations

- Independent from the size of the input matrices.

Does only find the principal subspace, not the orientation of the principal vectors. Thus we have to:

- Orthogonalize **C** to get $\mathbf{C_o}$
- Project $\mathbf{M_i}$ into $\mathbf{C}_o$: $\mathbf{P} = \mathbf{C}_o^T\mathbf{M_i}$
- Calculate SVD of **P** and reverse the projection

Note: Block mean can be different from dataset mean

- We add a further column to **C** with the block mean

# EM-PCA (Run-Time)

**E-Step:** $$\mathbf{X} = (\mathbf{C}^T\mathbf{C})^{-1}\mathbf{C}^T\overline{\mathbf{M}}_i$$

**M-Step:** $$\mathbf{C}^{new} = \overline{\mathbf{M}}_i\mathbf{X}^T(\mathbf{X}\mathbf{X}^T)^{-1}$$

Run-Time dominated by the two multiplications with the large matrix **M**.

- These can be performed efficiently on the GPU
- Linear in input size and *k*: *O(mnk)*

Remaining operations have only small contribution to runtime

- Matrix inversions are only on matrices of size $k \times k$
- Final SVD calculation is performed on matrix with *k+1* columns

# Merge Step

**Given:**

SVD of two blocks: $\mathbf{M}_1 \approx \mathbf{U}_1\mathbf{S}_1\mathbf{V}_1^T$ and $\mathbf{M}_2 \approx \mathbf{U}_2\mathbf{S}_2\mathbf{V}_2^T$

**U,V** orthonormal, **S** diagonal matrix

$\mathbf{U}_1$ and $\mathbf{U}_2$ have $k_1$ and $k_2$ columns

**Searched:**

SVD of the joined matrix**:**

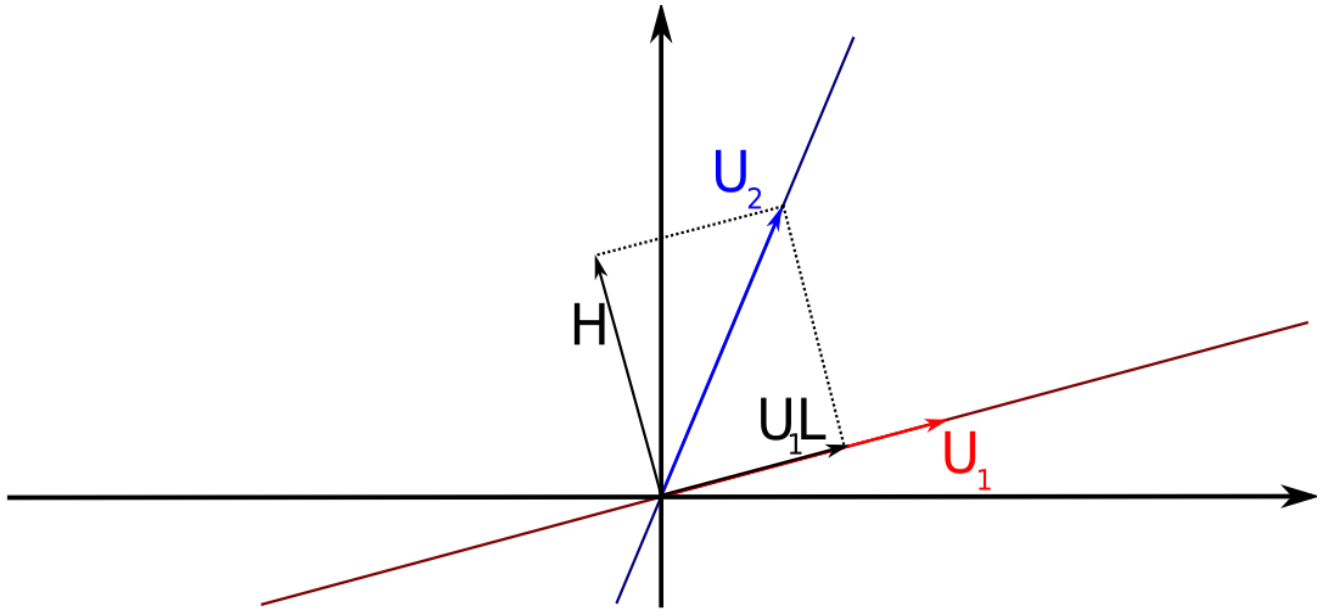
$$\mathbf{M} = [\mathbf{U}_1\mathbf{S}_1\mathbf{V}_1^T \quad \mathbf{U}_2\mathbf{S}_2\mathbf{V}_2^T] \approx [\mathbf{M}_1 \quad \mathbf{M}_2]$$

Calculating the joined matrix **M** impractical

Several techniques for this exist (eg. Hall et al. 2000)

- Our approach is similar to the **Incremental SVD** described by Matthew Brand

# Combined Orthogonal Basis




Project **U**$_2$ into **U**$_1$: $\mathbf{L} = \mathbf{U}_1^T \mathbf{U}_2$

Determine the orthogonal space **H**: $\mathbf{H} = \mathbf{U}_2 - \mathbf{U}_1\mathbf{L}$

Find orthogonal basis for **H**: $\mathbf{QR} = \mathbf{H}$

New orthogonal basis for **M**: $\mathbf{U}' = [\mathbf{U}_1\ \mathbf{Q}]$

# SVD Merging

$$\mathbf{M} = \mathbf{U}'\mathbf{U}'^{T}\mathbf{M}$$

$$= \begin{bmatrix}\mathbf{U_1} & \mathbf{Q}\end{bmatrix} \begin{bmatrix}\mathbf{U_1}^T \\ \mathbf{Q}^T\end{bmatrix} \begin{bmatrix}\mathbf{U}_1\mathbf{S}_1\mathbf{V}_1^T & \mathbf{U}_2\mathbf{S}_2\mathbf{V}_2^T\end{bmatrix}$$

$$= \begin{bmatrix}\mathbf{U_1} & \mathbf{Q}\end{bmatrix} \begin{bmatrix}\mathbf{U}_1^T\mathbf{U}_1\mathbf{S}_1 & \mathbf{U}_1^T\mathbf{U}_2\mathbf{S}_2 \\ \mathbf{Q}^T\mathbf{U}_1\mathbf{S}_1 & \mathbf{Q}^T\mathbf{U}_2\mathbf{S}_2^T\end{bmatrix} \begin{bmatrix}\mathbf{V}_1 & 0 \\ 0 & \mathbf{V}_2\end{bmatrix}^T$$

$$= \begin{bmatrix}\mathbf{U_1} & \mathbf{Q}\end{bmatrix} \begin{matrix}k_1 \\ k_2\end{matrix}\!\!\begin{bmatrix}\mathbf{S}_1 & \mathbf{L}\mathbf{S}_2 \\ 0 & \mathbf{R}\mathbf{S}_2\end{bmatrix} \begin{bmatrix}\mathbf{V}_1 & 0 \\ 0 & \mathbf{V}_2\end{bmatrix}^T$$

($k_1$ under the first column, $k_2$ under the second column)

SVD

$$= \underbrace{\begin{bmatrix}\mathbf{U_1} & \mathbf{Q}\end{bmatrix} \mathbf{U}''}_{\mathbf{U}} \quad \mathbf{S}'' \quad \underbrace{\mathbf{V}'' \begin{bmatrix}\mathbf{V}_1 & 0 \\ 0 & \mathbf{V}_2\end{bmatrix}^T}_{\mathbf{V}}$$

$$= \mathbf{U}\,\mathbf{S}\,\mathbf{V}$$

$$\mathbf{L} = \mathbf{U}_1^T\mathbf{U}_2$$

$$\mathbf{R} = \mathbf{Q}^T\mathbf{H}$$

$$\mathbf{U}' = \begin{bmatrix}\mathbf{U}_1 & \mathbf{Q}\end{bmatrix}$$

# SVD Merging

V not needed for the merging operation

- Can be perfromed only with **U** and **S**

After merging, the matrix has to be truncated

- We keep $2k$ singular values to reduce error caused by truncation

SVD has only to be calculated on central matrix

- Only $3k \times 3k$ matrix

# Implementation (GPU Processing)

Most Operations can be accelerated on the GPU:

- Matrix Multiplication
- Mean Calculation
- Mean Subtraction
- Orthogonalization

► Easy implementation with the NVIDIA CUBLAS™ Library

Not on the GPU:

- SVD
- Matrix inversions

► But only on small matrices

- not dominant for run-time

# Implementation (IO)

IO can be performed asynchonously with calculation

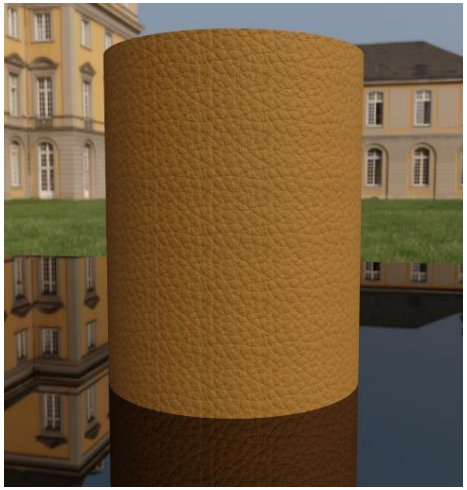
- Additional thread pre-loads the next block, while the current one is processed
- Eliminates most of the IO overhead during the calculation
  - Block IO and calculation require about the same time on our hardware

Mean calculation and final projection require two further IO passes

- On 26GB dataset:
  - 700s for one IO pass
  - 2398s total runtime
- Requires more than half of the total run-time

Block size should be chosen in dependence on available GPU memory

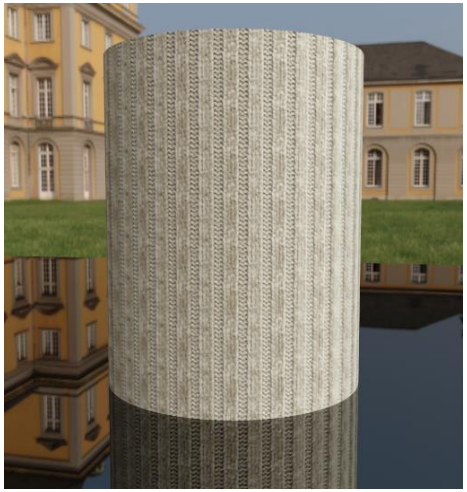
# Sample BTFs



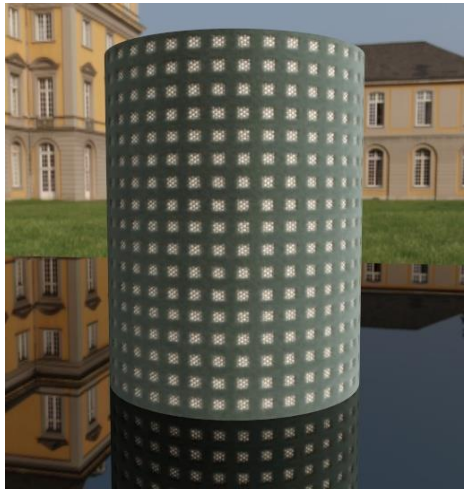
Leather1

Leather2

Leather3



Pulli



Fabric

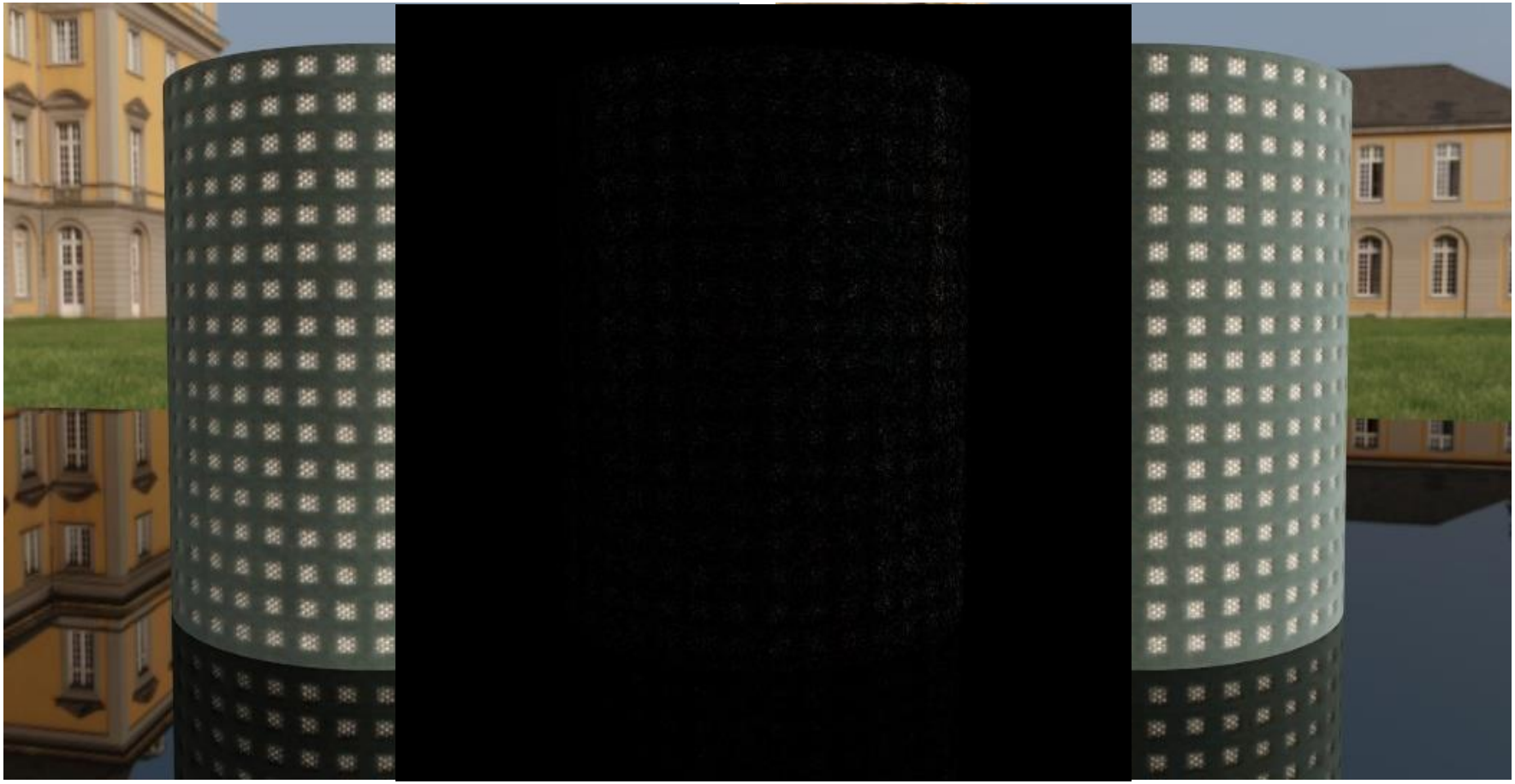
# Results

Comparison to a purely CPU based out-of-core EM-PCA implementation:

| **Material** | **Resolution** | **Size** | **Time** | **Speedup** | **Relative error increase** |
|---|---|---|---|---|---|
| Leather1 | $256^2 \times 95^2$ | 6.6 GB | 317 s | 36.78 | 0.068% |
| Leather1 | $512^2 \times 95^2$ | 26.4 GB | 2398 s | 19.61 | 0.030% |
| Leather2 | $128^2 \times 151^2$ | 4.2 GB | 280 s | 27.54 | 0.054% |
| Leather3 | $256^2 \times 81^2$ | 4.8 GB | 261 s | 31.07 | 0.021% |
| Pulli | $256^2 \times 81^2$ | 4.8 GB | 266 s | 30.56 | 0.045% |
| Fabric | $256^2 \times 81^2$ | 4.8 GB | 223 s | 36.53 | 0.108% |

Q6600 CPU, Geforce 8800 GTX, 8GB RAM

# Comparison

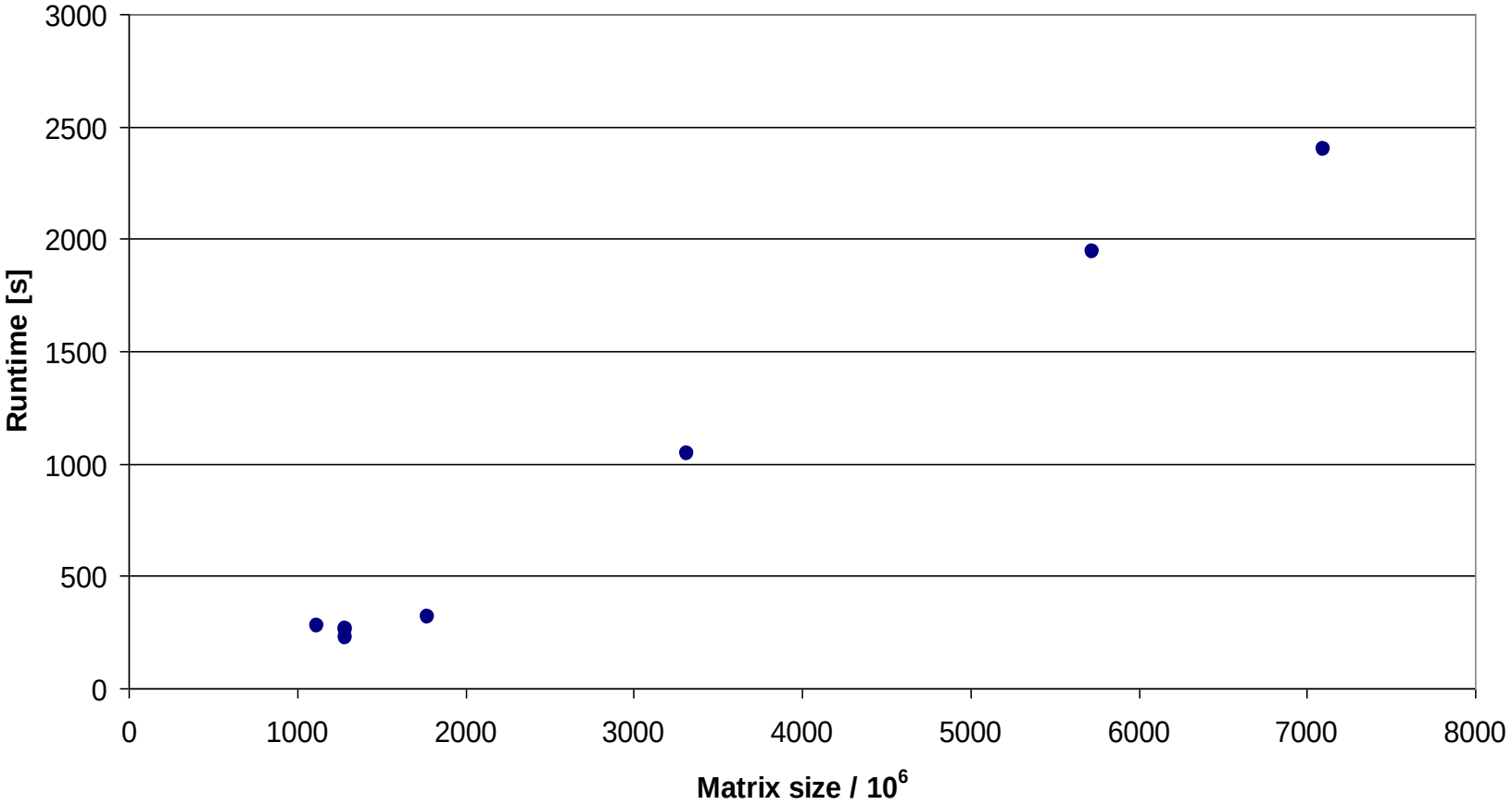


EM-PCA (CPU)

Difference Image (Scaled x10)

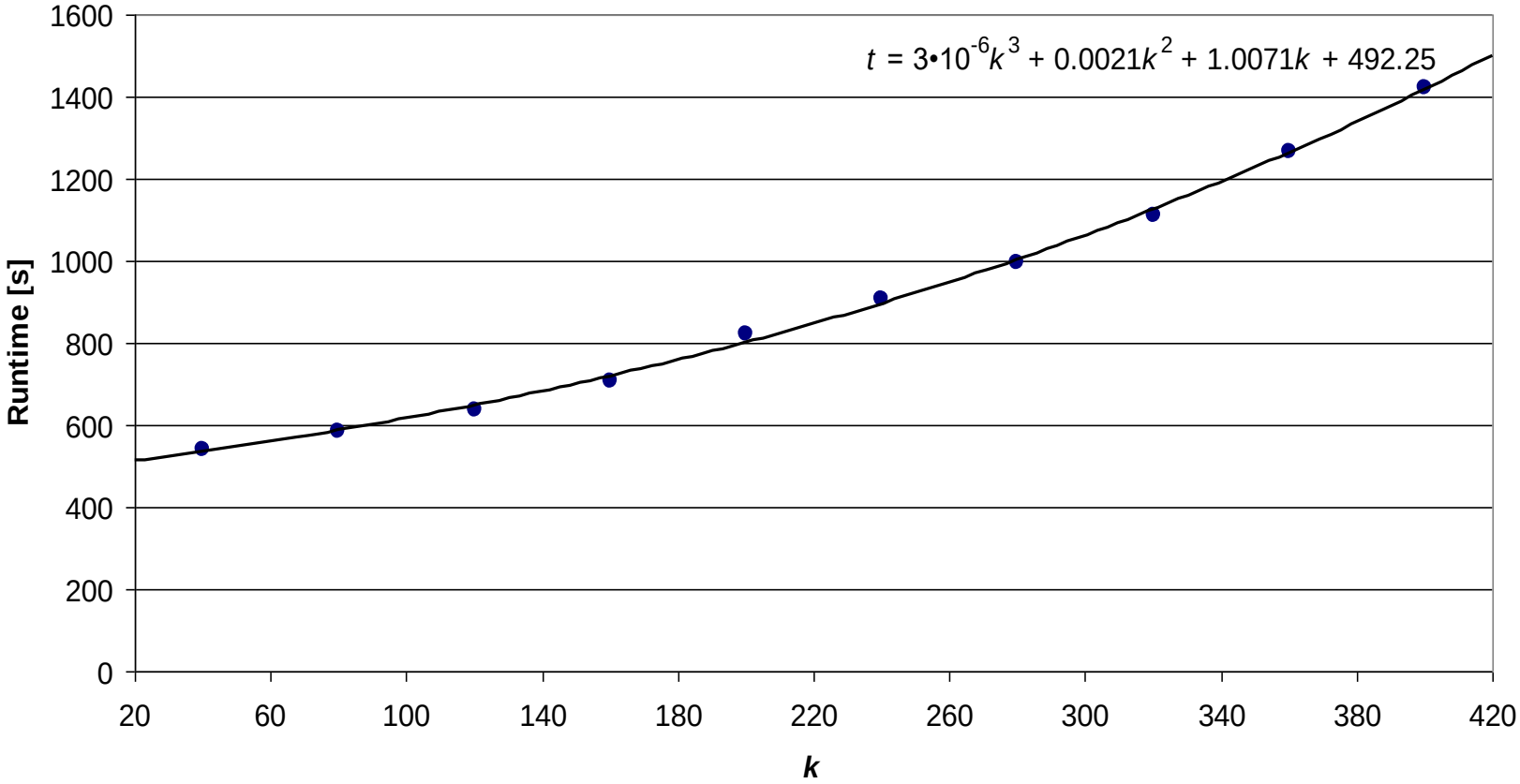
Our Algorithm (GPU)

# Run Time (Matrix Size *mn*)




- Runtime linear in matrix size for big matrices

# Run Time (Singular Values *k*)




Small contribution of non-linear part

→Matrix multiplications and IO dominate the total run-time

# Conclusion on parallel EM-PCA

Out-of-Core factorization of large matrices:

- Independent factorization of blocks using EM-PCA
- Merging of the resulting factorizations

Speedup by a factor of 20-36

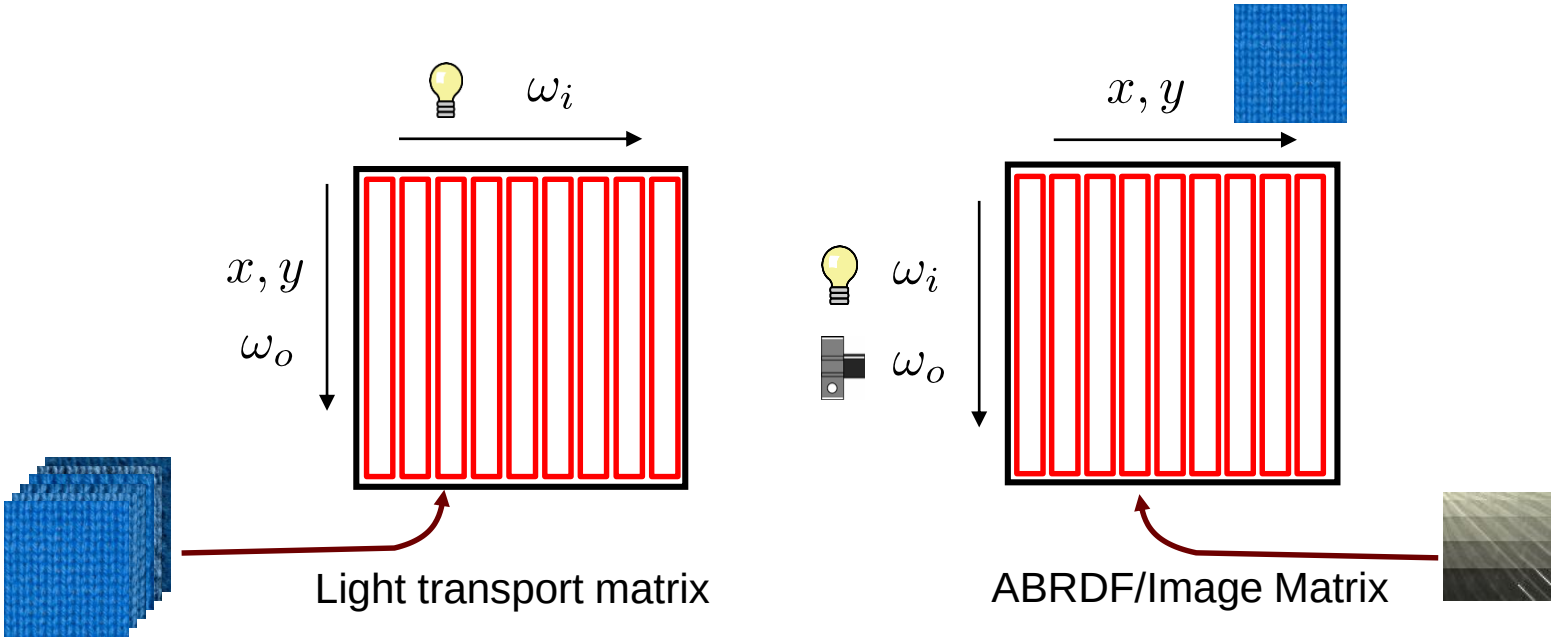
Only minimal increase in reconstruction error

- Below 0.11% in our experiments

BTF datasets with high spatial and angular resolution can now be processed in reasonable time

# Sparse acquisition[1]

BTF is organized as matrix



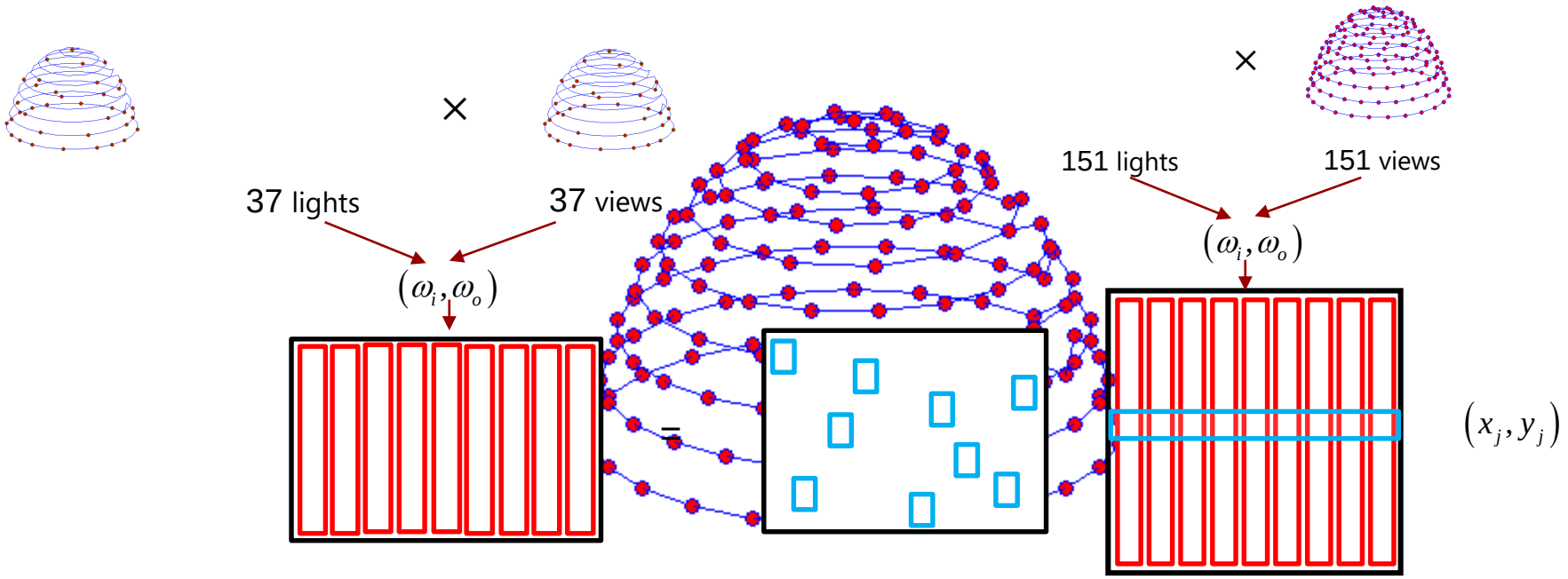

Is it necessary to measure and store the complete matrix?

Compressive sensing

1. Dennis den Brok, Michael Weinmann und Reinhard Klein, *Rapid material capture through sparse and multiplexed measurements,* In: Computers & Graphics (Juni 2018), 73(26-36)

# Sparse acquisition

The measurement matrix




$$\tilde{B} = M \bullet B$$

# Sparse acquisition

Reconstruction of B

- Consider database D of representative ABRDFs

$$\mathrm{D} \approx \mathrm{U}\Sigma\mathrm{V}^{\mathrm{t}}$$

- We expect that $B \approx UC_B, \qquad \tilde{B} \approx MUC_B$

- Then an approximation of $B$ might be derived from $MU$

$$C_B \approx \left(MU\right)^{\dagger} \tilde{B}$$

by

$$B \approx U\left(MU\right)^{\dagger} \tilde{B}$$

where $\left(MU\right)^{\dagger}$ denotes the Moore Penrose inverse of $MU$

- **Idea:** find *M* by optimizing the condition of $MU$

# Sparse acquisition

**Algorithm 1** Generation of a measurement matrix.

**Input:** desired number $n_s$ of samples

**Output:** optimized measurement matrix $\mathbf{M} \in \mathbf{R}^{n_s \times n}$

$\mathbf{M} \leftarrow$ random binary with exactly one 1 per row
**while** not converged **do**
  $\mathbf{M}' \leftarrow \mathbf{M}$
  $\mathbf{r} \leftarrow$ random binary row vector with $||\mathbf{r}||_0 = 1$
  random row of $M \leftarrow \mathbf{r}$
  **if** $\kappa(\mathbf{M}'\mathbf{U}) < \kappa(\mathbf{M}\mathbf{U})$ **then**
    $\mathbf{M} \leftarrow \mathbf{M}'$
  **end if**
**end while**
**return** $\mathbf{M}$

# Sparse acquisition

Reconstruction of B

- Unfortunately, simple reconstruction of B by

$$B \approx U \left( MU \right)^{\dagger} C_B$$

leads to severe overfitting.

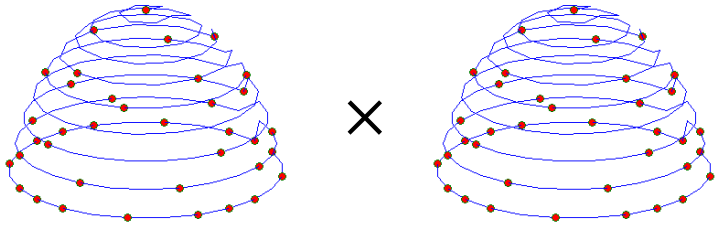
- Better results are achieved by using a Tikhonov regularization which leads to

$$C_B \approx \left( \left( MU \right)^{*} \left( MU \right) - \eta I \right) \left( MU \right)^{*} \tilde{B}$$

where $\eta$ is an appropriate weight.

# Experimental Setup

- Samplings:
  - "Kaleidoscope" sampling



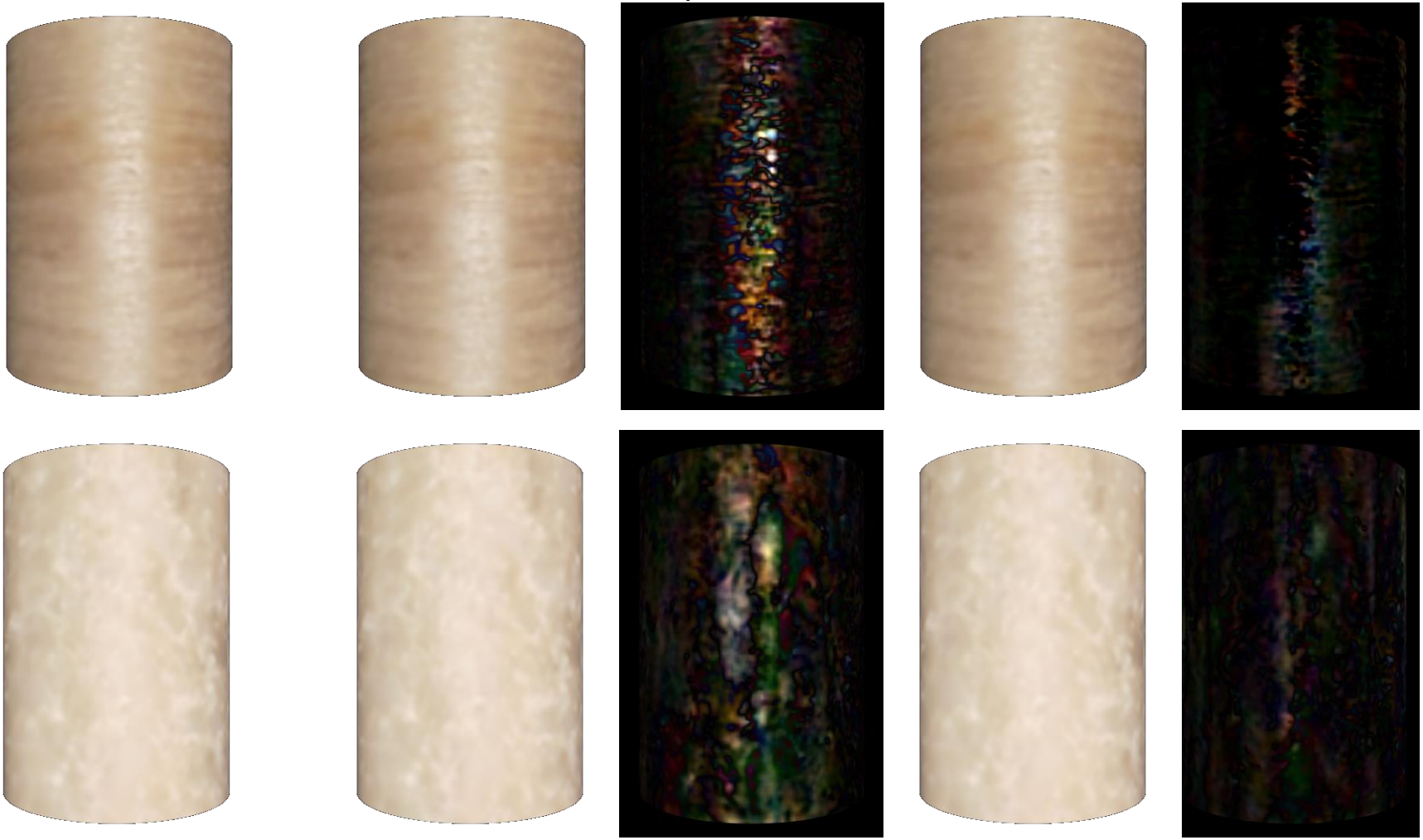

    - 1369 samples ≈ 6% of full sampling, 25% of full lights
  - Per-cluster optimized sampling with same #samples
- Representative ABRDFs by semantic class

# Results

Reference

Kaleidoscope

Optimized

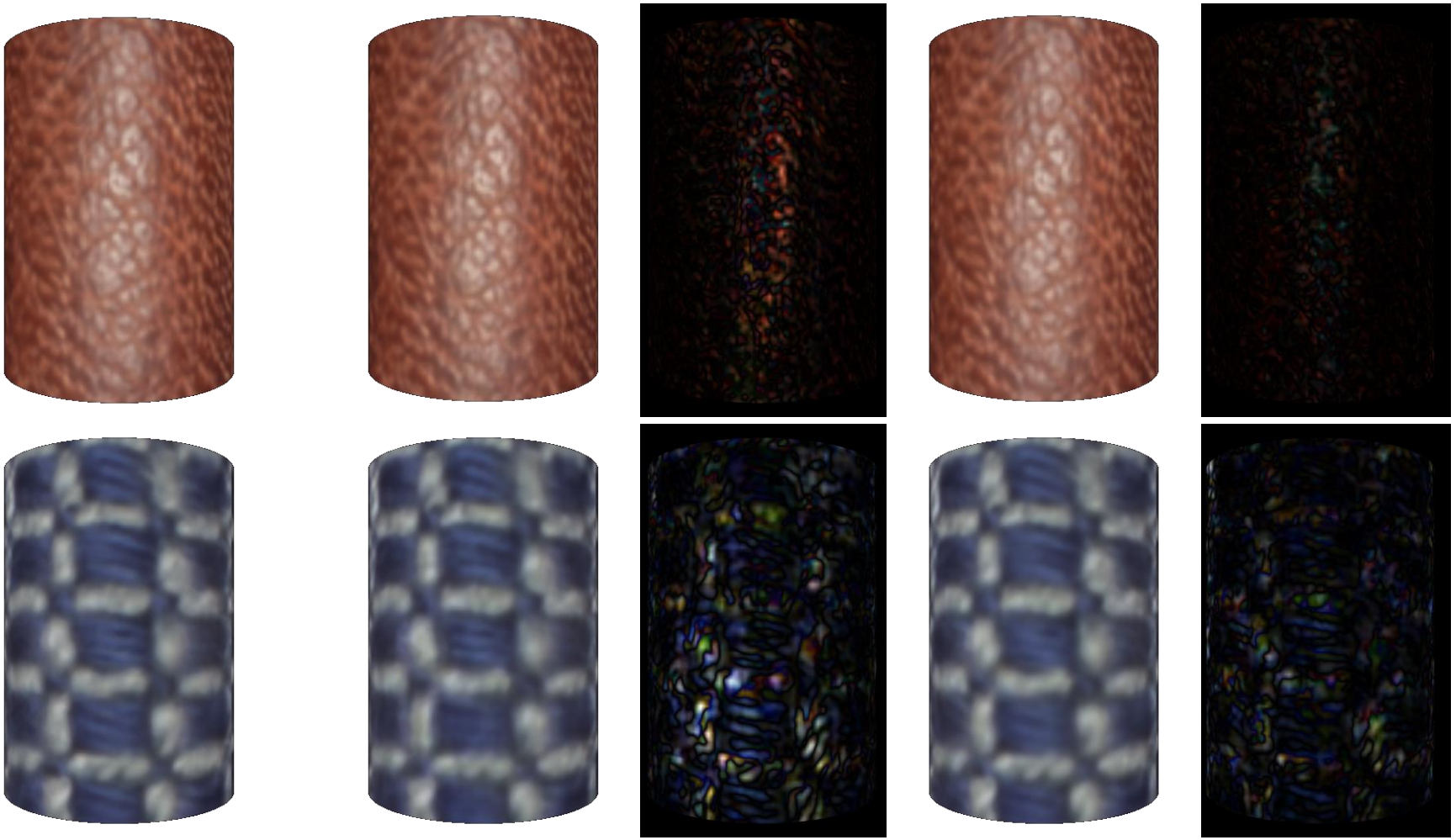


# Results (cont.)

Reference

Kaleidoscope

Optimized



# Dealing with high dynamic range

We reduced the data's dynamic range by

- converting the measured HDR RGB data to the color space

$$\left(\log(Y), U/Y, V/Y\right)$$

Nielsen et al*., suggest to use the even better mapping

$$\rho \mapsto \log\left(\frac{\rho \cos_{weight} + \varepsilon}{\rho \cos_{ref} + \varepsilon}\right)$$

- where $\cos_{ref}$ is the median BRDF and

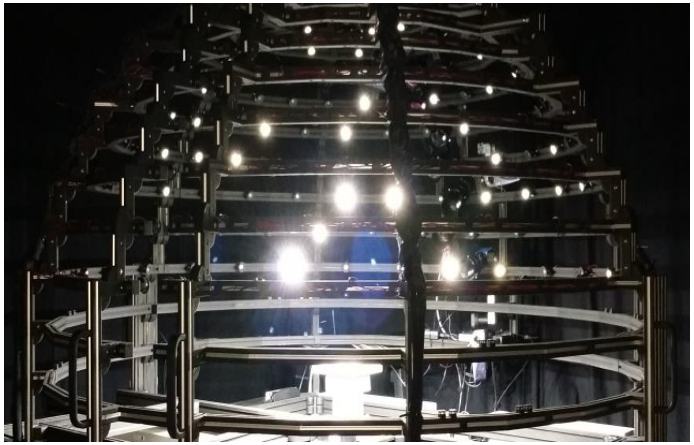
$$\cos_{weight} := \max\left(\cos\theta_i, \cos\theta_o, \varepsilon\right),\ \varepsilon = 0.0001$$

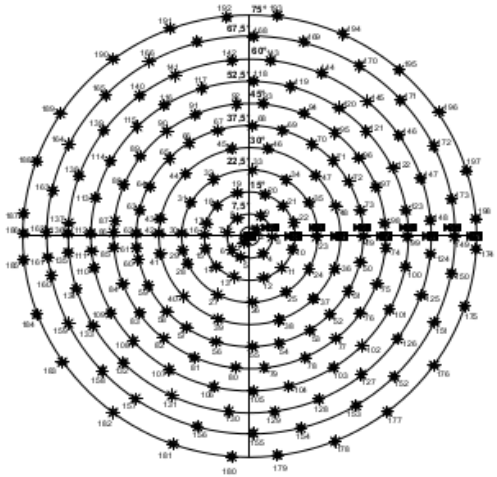
*Nielsen, JB, Jensen, HW, Ramamoorthi, R. On optimal, minimal BRDF sampling for reflectance acquisition. ACM Trans Graph 2015;34(6)

# Optimizing acquisition time

**(Den Brok et al., Multiplexed acquisition of bidirectional texture functions for materials, 2015)**

- **11** cameras (8 Hz), **198** LEDs, turntable (**24** rotations)





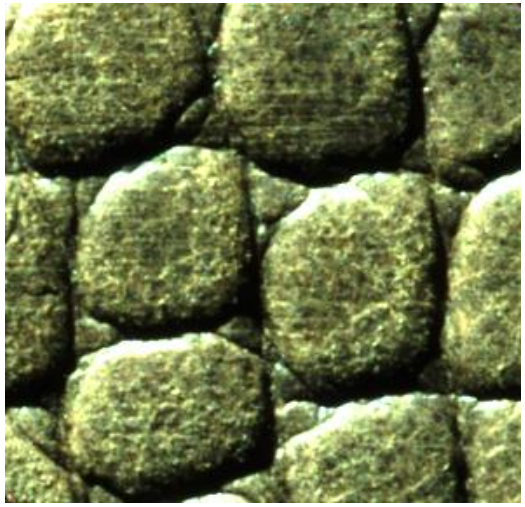
Typical acquisition time: **4 – 24 h**

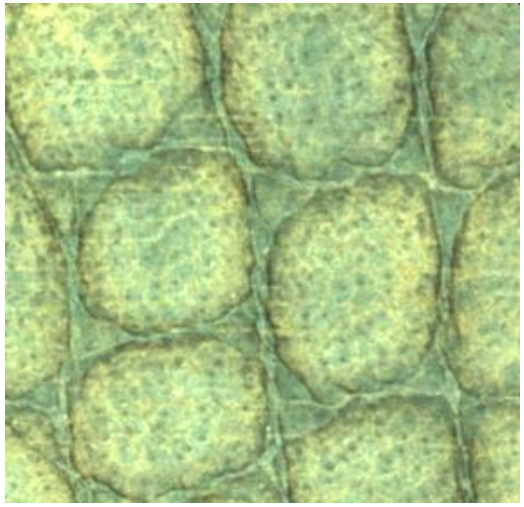
- Due to various bottle-necks (LEDs, camera frame-rate, etc.)

# Optimizing acquisition time

Brightness may vary greatly even in single images (shadows, highlights…)

Many and/or long exposure steps necessary



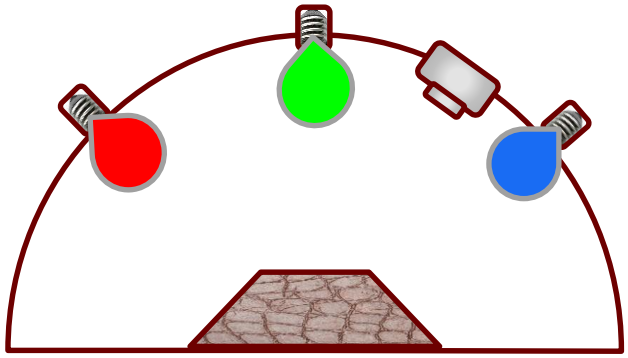


…much better with many lights at once!

Reduced acquisition time using multiplexed aquisition ≈ **1.2 -1,8 h**

# Multiplexed illumination

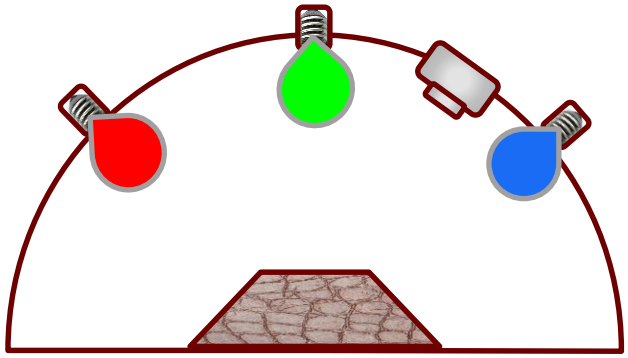
- Example (Light Transport Matrix)



$$\underbrace{\begin{pmatrix} l_{11} & l_{12} & l_{13} \end{pmatrix}}_{I_{single}}$$

# Multiplexed illumination

- Example (Light Transport Matrix)

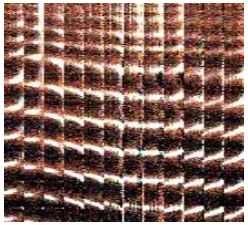


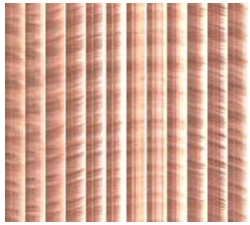
$$\underbrace{\begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}}_{S} \underbrace{\begin{pmatrix} l_{11} & l_{12} & l_{13} \\ l_{21} & l_{22} & l_{23} \\ l_{31} & l_{32} & l_{33} \end{pmatrix}}_{I_{single}} = \underbrace{\begin{pmatrix} l_{11}+l_{21} & l_{12}+l_{22} & l_{13}+l_{23} \\ l_{21}+l_{31} & l_{22}+l_{32} & l_{23}+l_{33} \\ l_{11}+l_{31} & l_{12}+l_{32} & l_{13}+l_{33} \end{pmatrix}}_{I_{multiplexed}}$$

# Multiplexed illumination

- Drastically reduced shutter times by using a series of illumination patterns

$$M \times \quad = \quad \left( M \times I_{single} = I_{multiplexed} \right)$$



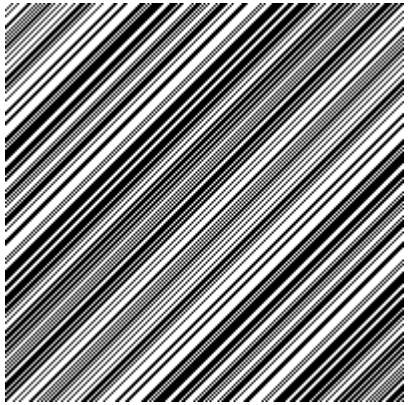


where $M \in$ $^{n \times n}$ is a binary matrix indicating if the corresponding light source is on / of, and

$I_{\text{single}} \in$ $^{n \times (w \times h)}$, $I_{\text{multiplexed}} \in$ $^{n \times (w \times h)}$ are the respective ABRDF/image matrices

# S-Matrices

**Question:** How to choose $S$?



Common choice: *Hadamard* matrices*

- binary variant: S-matrices

* A Hadamard matrixis a square matrix whose entries are either +1 or −1 and whose rows are mutually orthogonal.

Advantages:

- suitable choices for any $n_l = \#\text{LEDs}$
- $\approx$ maximum reduction of *signal-independent noise*
- $\approx \frac{n_l}{2}$ LEDs / pattern, equal spread → low dynamic range
  → less exposure steps for HDR capture

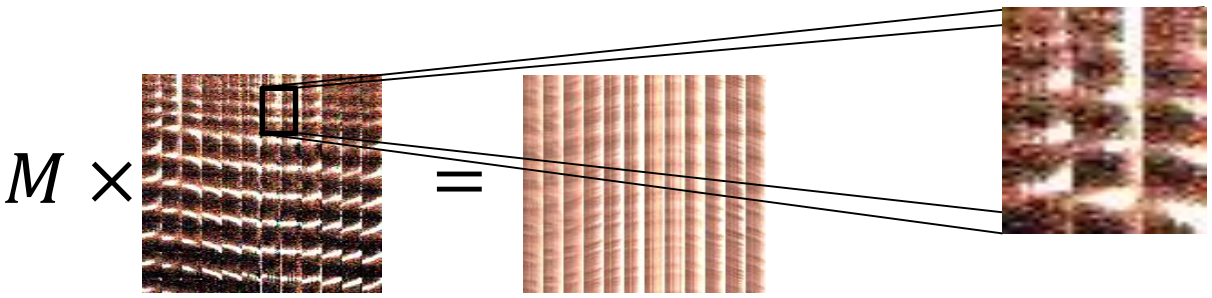
* Sloane, Neil JA, und Martin Harwit. „Masks for Hadamard transform optics, and weighing designs“. *Applied optics* 15, Nr. 1 (1976): 107–114
* Schechner, Yoav Y., Shree K. Nayar, und Peter N. Belhumeur. „Multiplexing for Optimal Lighting“. IEEE Transactions on Pattern Analysis and Machine Intelligence 29, Nr. 8 (August 2007): 1339–54.

# Multiplexed illumination

Disadvantage (of patterns in general):

- more light → increased *signal-dependent noise*

leads to noisy reconstructions:

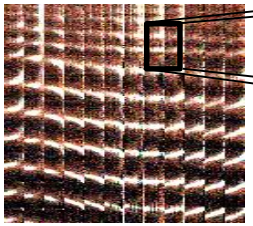
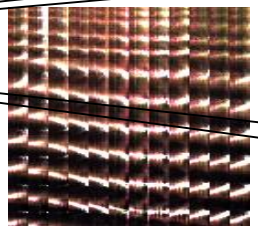


# Multiplexed illumination

De-noise using data-driven linear model $U$:

- Let $\tilde{B} = M^{\dagger}MB$ the reconstructed (noisy) measurement matrix and $U$ the matrix of basis vectors
- Compute

$$\hat{B} = \arg\min_V \left\| UV - \tilde{B} \right\|_F^2$$

$U \times (U^t \times$  $) =$ 

# Experimental Setup

- 3 classes, 12 materials each
  - ► *wood*, *cloth*, *leather*
  - ► 4 test materials / class
- ground-truth measurement
  - ► Camera gain 10 dB (increases speed)
  - ► 768 basis ABRDFs / class
- multiplexed measurement
  - ► Camera gain 0 dB (reduces noise)

# Results: Leather

demultiplexed
$t =$ **1.2 h**, rel. err. **15.7%**

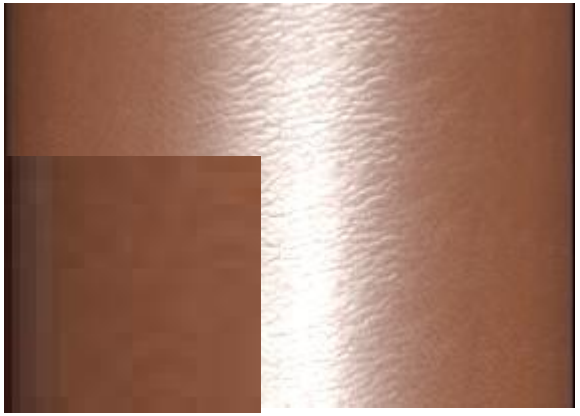
ground truth
$t =$ **23.6 h**

de-noised
$t =$ **1.2 h**, rel. err. **6.2%**

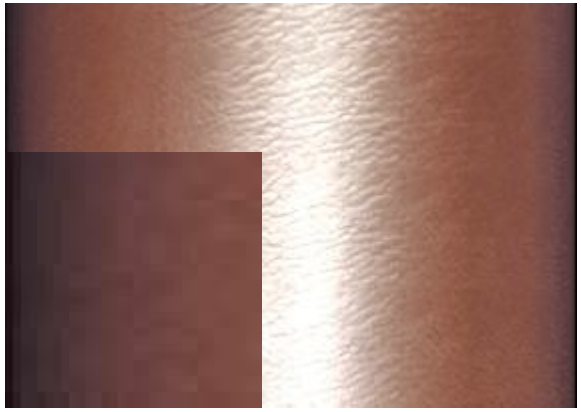
textures for camera at $\theta = 75°$

# Results: Leather #2

demultiplexed
$t =$ **1.2 h**, rel. err. **19.9%**



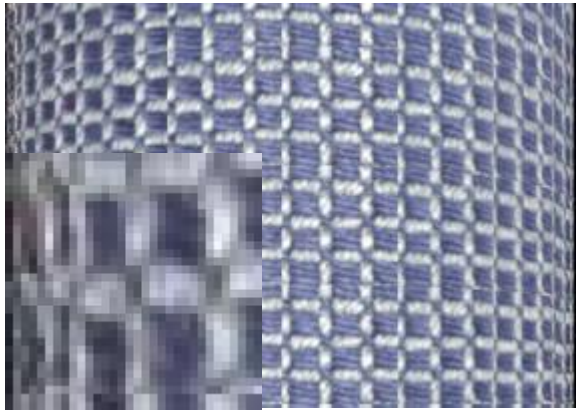
ground truth
$t =$ **23.6 h**



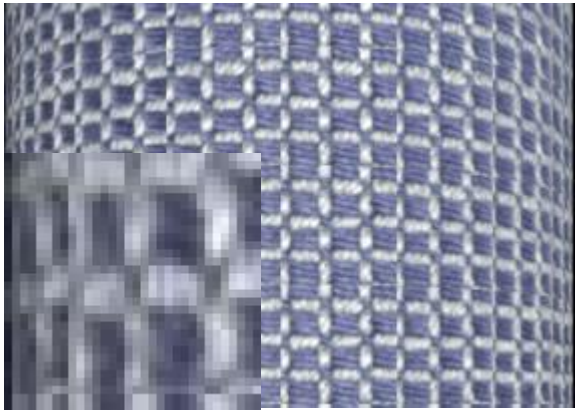
de-noised
$t =$ **1.2 h**, rel. err. **7.1%**

textures for camera at $\theta = 75°$

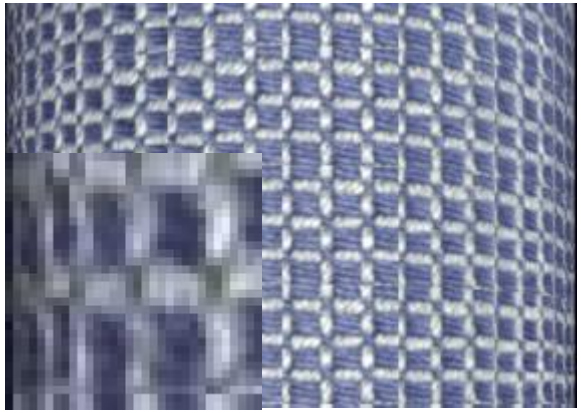
# Results: Cloth



demultiplexed
$t =$ **1.8 h**, rel. err. **13.1%**



ground truth
$t =$ **10.8 h**



de-noised
$t =$ **1.8 h**, rel. err. **6.2%**

textures for camera at $\theta = 75°$

# Results: Wood

demultiplexed
$t =$ **1.2 h**, rel. err. **7.0%**

ground truth
$t =$ **4.4 h**

de-noised
$t =$ **1.2 h**, rel. err. **3.5%**

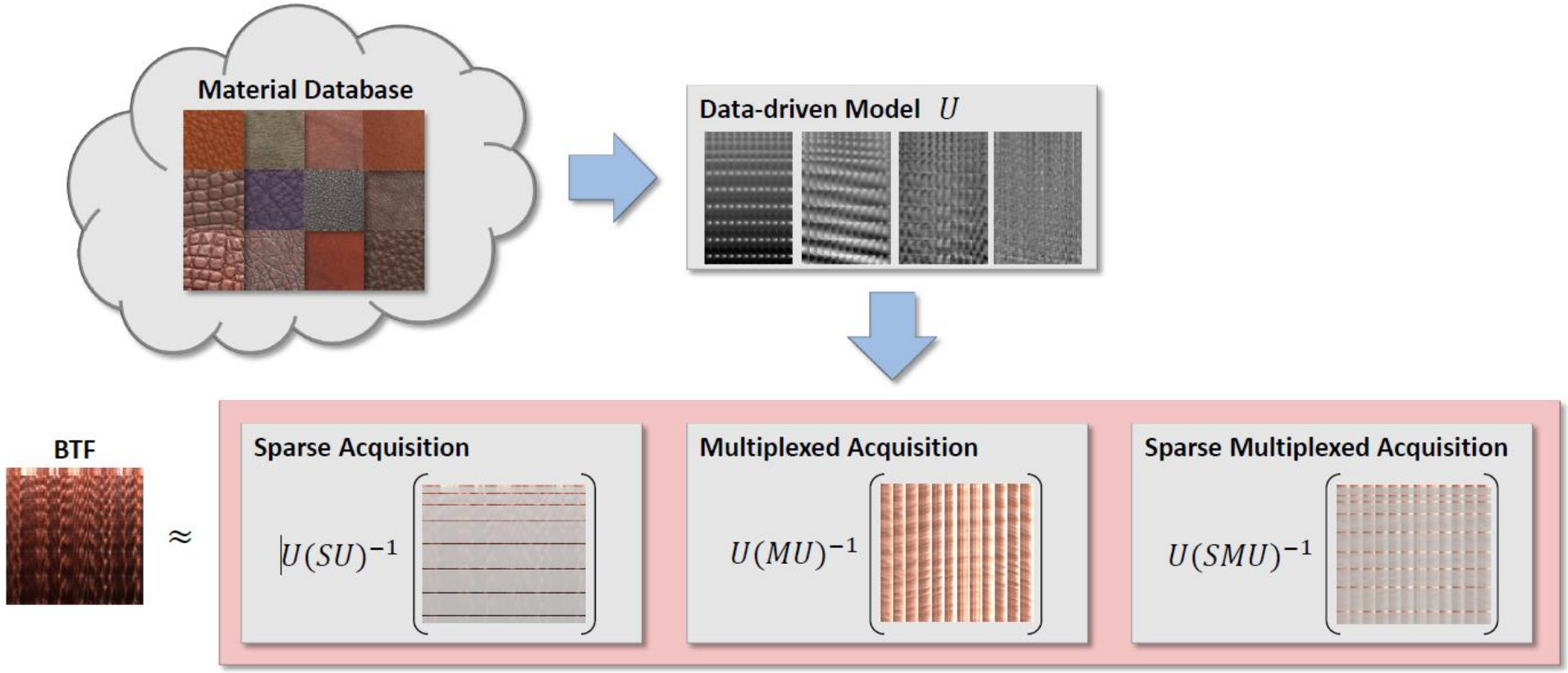
textures for camera at $\theta = 75°$

# Conclusion

- Multiplexed BTF acquisition feasible
  - Acquisition time reduced by 75–95%
  - Well behaved materials (e.g. wood) : noise nearly unnoticeable
  - De-noising via projection
    - typically at least halves relative error
    - much more plausible appearance
- Conceivable improvements:
  - Measure offending images conventionally
  - Determine noisy textures using statistics and treat as missing values
  - Extend prior-based patterns (Mitra et al. 2014) that cope with bright images to BTFs

# Sparse and Multiplexed Aquisition

Combining sparse and Multiplexed Acquisition



# Sparse and Multiplexed Aquisition

Optimization problem

$$V = \arg\min_{\tilde{V}} \left\| \underbrace{S}_{sparse} \underbrace{M}_{multiplex} \underbrace{U}_{basis} \tilde{V} - \underbrace{S}_{sparse} \underbrace{M}_{multiplex} B \right\|_F^2 + \underbrace{\left\| R\tilde{V} \right\|_F^2}_{regularization}$$

Measured data are HDR data, the basis $U$ is given in log space:

$$\left( \rho \mapsto \log\left( \frac{\rho \cos_{weight} + \varepsilon}{\rho \cos_{ref} + \varepsilon} \right) \right)$$

Problem: SMB and SMU are incompatible

- We cannot infer $M \log(B)$ from the measurement $MB$

$$\left( \log\left( MB \right) \neq M \log(B) \right)$$

# Sparse and Multiplexed Aquisition

Solution: per-entry weights for basis computation instead of log-space transform!*

Optimization problem:

$$U,V = \arg\min_{\tilde{U},\tilde{V}} \left\| \underbrace{W}_{weights} \odot \left( \tilde{U}\tilde{V} - \underbrace{D}_{data\ base} \right) \right\|_F^2$$

$\odot$ entry-wise matrix product

$D \in \mathbb{R}^{n_{lv} \times n_{tx}}$

* Ruiters, Roland, Christopher Schwartz, und Reinhard Klein. „Data driven surface reflectance from sparse and irregular samples“. In *Computer Graphics Forum*, 31:315–324. Wiley Online Library, 2012.
* Bagher, Mahdi M., John Snyder, und Derek Nowrouzezahrai. „A Non-Parametric Factor Microfacet Model for Isotropic BRDFs“. *ACM Transactions on Graphics* 35, Nr. 5 (28. Juli 2016): 1–16.

# Sparse and Multiplexed Aquisition

Solution: per-entry weights for basis computation instead of log-space transform!* **

Optimization problem:

$$U, V = \arg\min_{\tilde{U},\tilde{V}} \left\| \underbrace{W}_{weights} \square \left( \tilde{U}\tilde{V} - \underbrace{D}_{data\ base} \right) \right\|_F^2$$

Taking $\boldsymbol{W}$ as the entry-wise inverse of $\boldsymbol{D}$ the relative L2-error instead of the absolute one is minimized.*

$\square$ entry-wise matrix product

$$D \in \quad ^{n_{lv} \times n_{tx}}$$

* Ruiters, Roland, Christopher Schwartz, und Reinhard Klein. „Data driven surface reflectance from sparse and irregular samples“. In *Computer Graphics Forum*, 31:315–324. Wiley Online Library, 2012.
** Bagher, Mahdi M., John Snyder, und Derek Nowrouzezahrai. „A Non-Parametric Factor Microfacet Model for Isotropic BRDFs“. *ACM Transactions on Graphics* 35, Nr. 5 (28. Juli 2016): 1–16.

# Sparse and Multiplexed Aquisition

Reconstruction given a basis is straight forward:

$$V = \left( \left( SMU \right)^t \left( SMU \right) + R^t R \right)^{-1} \left( SMU \right)^t \left( SMB \right)$$

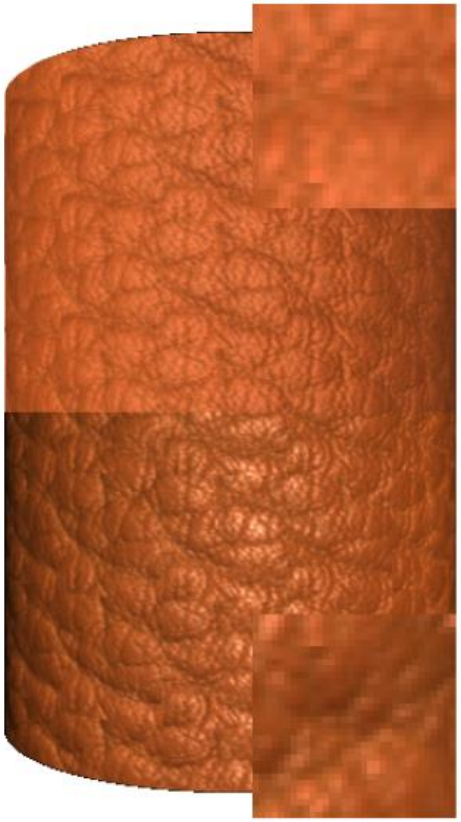
$R$ is appropriate regularization matrix,

e.g. $R = \lambda \Sigma^{-1}, \lambda \in$ *

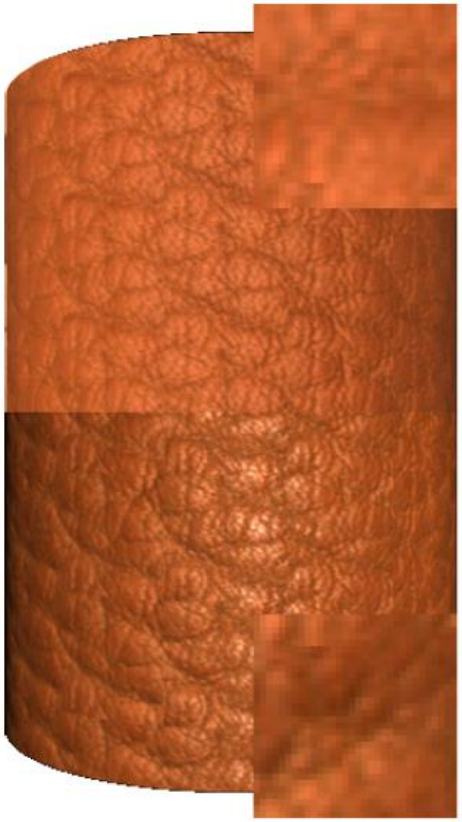
By taking $S = I$ or $M = I$ the problem is reduced to sparse and multiplexed aqcuisition, respectively.

*Nielsen, JB, Jensen, HW, Ramamoorthi, R. On optimal, minimal BRDF sampling for reflectance acquisition. ACM Trans Graph 2015;34(6)

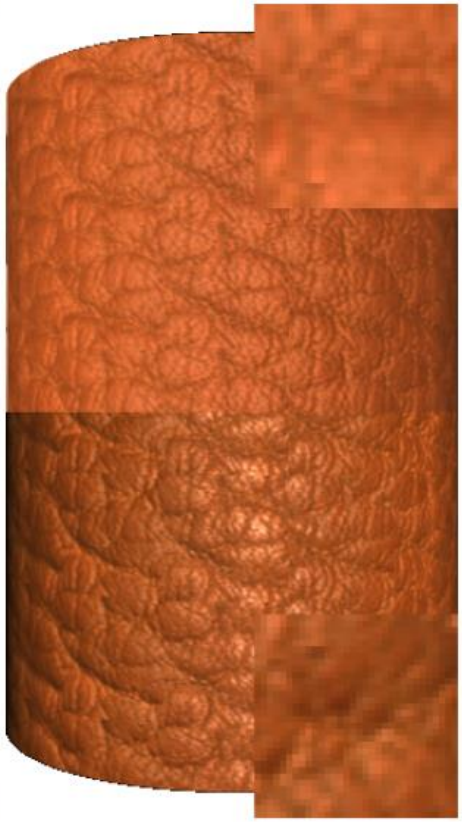
# Results: Leather



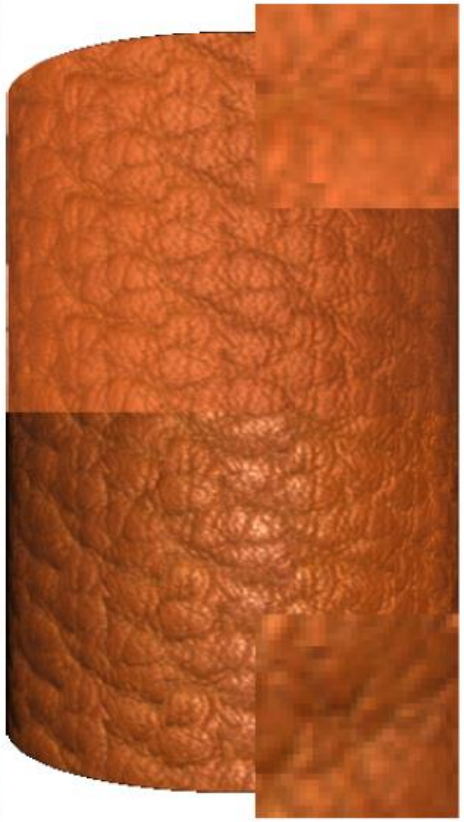
Refernce
$t =$ **22.8 h**



sparse & multiplexed
$t =$ **12 min**,
PSNR 36,7 dB

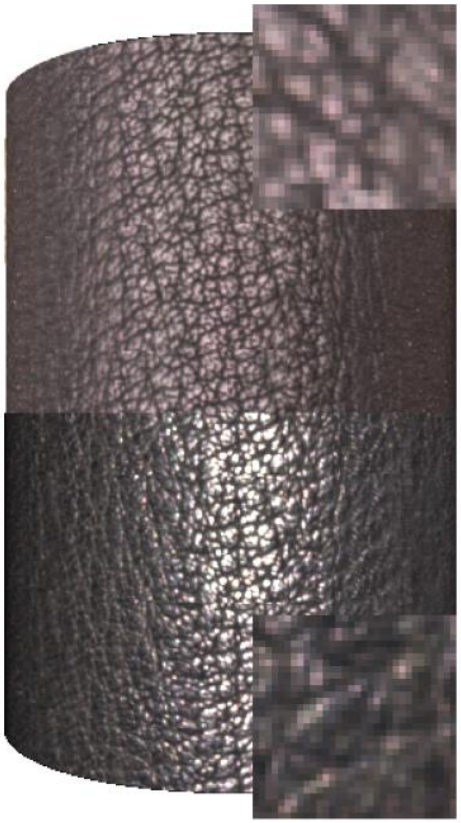


sparse (Nielsen et al.)
$t =$ **4h**,
PSNR 36,4 dB

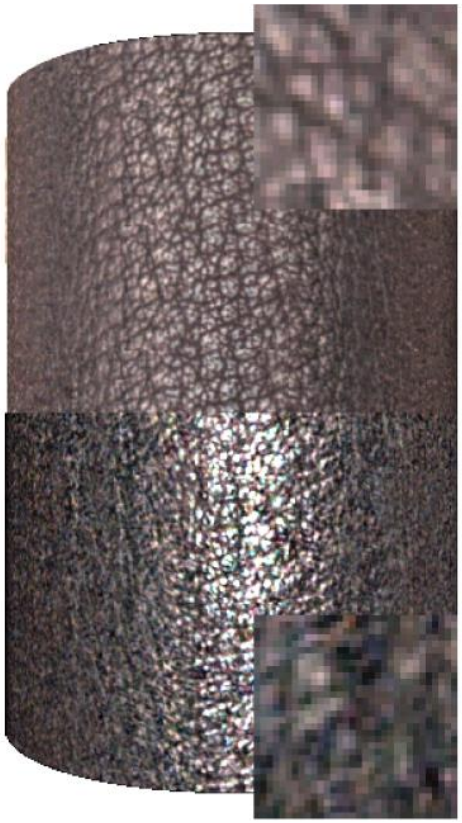


multiplexed
$t =$ **1.2 h**,
PSNR 34.8 dB

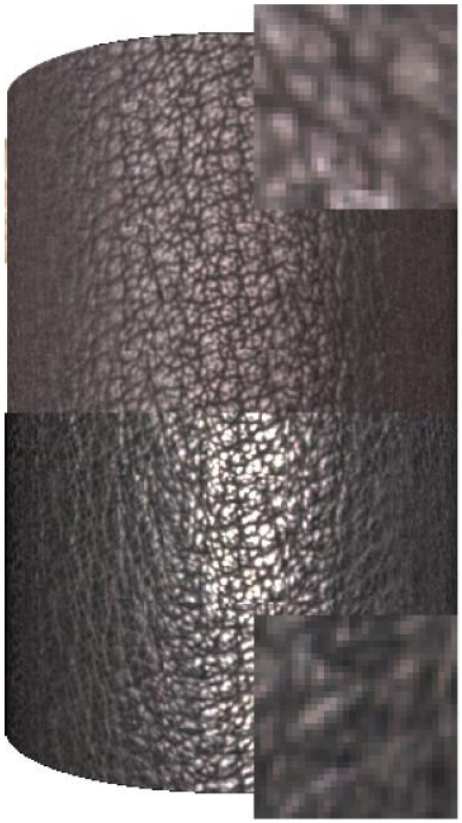
# Results: Leather



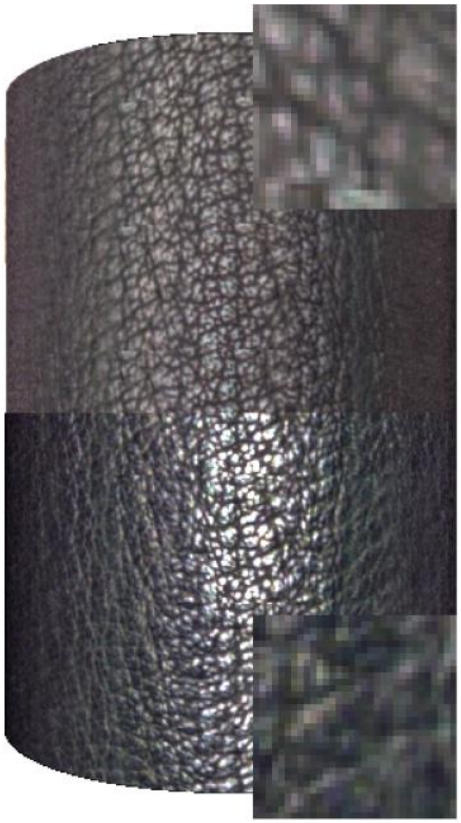
Refernce
$t =$ **20.6 h**



sparse & multiplexed
$t =$ **16 min**,
PSNR 22 dB



sparse (Nielsen et al.)
$t =$ **3.4h**,
PSNR 28.5 dB



multiplexed
$t =$ **1.5 h**,
PSNR 25.7 dB

# Results: Cloth

Refernce
$t =$ **10.8 h**

sparse & multiplexed
$t =$ **16 min**,
PSNR 27.8 dB

sparse (Nielsen et al.)
$t =$ **1.8 h**,
PSNR 32 dB

multiplexed
$t =$ **1.6 h**,
PSNR 28.1dB

# BRDF Factorization

**Idea:** Approximate BRDF by a product of functions

- E.g.: 4D BRDF as a product of two bivariate Functions ($\pi$ is a projection to a 2D parameter space):

$$f(\varphi_i, \theta_i, \varphi_o, \theta_o) = G(\pi_G(\varphi_i, \theta_i, \varphi_o, \theta_o)) \cdot H(\pi_H(\varphi_i, \theta_i, \varphi_o, \theta_o))$$

- E.g.

$$f(\varphi_i, \theta_i, \varphi_o, \theta_o) = G(\varphi_i, \theta_i) \cdot H(\varphi_o, \theta_o)$$

$$\Rightarrow dL_o = G \cdot H \cdot L_i \cdot \cos\theta_i$$

- 2D functions ***G*** and ***H*** require less storrage than the original 4D BRDF ***f***

# BRDF Factorization

Generalization:

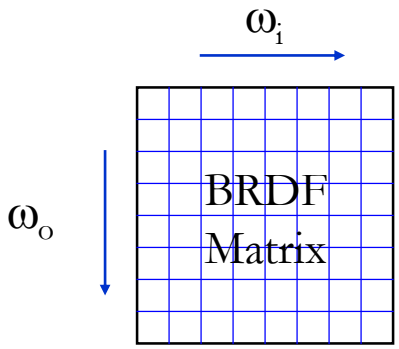
- Store a sum of several such functions:

- May increase accuracy of the approximation

$$f = \sum_j G_j \cdot H_j$$

# Matrix-Based BRDF Approximations

1. Step:
   - Sampling of the BRDF und storage in a ***m***x***n*** 2D matrix
   - Each column / row corresponds to a incoming- / outgoing-direction



# Matrix-Based BRDF Approximations

- E.g: Matrix notation for a BRDF sampled by 2x2x2x2 viewing/lighting direction pairs (resulting matrix: ***m=4,n=4***)

$$
\begin{array}{c|cccc}
 & \theta_{i0},\phi_{i0} & \theta_{i0},\phi_{i1} & \theta_{i1},\phi_{i0} & \theta_{i1},\phi_{i1} \\
\hline
\theta_{o0},\phi_{o0} & \rho(\theta_{i0},\phi_{i0},\theta_{o0},\phi_{o0}) & \rho(\theta_{i0},\phi_{i1},\theta_{o0},\phi_{o0}) & \rho(\theta_{i1},\phi_{i0},\theta_{o0},\phi_{o0}) & \rho(\theta_{i1},\phi_{i1},\theta_{o0},\phi_{o0}) \\
\theta_{o0},\phi_{o1} & \rho(\theta_{i0},\phi_{i0},\theta_{o0},\phi_{o1}) & \rho(\theta_{i0},\phi_{i1},\theta_{o0},\phi_{o1}) & \rho(\theta_{i1},\phi_{i0},\theta_{o0},\phi_{o1}) & \rho(\theta_{i1},\phi_{i1},\theta_{o0},\phi_{o1}) \\
\theta_{o1},\phi_{o0} & \rho(\theta_{i0},\phi_{i0},\theta_{o1},\phi_{o0}) & \rho(\theta_{i0},\phi_{i1},\theta_{o1},\phi_{o0}) & \rho(\theta_{i1},\phi_{i0},\theta_{o1},\phi_{o0}) & \rho(\theta_{i1},\phi_{i1},\theta_{o1},\phi_{o0}) \\
\theta_{o1},\phi_{o1} & \rho(\theta_{i0},\phi_{i0},\theta_{o1},\phi_{o1}) & \rho(\theta_{i0},\phi_{i1},\theta_{o1},\phi_{o1}) & \rho(\theta_{i1},\phi_{i0},\theta_{o1},\phi_{o1}) & \rho(\theta_{i1},\phi_{i1},\theta_{o1},\phi_{o1})
\end{array}
$$

k=1,…,n

j=1,..,m

2. Step:

- Decomposition
  - Normalized Decomposition (ND)
  - Singular Value Decomposition (SVD)
  - …

# Normalized Decomposition

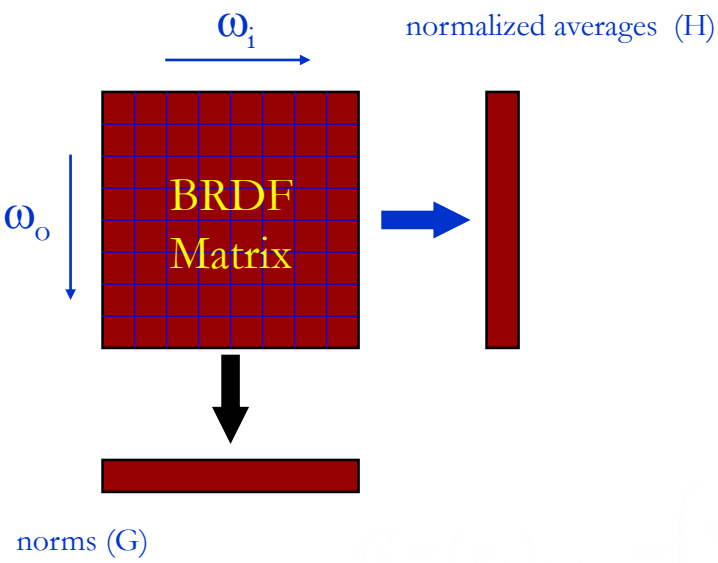
- Consider single term factorization of a matrix ***f*** with residual R:

$$f(j,k) = G^T(k)H(j) + R \approx f'(j,k) = G^T(k)H(j)$$

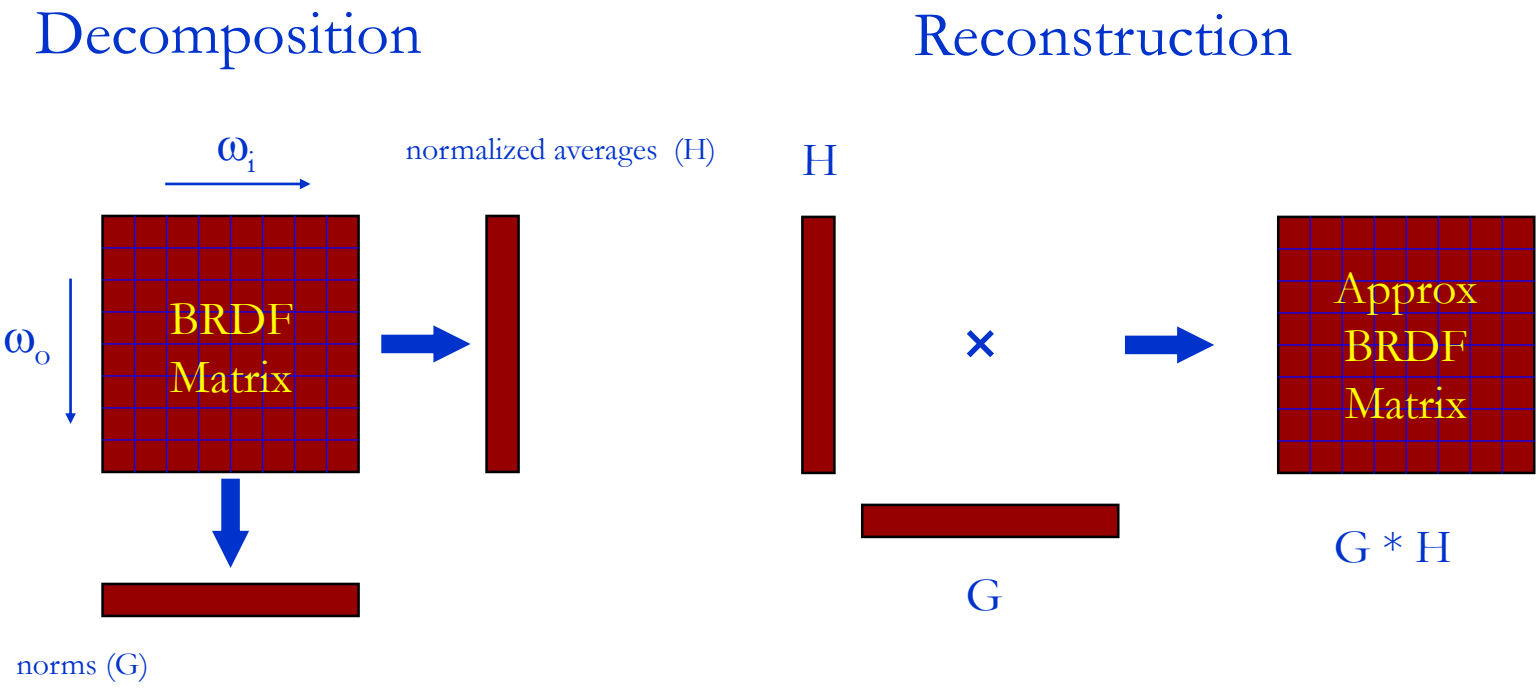
  - If ***k*** is kept fixed ***G*** is simply a constant scaling a vector ***H***

# Normalized Decomposition

## Decomposition

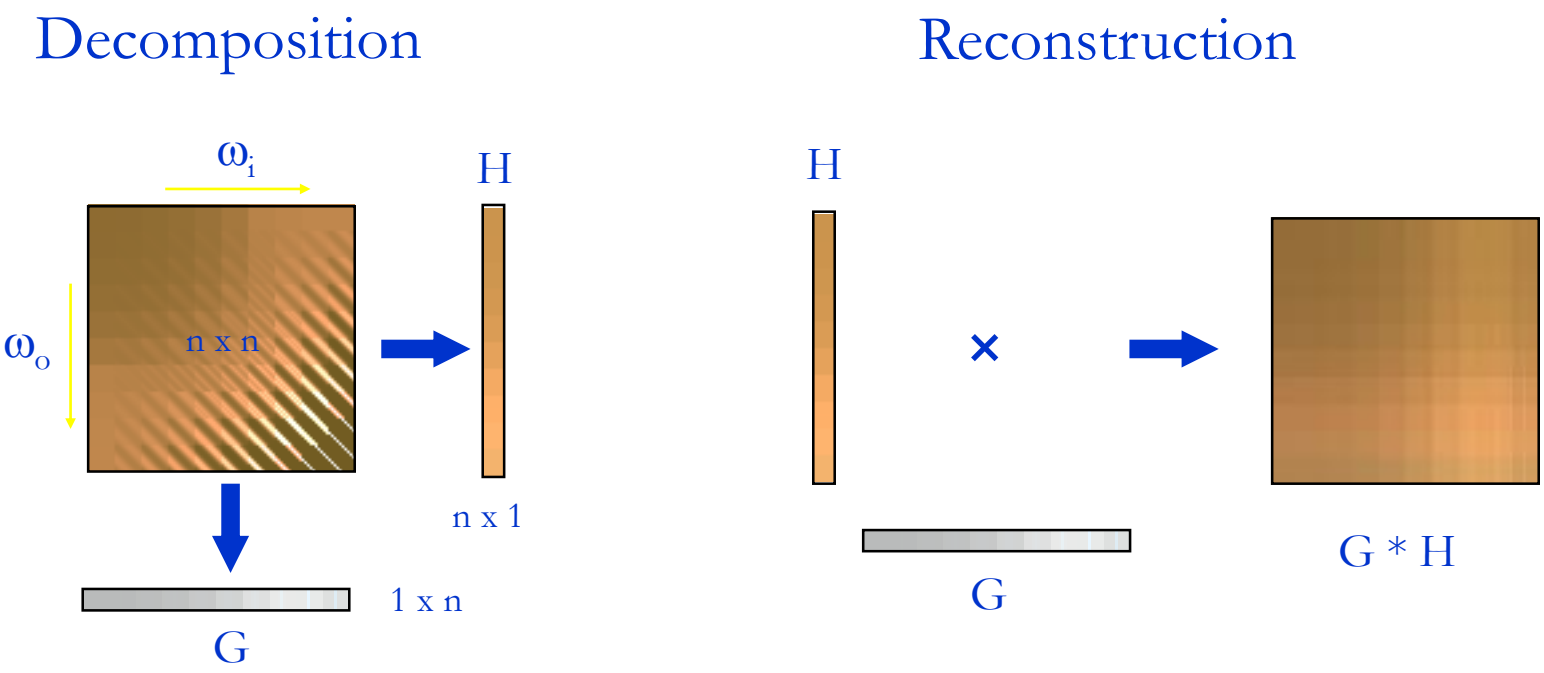


# Normalized Decomposition



# Normalized Decomposition

Illustrating example

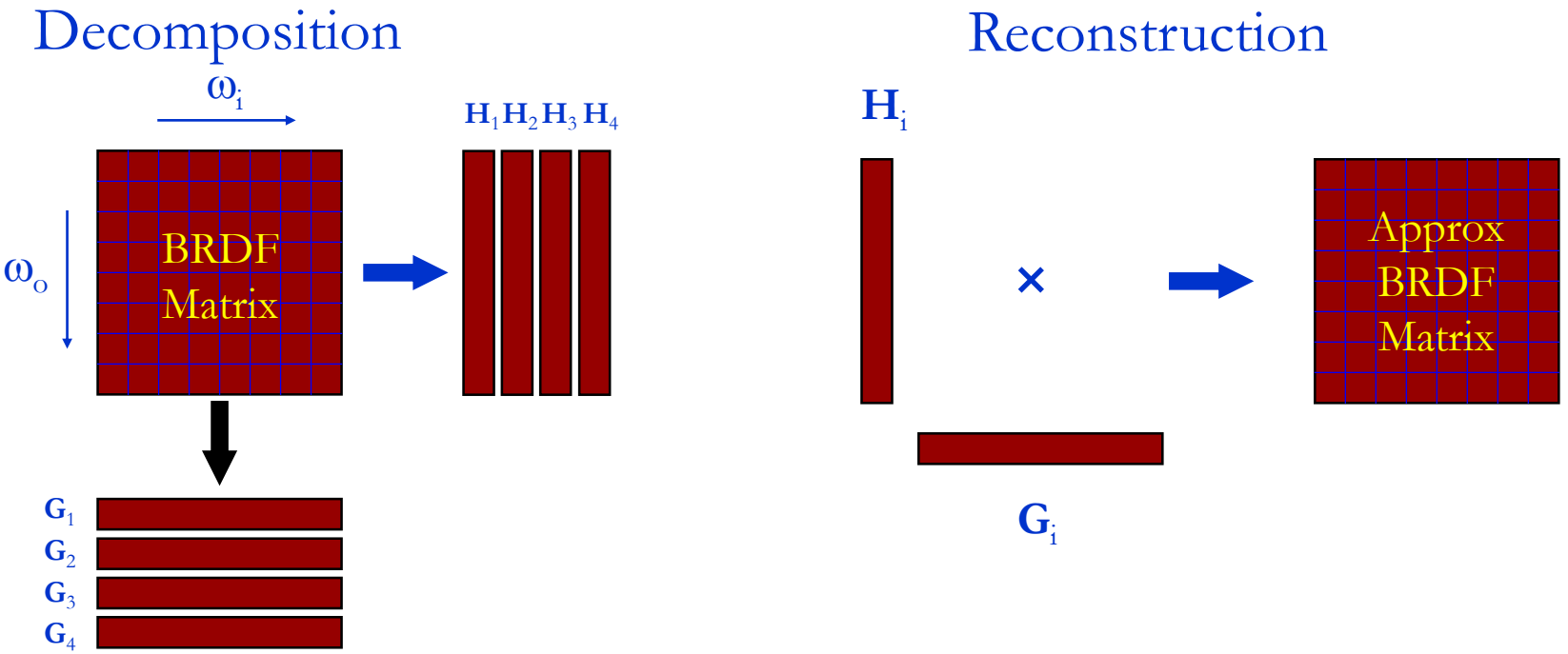


# Normalized Decomposition

- Higher order“ decomposition (sum of matrices):
  - Continue with ND Factorization of residual R:

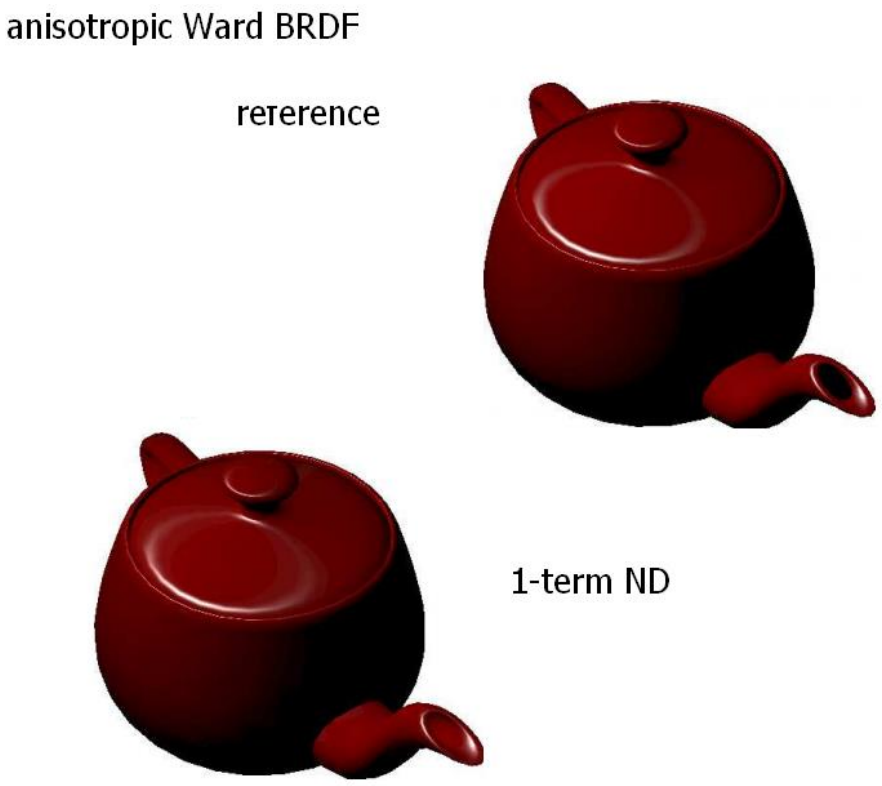
$$f(j,k) = G_1^T(k)H_1(j) + R_1 = G_1^T(k)H_1(j) + G_2^T(k)H_2(j) + R_2 = \ldots$$

- „Higher order“ decomposition (example)



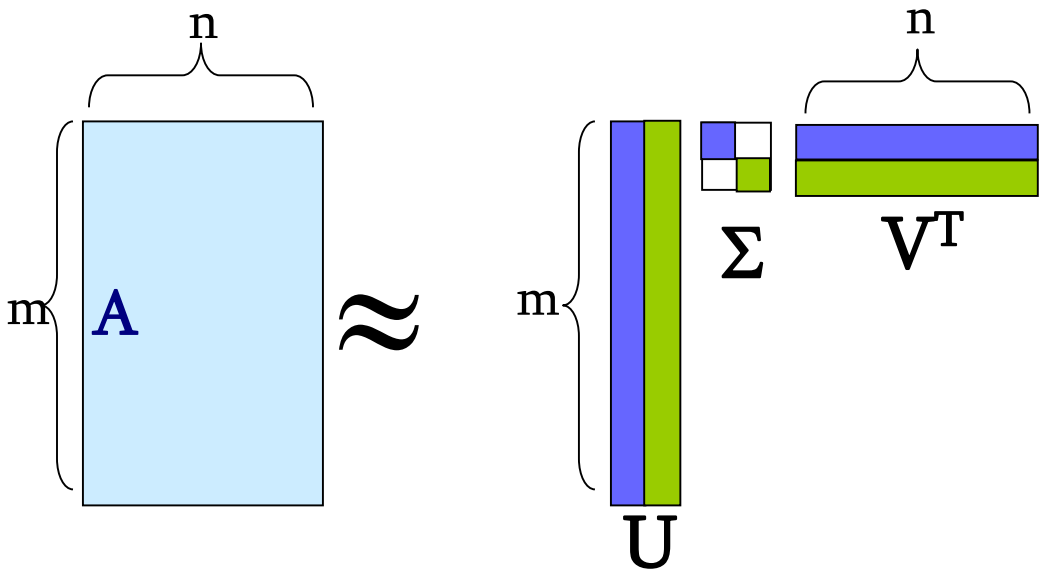
# Normalized Decomposition

- Results
  - See also slides about BRDF parameterization



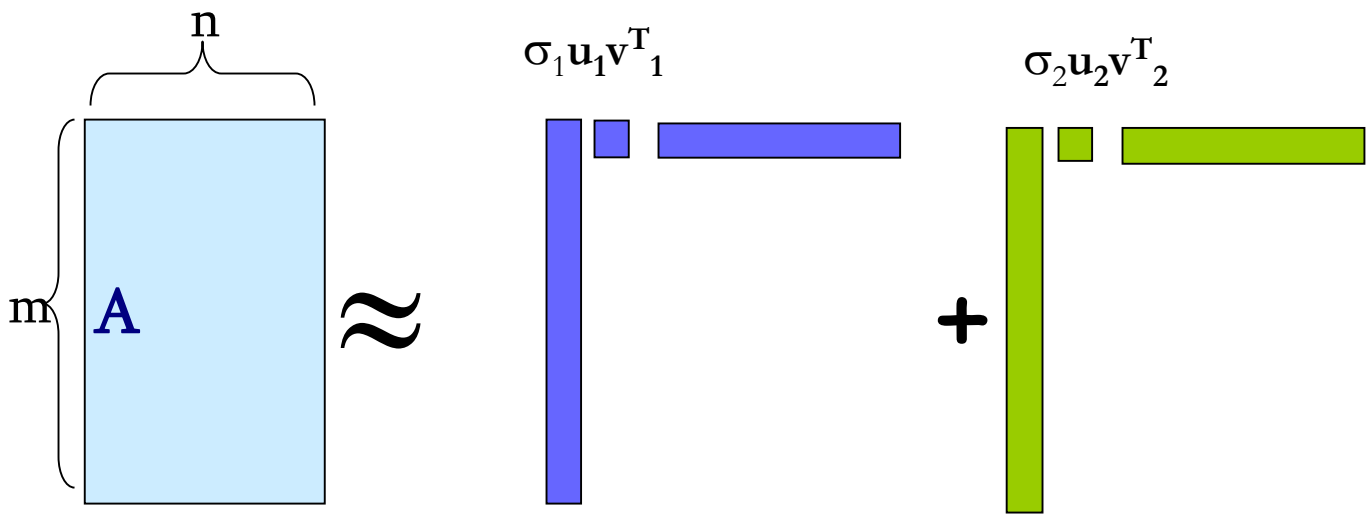
# Singular Value Decomposition

$$A = U\Sigma V^T \approx \sum_{k=1}^{2} \sigma_k u_k \circ v_k^T$$



# Singular Value Decomposition

$$A = U\Sigma V^T \approx \sum_{k=1}^{2} \sigma_k u_k \circ v_k^T$$




Matrix SVD

- rank reducibility
- orthonormal row/column matrices

# SVD vs. ND

- SVD
  - Optimality (least square sense)
  - Slow, memory hungry  (Full decomposition needs to be computed first !)
- ND
  - Not optimality guaranteed but similar results to SVD in practice
  - Fast, memory efficient (Incremental „higher order“ decomposition possible !)

# Tensor Decomposition

- Matrix factorization only allows decomposition into products of **two** functions
- BRDFs are higher dimensional functions
  - Better compression can be achieved by directly taking advantage of these additional dimensions by storing the BRDF in a **tensor** (a higher dimensional array)
- Represent it as a sum of products of more than two functions:

$$f \approx \sum_{j=1}^{N} \prod_{k=1}^{K} p_{jk}(\omega_i, \omega_o)$$

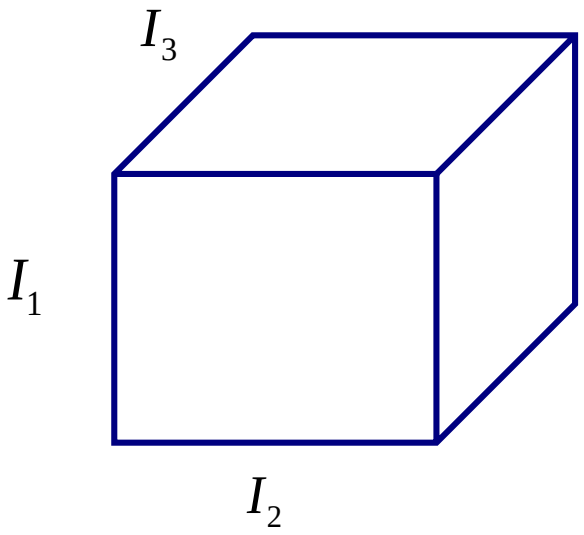
# What is a tensor

- Tensors are multi-linear mappings that generalize vectors (1st order tensor) and matrices (2nd order tensor)
- For simplicity think of tensors as multidimensional arrays:

$$A \in I_1 \times I_2 \times \ldots \times I_N$$

- Examples are data sets
  - images, image collections, video, volume data etc

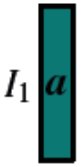
- Order 3: $A$



# What is a tensor

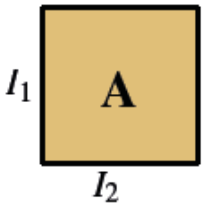
$a$

0-order tensor



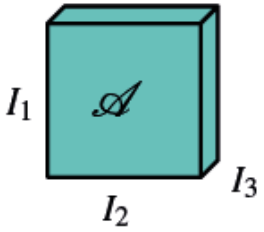

$i_1 = 1, \ldots, I_1$

1st-order tensor




$i_2 = 1, \ldots, I_2$

2nd-order tensor




$i_3 = 1, \ldots, I_3$

3rd-order tensor

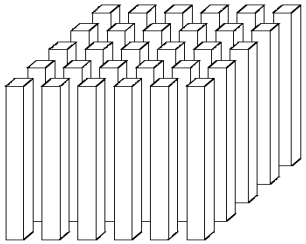
$\mathscr{A}^{N-\text{th order}} \in \mathbb{R}^{I_1 \times I_2 \cdots \times I_N}$

...

# Tensor Basics

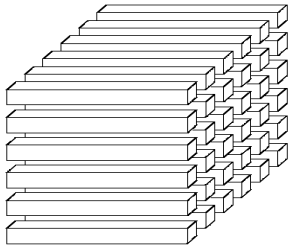
- A tensor can be divided into **fibers** by keeping the indices for all but one mode fixed:

mode-1 fibers



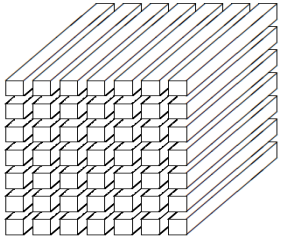
$\mathbf{X}_{:,j,k}$

mode-2 fibers



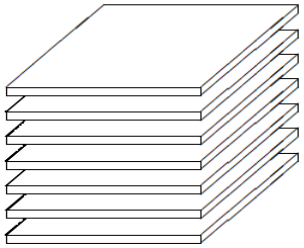
$\mathbf{X}_{i,:,k}$

mode-3 fibers

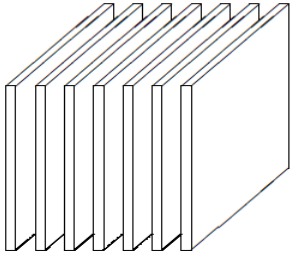


$\mathbf{X}_{i,j,:}$

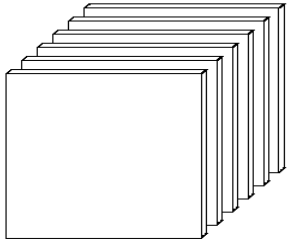
- **Slices** are obtained by keeping all but two components fixed:



$\mathbf{X}_{i,:,:}$



$\mathbf{X}_{:,j,:}$



$\mathbf{X}_{:,:,k}$

# Tensor Basics

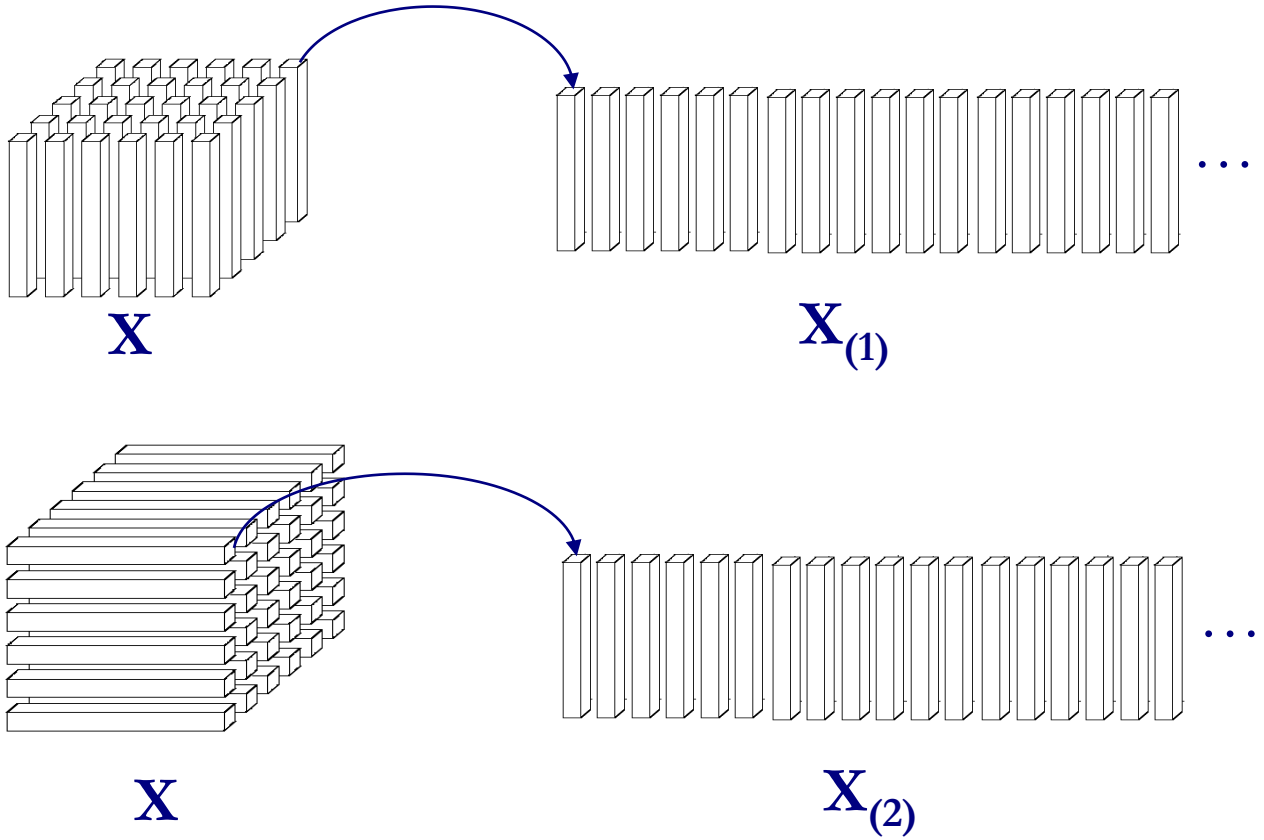
mode-n vectors:

- column vectors of *flattened* matrix

$$\mathbf{A}_{(n)} \in \quad I_n \times (I_1 I_2 \ldots I_{n-1} I_{n+1} \ldots I_N)$$

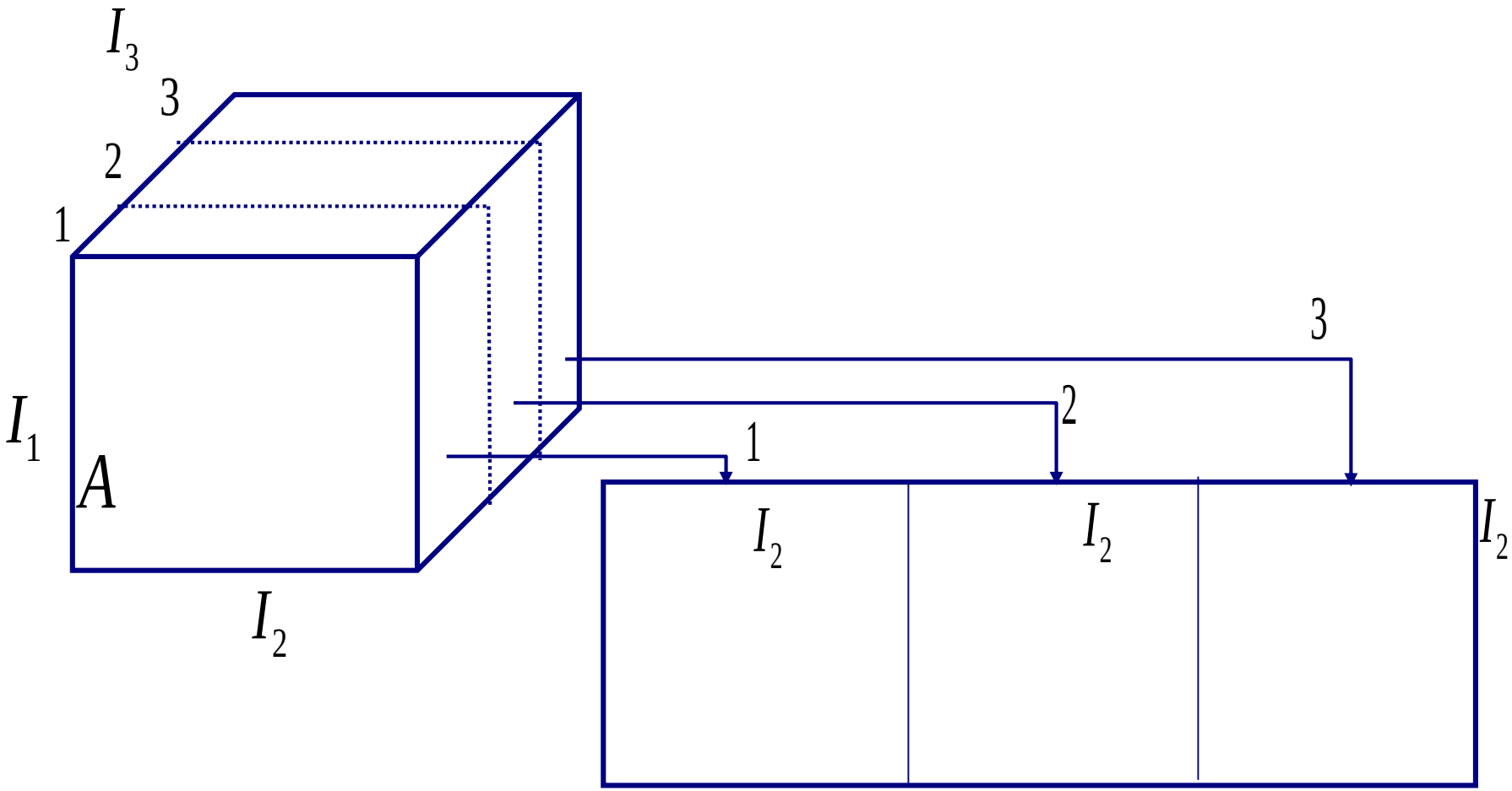
# Tensor Basics

- An unfolded tensor $\mathbf{X}_{(n)}$ is obtained by arranging all mode-n fibers in a matrix:



# Tensor Basics

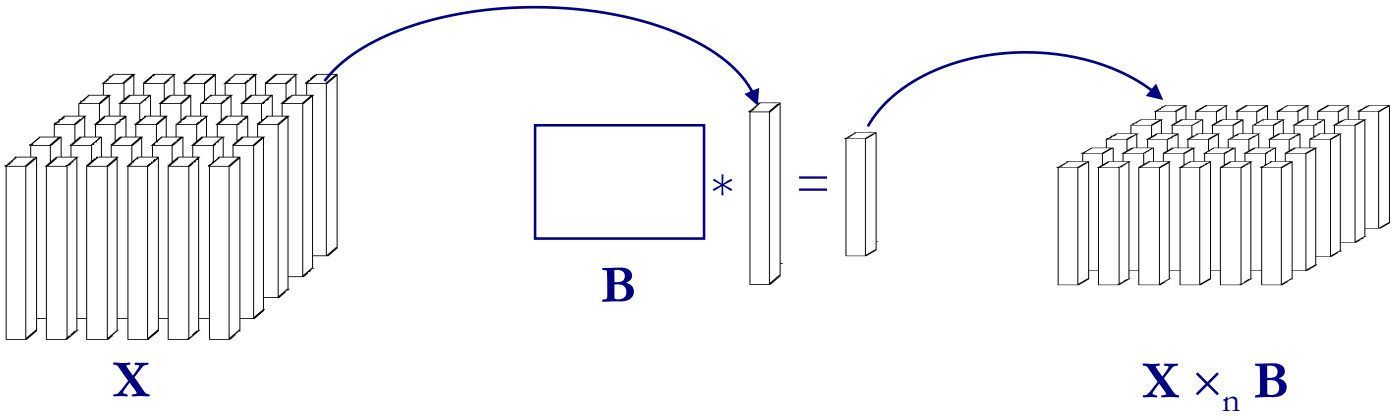
Example (mode-1 vectors of order-3 tensor)



# Tensor Basics

The **Mode-n product X** $\times_n$ **B** multiplies a tensor **X** with a matrix **B**

- Each fiber is multiplied separately with the matrix:




$$\mathbf{Y} = \mathbf{X} \times_n \mathbf{B}, \quad \mathbf{X} \in \mathbb{R}^{I_1 \times \ldots \times I_n \times \ldots \times I_N}, \ \mathbf{B} \in \mathbb{R}^{J_n \times I_n}$$

$$\mathbf{Y} \in \mathbb{R}^{I_1 \times \ldots \times I_{n-1} \times J_n \times I_{n+1} \times \ldots \times I_N}, \quad \mathbf{Y}_{(n)} = \mathbf{B}\mathbf{X}_{(n)}$$

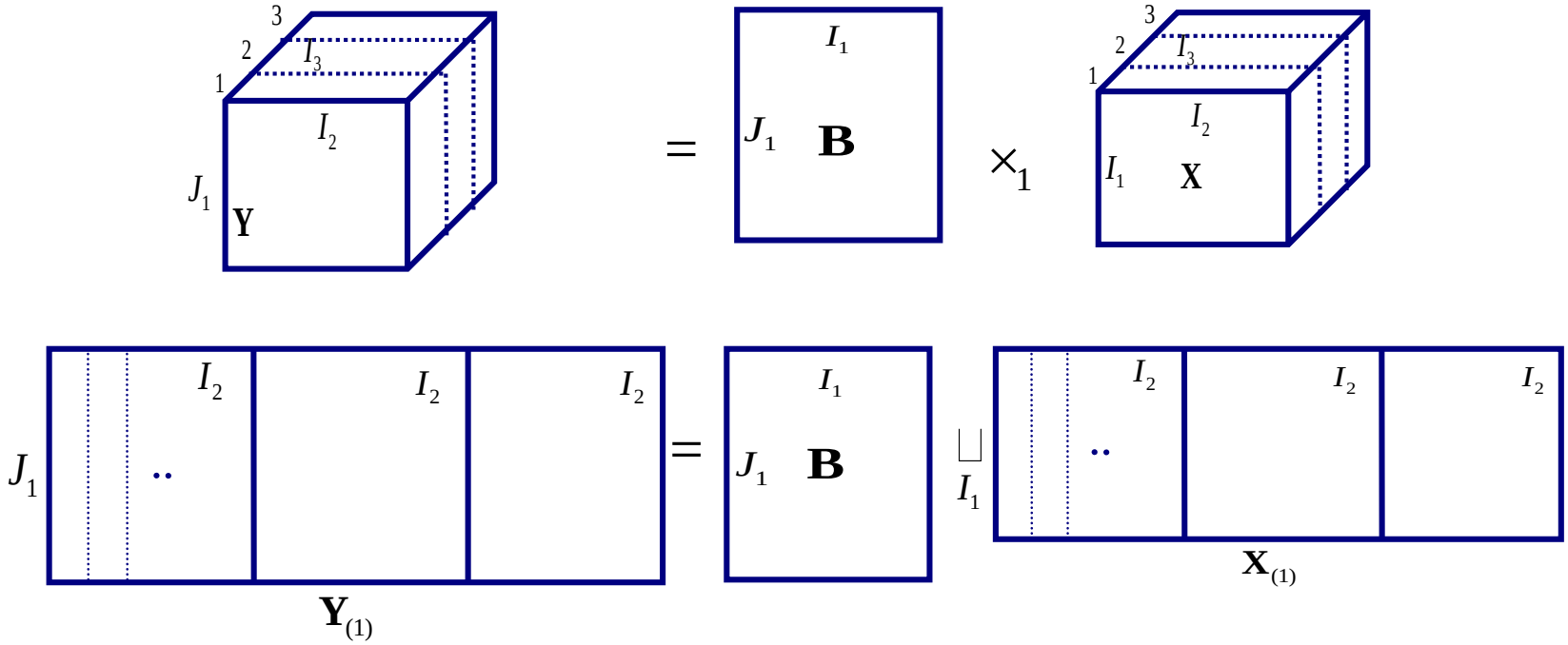
- The mode-n product can also be expressed in terms of unfolded tensors:

$$(\mathbf{X} \times_n \mathbf{B})_{(n)} = \mathbf{B} * \mathbf{X}_{(n)}$$

**Note: A** * **B** denotes here the standard matrix multiplication

# Tensor Basics

## Example

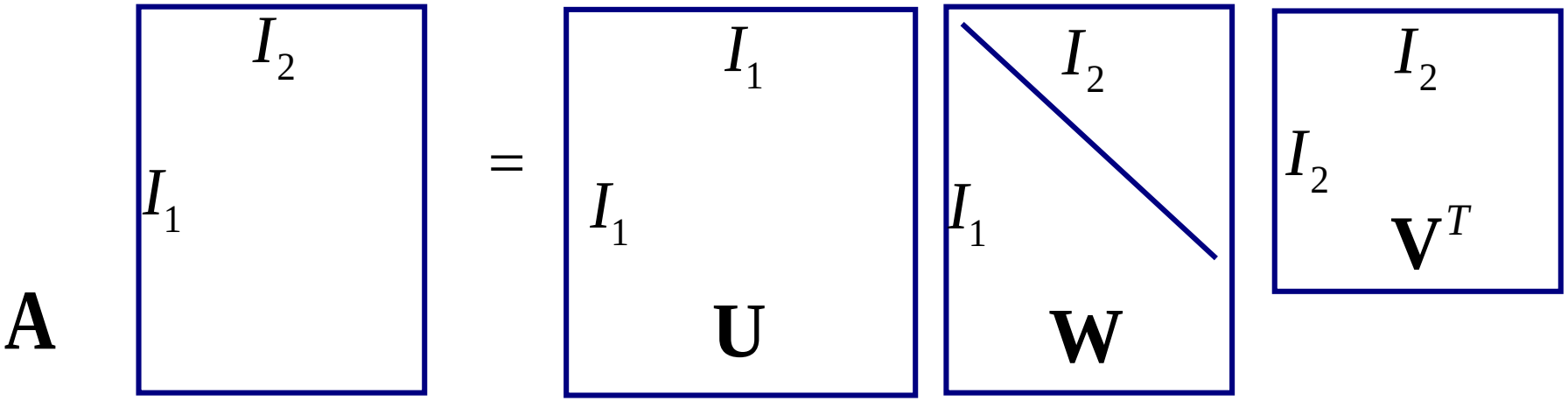


# Tensor Basics

- Example: SVD in tensor notation
  - SVD orthogonalizes column and row vector spaces of $\mathbf{A}$

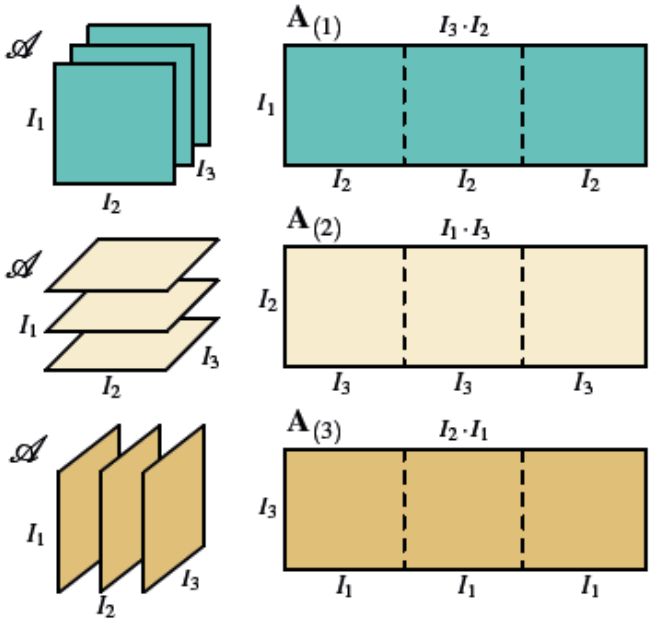
$$\mathbf{A} = \mathbf{U}\mathbf{W}\mathbf{V}^{\mathbf{T}}$$

$$\mathbf{A} = \mathbf{W} \times_1 \mathbf{U} \times_2 \mathbf{V}$$



# Unfolding and ranks

- Operations with tensors often performed as matrix operations using unfolded tensor representations
  - different tensor unfolding strategies possible
- Forward cyclic unfolding $\mathbf{A}_{(n)}$ of a 3rd order tensor $\mathcal{A}$ (or 3D volume)
- The $n$-rank of a tensor is typically defined on an unfolding
  - $n$-rank $R_n = \text{rank}_n(\mathcal{A}) = \text{rank}(\mathbf{A}_{(n)})$
  - multilinear rank-$(R_1, R_2, \ldots, R_N)$ of $\mathcal{A}$

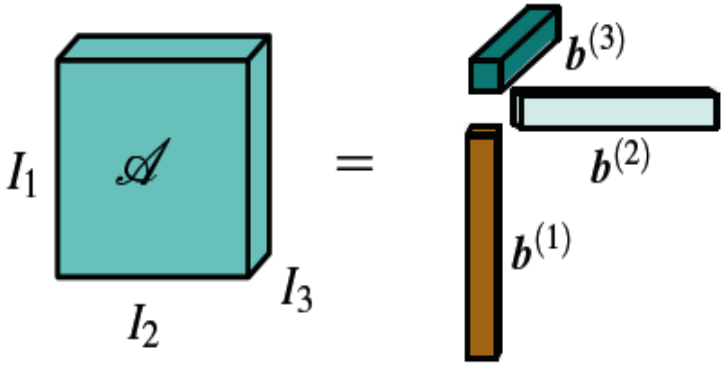


# Rank-1 Tensor

- $N$-mode tensor $\mathscr{A} \in \mathbb{R}^{I1 \times \ldots \times IN}$ that can be expressed as the outer product of $N$ vectors
  - Kruskal tensor

$$\mathscr{A} = \boldsymbol{b}^{(1)} \circ \boldsymbol{b}^{(2)} \circ \cdots \circ \boldsymbol{b}^{(N)}$$

- Useful to understand principles of rank-reduced tensor reconstruction
  - linear combination of rank-one tensors



# Rank-1 Tensor

- Outer product of vectors

If $b^{(1)} \in \mathbb{R}^{I_1}, b^{(1)} \in \mathbb{R}^{I_2}, \cdots, b^{(n)} \in \mathbb{R}^{I_n}$ then the outer product

$$\mathcal{A} = \mathbf{b}^{(1)} \circ \mathbf{b}^{(2)} \circ \cdots \circ \mathbf{b}^{(n)}$$

is defined by

$$\mathcal{A}(i_1, i_2, \cdots, i_n) = b^{(1)}(i_1) b^{(2)}(i_2) \cdots b^{(n)}(i_n)$$

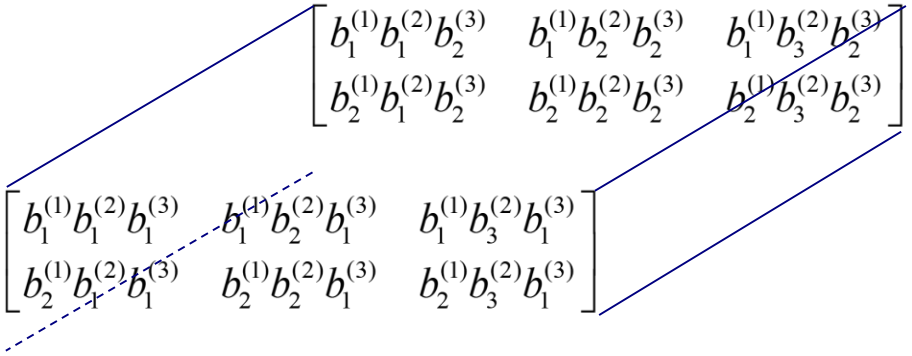
The tensor $\mathcal{A}(i_1, i_2, \cdots, i_n) \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_n}$ is a rank-1 tensor.

# Rank-1 Tensor

- Example:

$$b^{(1)} = \begin{pmatrix} b_1^{(1)} \\ b_2^{(1)} \end{pmatrix}, b^{(2)} = \begin{pmatrix} b_1^{(2)} \\ b_2^{(2)} \\ b_3^{(2)} \end{pmatrix}, b^{(3)} = \begin{pmatrix} b_1^{(3)} \\ b_2^{(3)} \end{pmatrix}$$

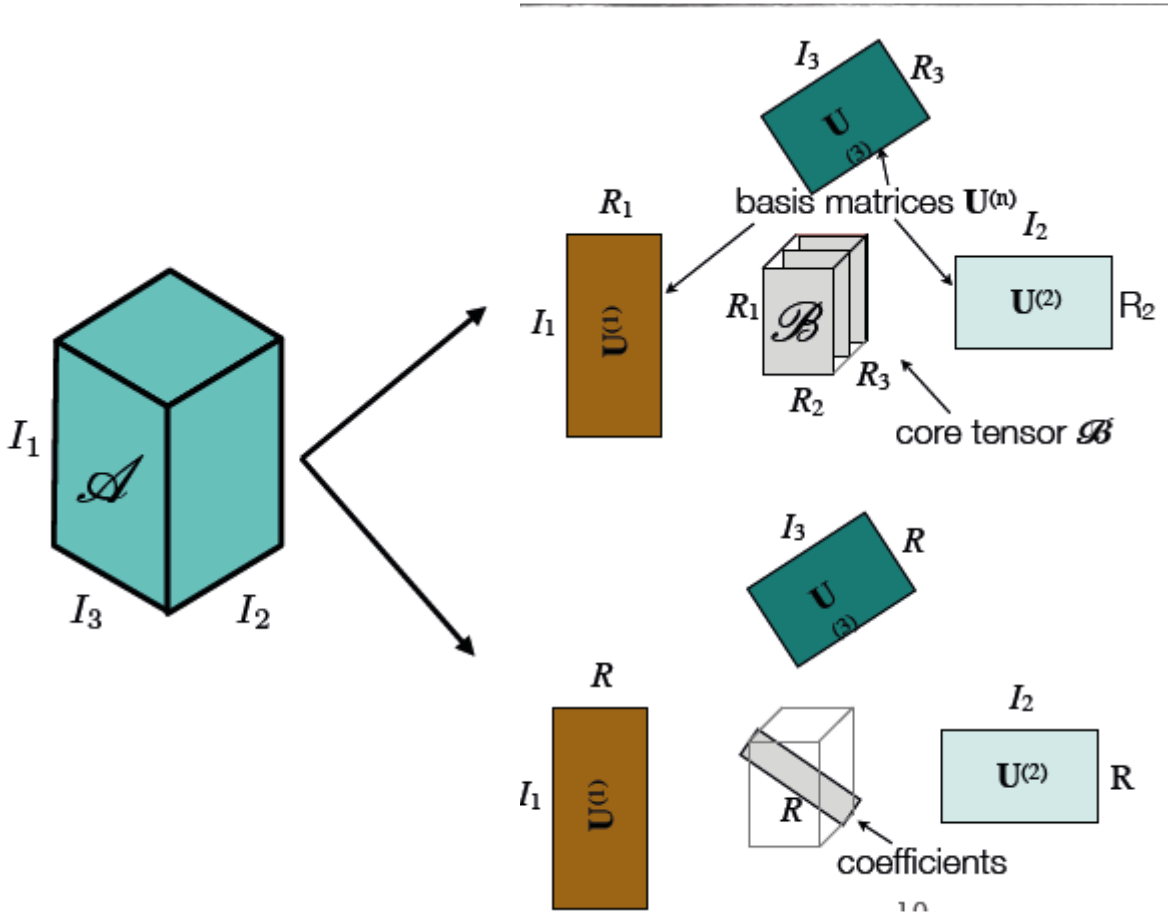
$$\mathcal{B} = b^{(1)} \circ b^{(2)} \circ b^{(3)} =$$



$$\mathcal{B}_{(1)} = \begin{bmatrix} b_1^{(1)}b_1^{(2)}b_1^{(3)} & b_1^{(1)}b_2^{(2)}b_1^{(3)} & b_1^{(1)}b_3^{(2)}b_1^{(3)} & b_1^{(1)}b_1^{(2)}b_2^{(3)} & b_1^{(1)}b_2^{(2)}b_2^{(3)} & b_1^{(1)}b_3^{(2)}b_2^{(3)} \\ b_2^{(1)}b_1^{(2)}b_1^{(3)} & b_2^{(1)}b_2^{(2)}b_1^{(3)} & b_2^{(1)}b_3^{(2)}b_1^{(3)} & b_2^{(1)}b_1^{(2)}b_2^{(3)} & b_2^{(1)}b_2^{(2)}b_2^{(3)} & b_2^{(1)}b_3^{(2)}b_2^{(3)} \end{bmatrix}$$

$$\mathcal{B}_{(2)} = \begin{bmatrix} b_1^{(1)}b_1^{(2)}b_1^{(3)} & b_1^{(1)}b_1^{(2)}b_2^{(3)} & b_2^{(1)}b_1^{(2)}b_1^{(3)} & b_2^{(1)}b_1^{(2)}b_2^{(3)} \\ b_1^{(1)}b_2^{(2)}b_1^{(3)} & b_1^{(1)}b_2^{(2)}b_2^{(3)} & b_2^{(1)}b_2^{(2)}b_1^{(3)} & b_2^{(1)}b_2^{(2)}b_2^{(3)} \\ b_1^{(1)}b_3^{(2)}b_1^{(3)} & b_1^{(1)}b_3^{(2)}b_2^{(3)} & b_2^{(1)}b_3^{(2)}b_1^{(3)} & b_2^{(1)}b_3^{(2)}b_2^{(3)} \end{bmatrix}$$

# Rank-1 Tensor




## Tucker

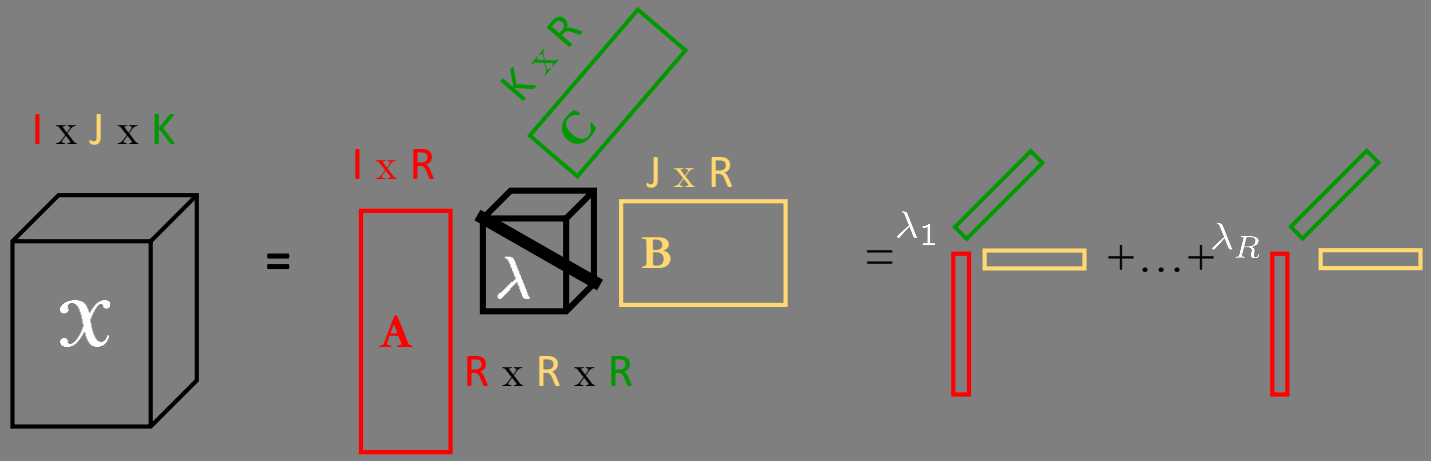
- Three-mode factor analysis (**3MFA/Tucker3**) [ Tucker, 1964+1966 ]
- Higher-order SVD (**HOSVD**) [ De Lathauwer et al., 2000a ]

## CP

- **PARAFAC** (parallel factors) [ Harshman, 1970 ]
- **CANDECOMP** (CAND) (canonical decomposition) [ Caroll & Chang, 1970 ]

Goal of Tucker model: Extension of SVD to >=3 modes

$$\mathbf{A} = \mathbf{U}\mathbf{W}\mathbf{V}^{\mathbf{T}} = \mathbf{W} \times_1 \mathbf{U} \times_2 \mathbf{V}$$




$$X = \lambda \times_1 A \times_2 B \times_3 C = \sum_{r=1}^{R} \lambda_r a_{:,r} \circ b_{:,r} \circ c_{:,r}$$

I x J x K tensor

$$x_{ijk} = \sum_{r=1}^{R} \lambda_r a_{ir} b_{jr} c_{kr}$$

# Tucker Model

## Based on N-mode SVD

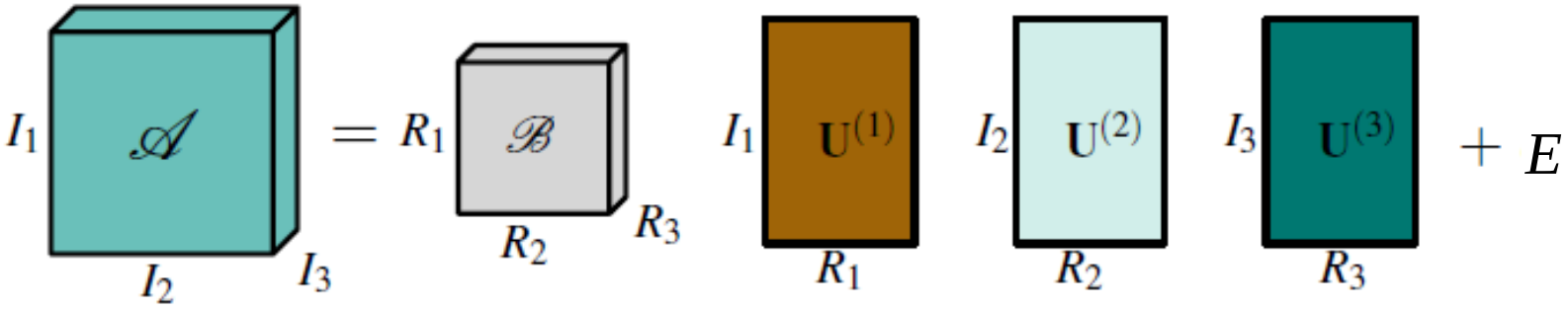
- Orthogonalizes each associated vectorspace
- For an order N tensor $\mathbf{A} \in {}^{I_1 \times \ldots \times I_n \times \ldots \times I_N}$:

$$A = Z \times_1 \mathbf{U}^{(1)} \times_2 \mathbf{U}^{(2)} \ldots \times_N \mathbf{U}^{(N)} + E \qquad Z \in {}^{R_1 \times \ldots \times R_N}$$

- In general the core tensor $Z$ is a full tensor!
- Mode matrices $\mathbf{U}_n \in {}^{I_n \times R_n}$ orthogonalize the column space of $A_{(n)}$

# Tucker Model

**N-mode SVD** $A = Z \times_1 \mathbf{U}^{(1)} \times_2 \mathbf{U}^{(2)} \ldots \times_N \mathbf{U}^{(N)} + E$




$\mathbf{A} \in I_1 \times \ldots \times I_n \times \ldots \times I_N$ $\quad Z \in R_1 \times \ldots \times R_N$ $\quad \mathbf{U}_n \in I_n \times R_n$

# Tensor Concepts

N-mode SVD algorithm for decomposition of A

1. For n=1,…,N compute SVD of mode-n flattening $\mathbf{A_{(n)}}$

2. Set

$$\mathbf{A}_{(n)} = \mathbf{U}_n \mathbf{W}_n \mathbf{V}_n^{\mathbf{T}}$$

$$Z = A \times_1 \mathbf{U}_1^T \times_2 \mathbf{U}_2^T \ldots \times_N \mathbf{U}_N^T$$

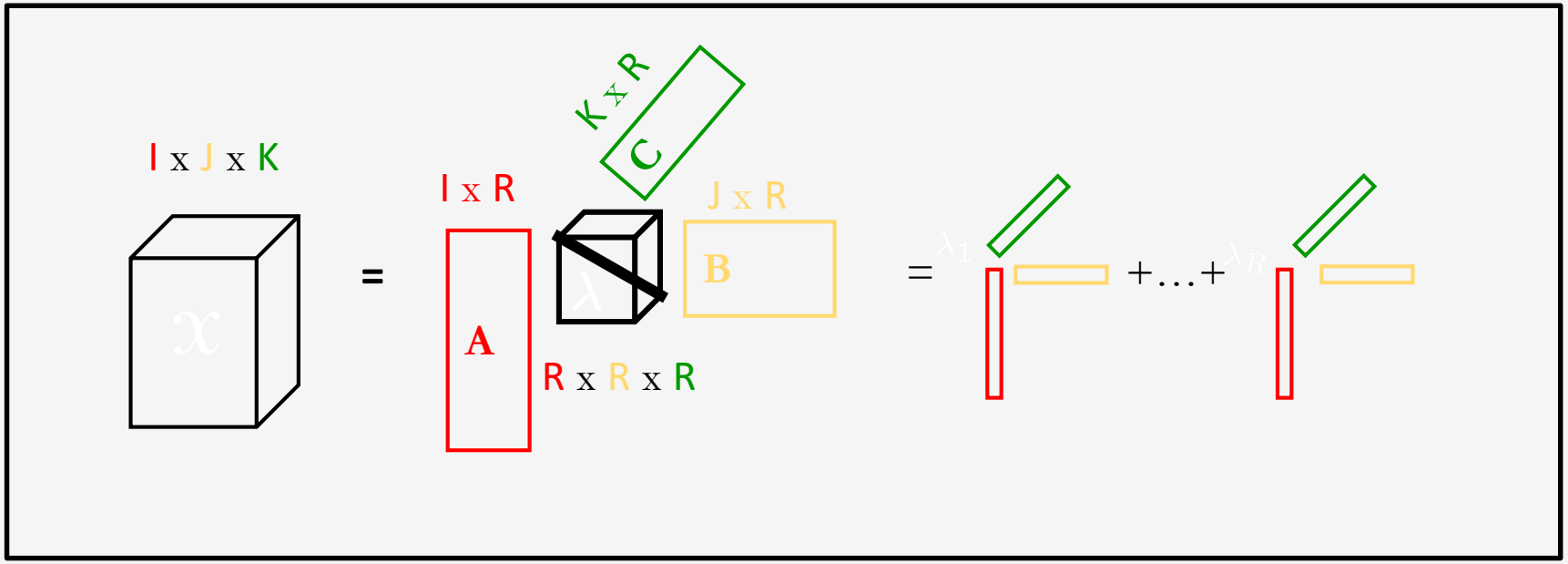
# PARAFAC/Candecomp Decomposition

PARAFAC = **Para**llel **Fac**tors (Harshman, 1970)

Candecomp = **Can**onical **Decomp**osition (Carol & Chang, 1970)

Columns of **A**, **B**, and **C** are <u>not</u> orthonormal

If **R** is *minimal*, then **R** is called the **rank** of the tensor (Kruskal 1977)

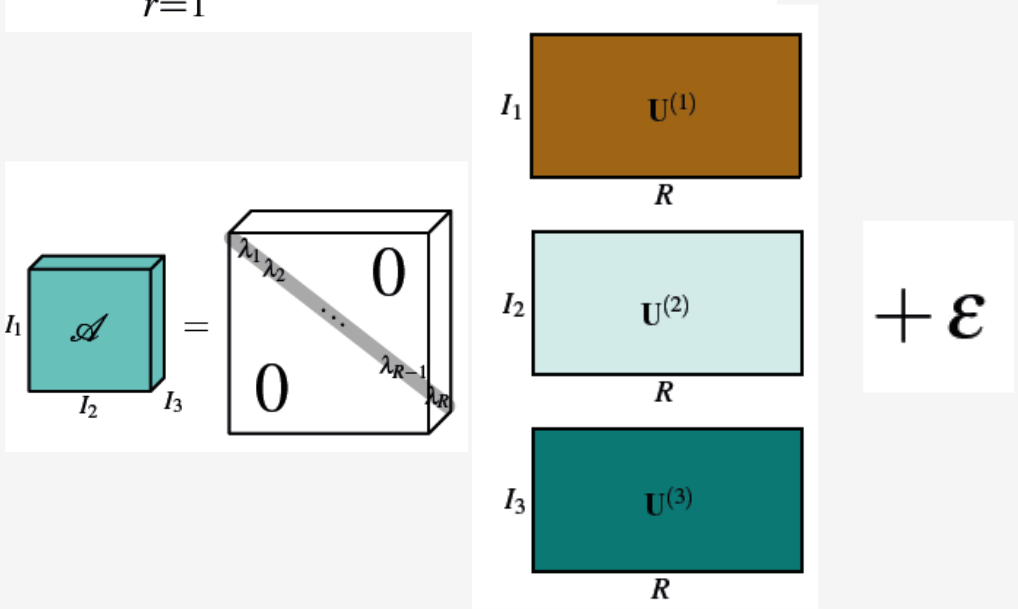


# PARAFAC Decomposition

Higher order tensor **A** factorized into a sum of rank-one tensors

- normalized column vectors $\boldsymbol{u}_r^{(n)}$ define factor matrices $\mathbf{U}(n) \in \mathbb{R}^{In \times R}$ and weighting factors $\lambda_r$

$$\mathscr{A} = \sum_{r=1}^{R} \lambda_r \cdot \boldsymbol{u}_r^{(1)} \circ \boldsymbol{u}_r^{(2)} \circ \ldots \boldsymbol{u}_r^{(N)} + \varepsilon$$



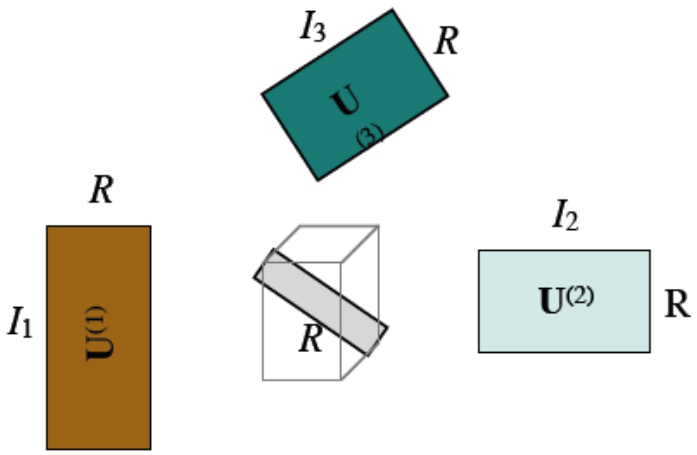
# PARAFAC Decomposition

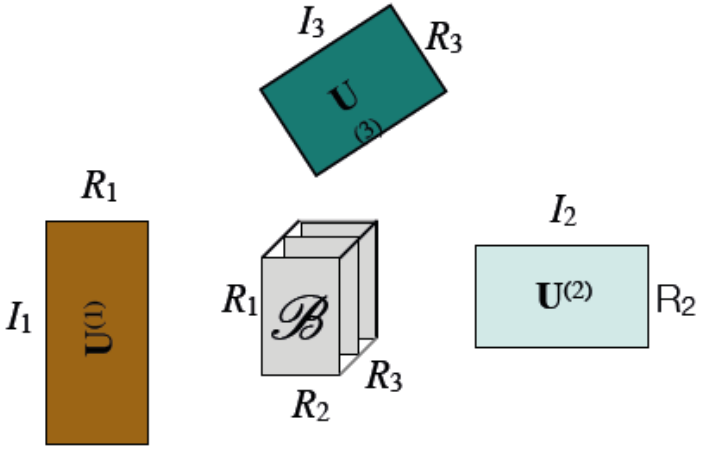
- Good Tensor Decompositions can be found via

  - *Alternating Least Squares*
    - Iterate over the tensor modes *i*
    - Keep all but the current mode *i* fixed
    - Solve a least squares problem

- Currently no known algorithm for optimal global solution !

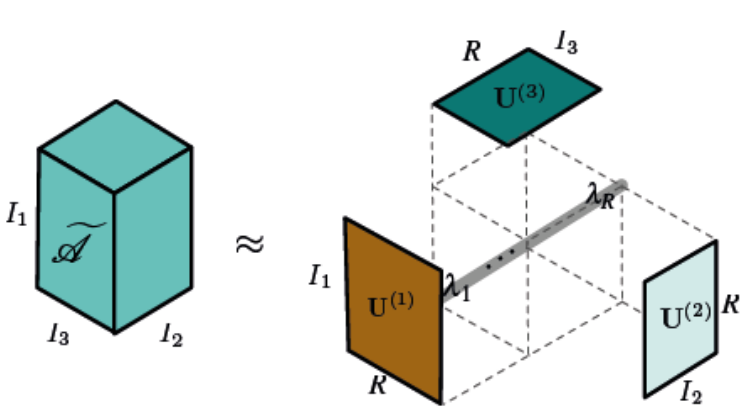
# CP a special case of Tucker

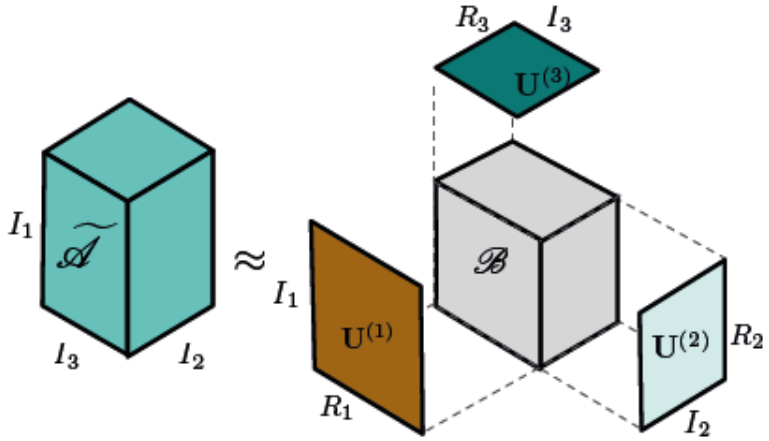



CP

Tucker



# Rank Trunctation

SVD allows for progressive rank truncation

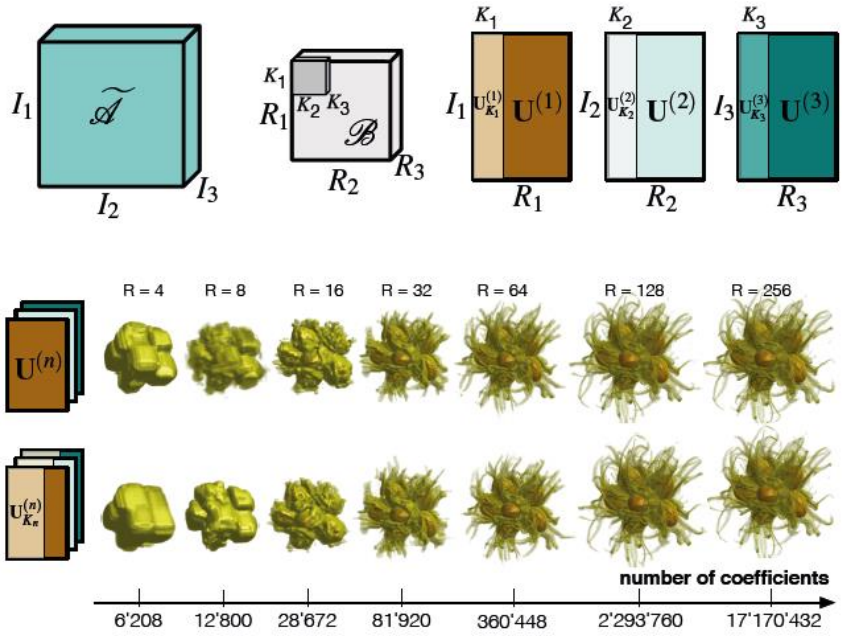
- orthogonality of singular vectors
- order of increasing singular values

CP does not exhibit good progressive truncation behavior

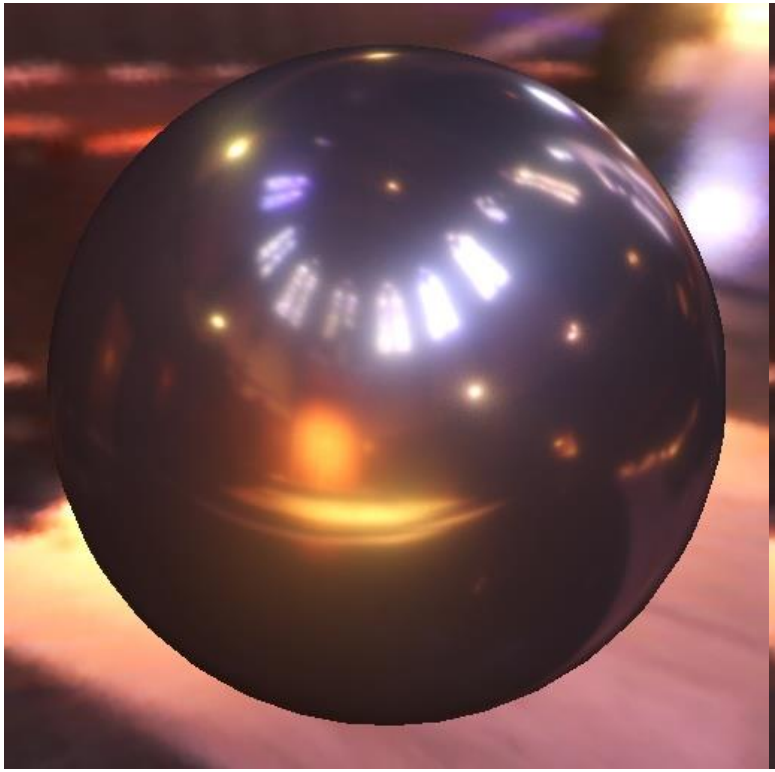
- non-orthogonal factor matrices

All-orthogonal Tucker model supports progressive truncation

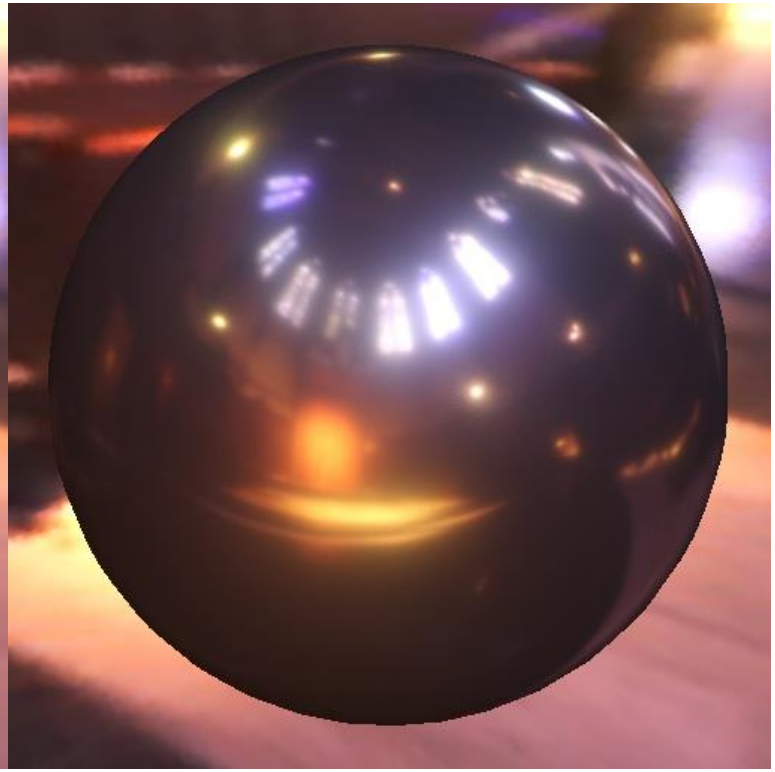
- does not necessarily give best possible progression



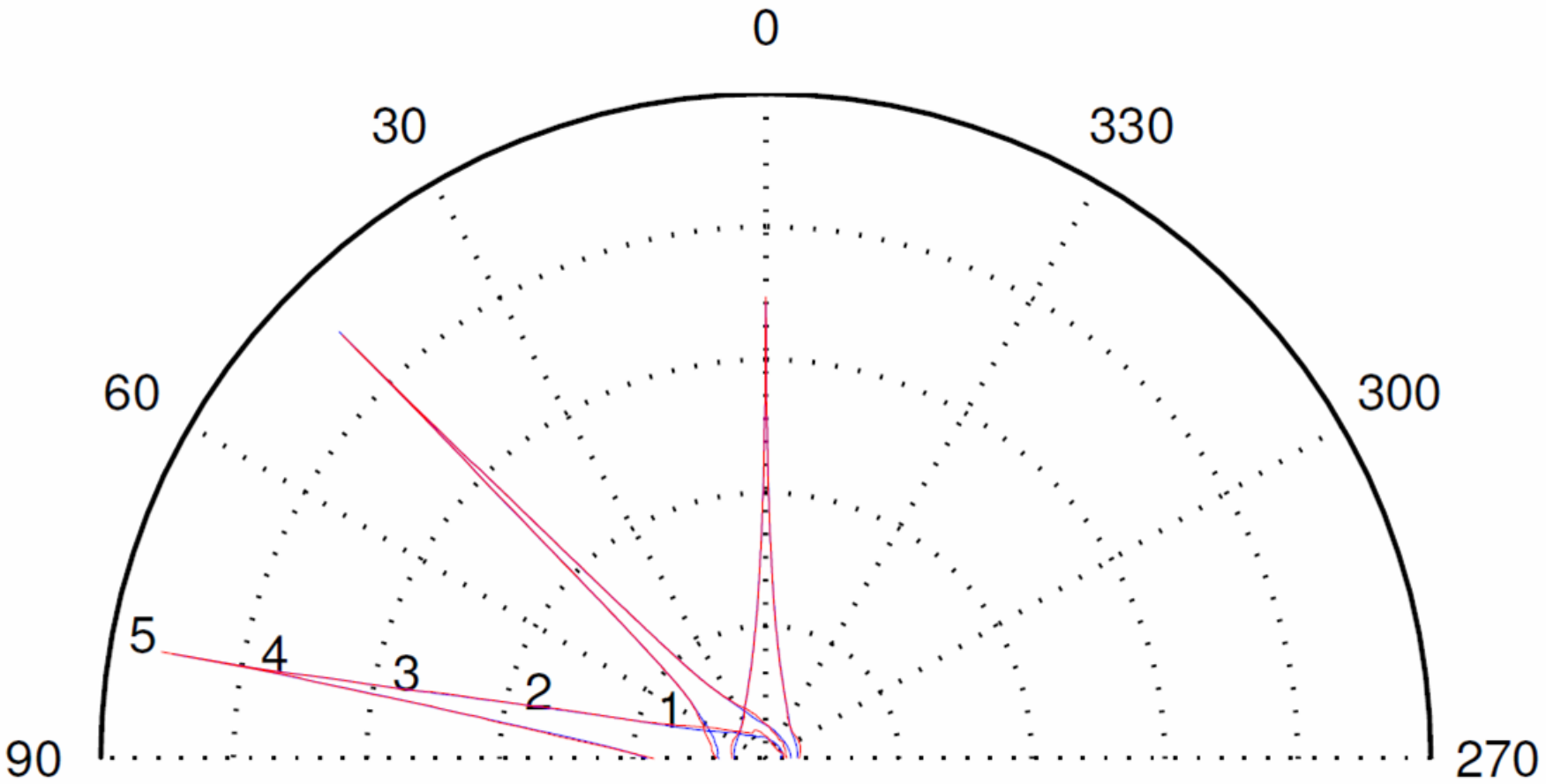
# Tensor Decomposition



Acquired data – aluminium
16.5MB



Tensor decomposition
11.5 KB




Fitting result (red original, fitted blue)

# Tensor Approximation

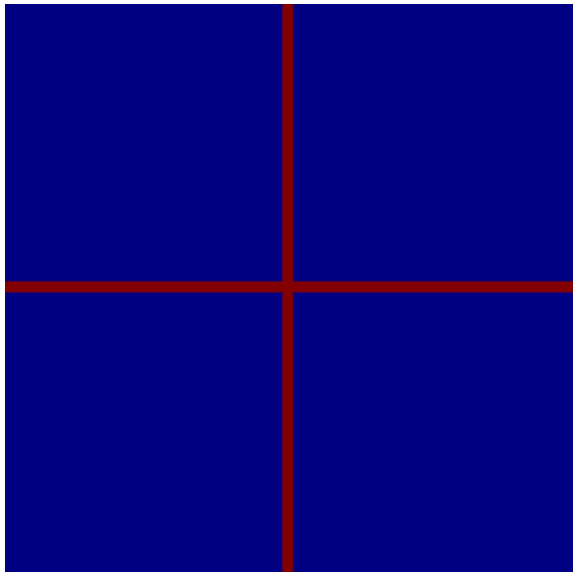
Several important questions have to be considered:

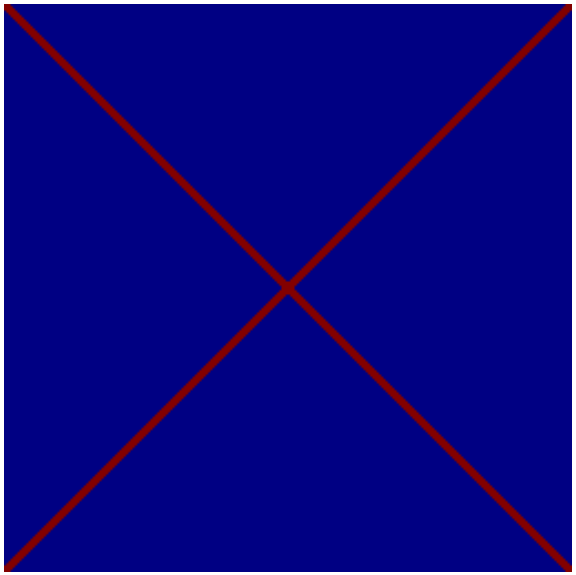
- Which parameterization?
  - Is my input data registered correctly?
- Which error measure?
- Which decomposition?
- Should every dimension be represented in an individual mode?

# Parameterization

Why is the parameterization of our function important?

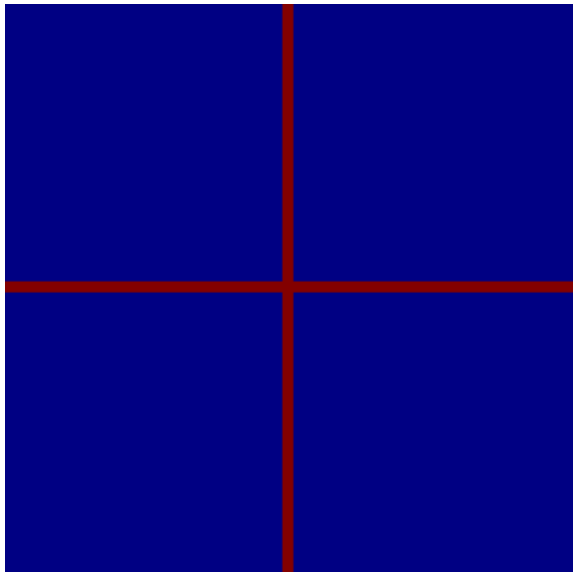
Lets consider two simple test cases (256x256 matrix with 0/1 values):



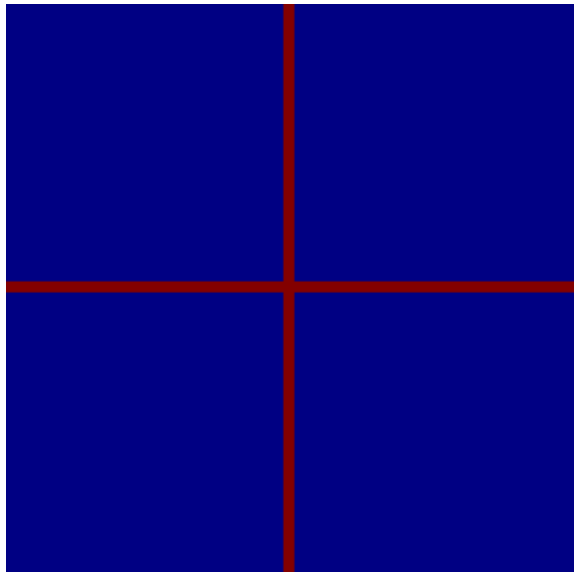


# Parameterization

The first case can be approximated easily:



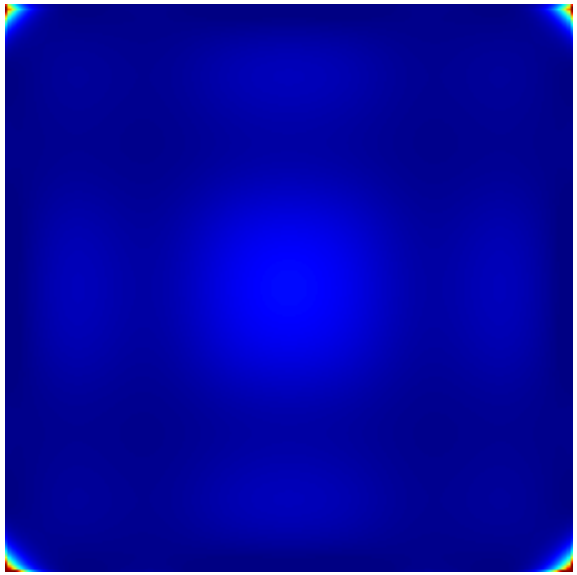
**CP Decomposition**
with 2 components



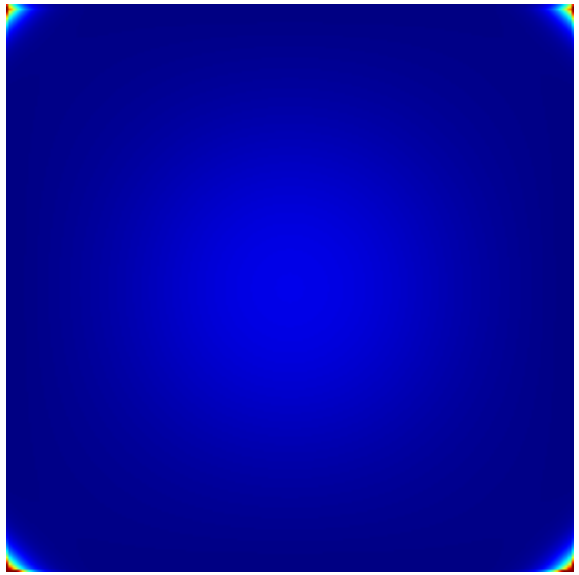
**TUCKER Decomposition**
with 2x2 core tensor

# Parameterization

But the second case is far more difficult:



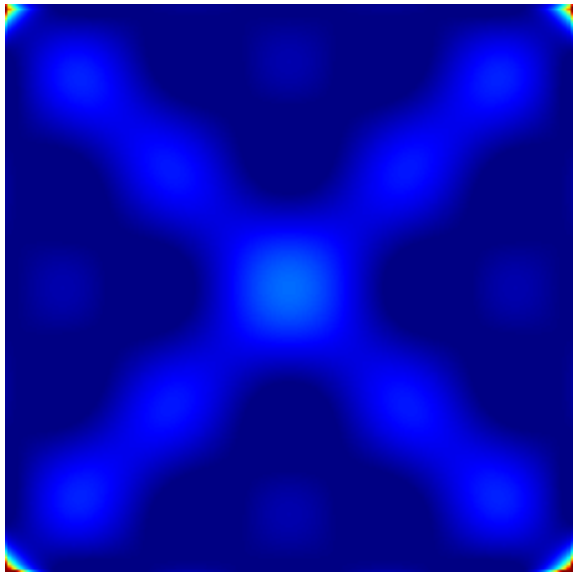
**CP Decomposition**
with 2 components



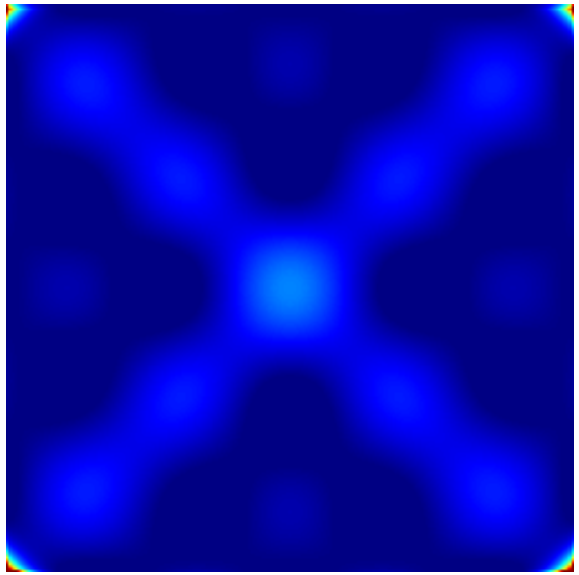
**TUCKER Decomposition**
with 2x2 core tensor

# Parameterization

But the second case is far more difficult:



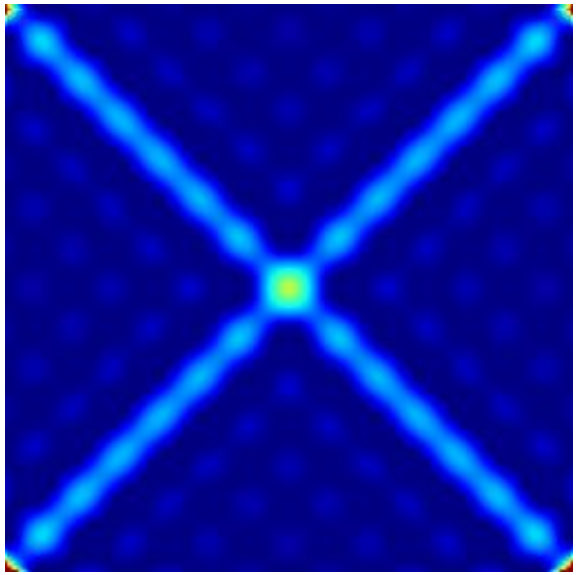
**CP Decomposition**
with 4 components



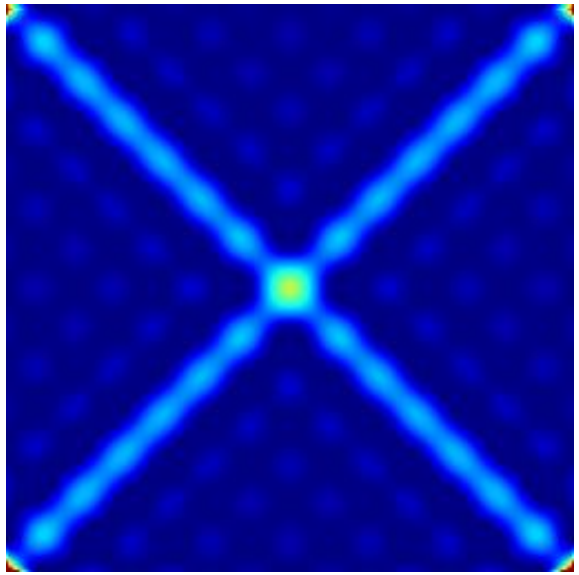
**TUCKER Decomposition**
with 4x4 core tensor

# Parameterization

But the second case is far more difficult:



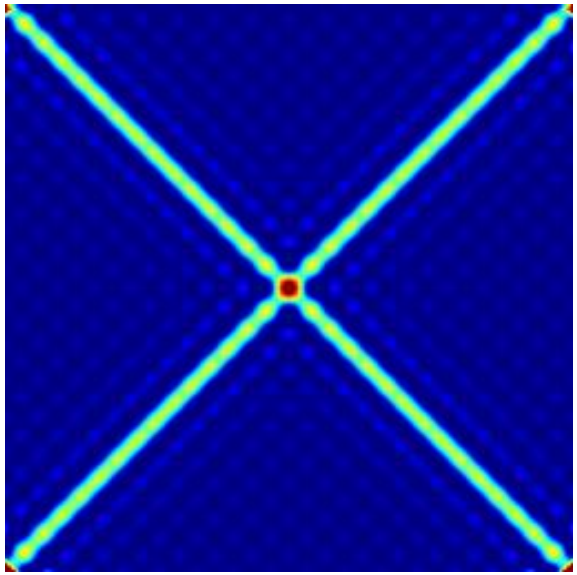
**CP Decomposition**
with 8 components



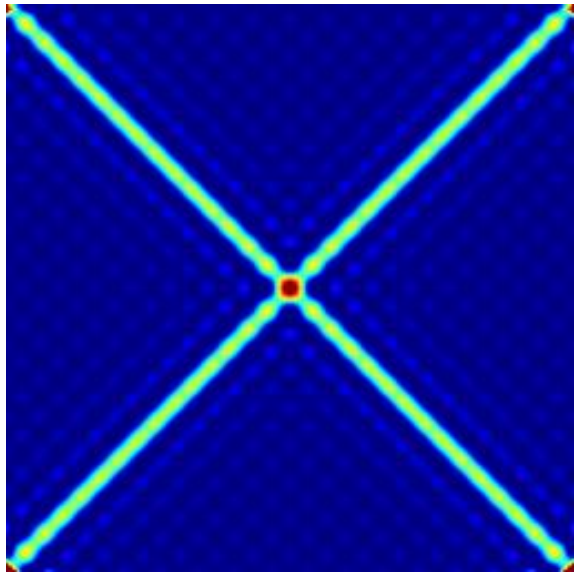
**TUCKER Decomposition**
with 8x8 core tensor

# Parameterization

But the second case is far more difficult:



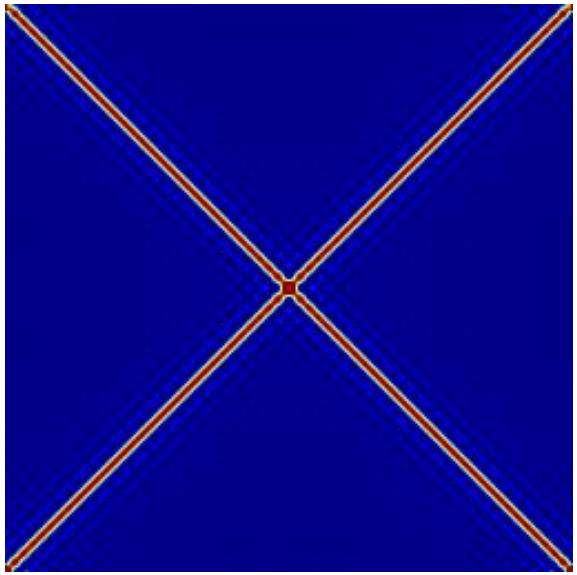
**CP Decomposition**
with 16 components



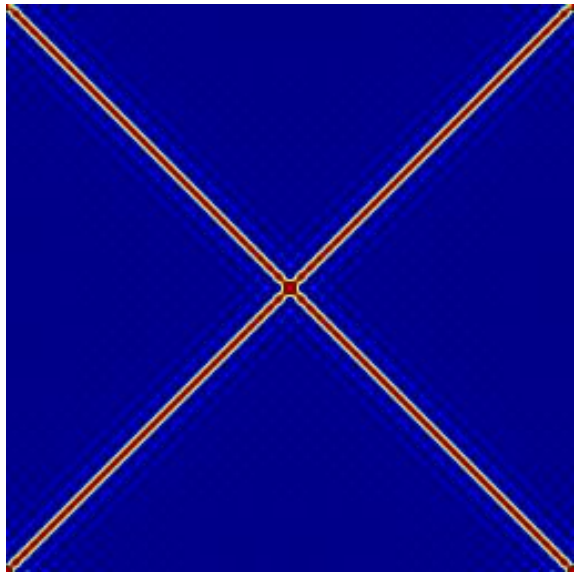
**TUCKER Decomposition**
with 16x16 core tensor

# Parameterization

But the second case is far more difficult:



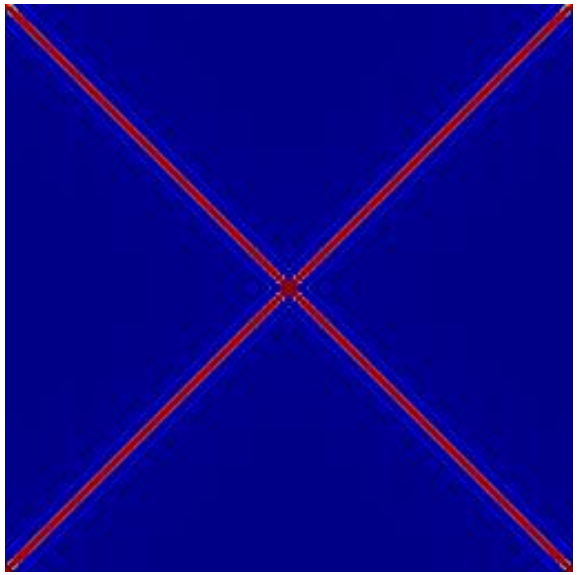
**CP Decomposition**
with 32 components



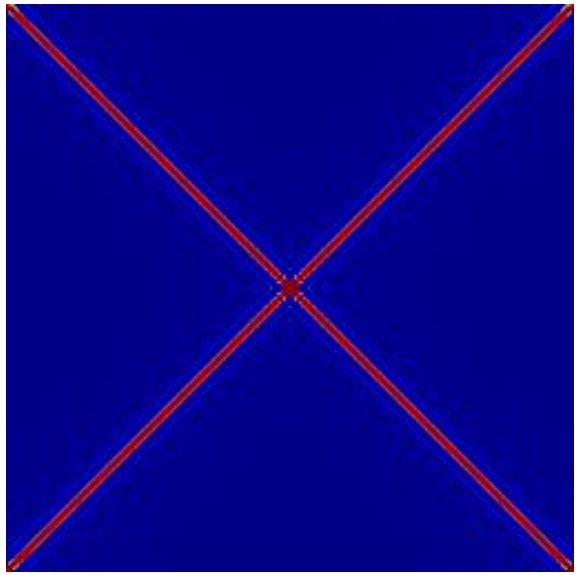
**TUCKER Decomposition**
with 32x32 core tensor

# Parameterization

But the second case is far more difficult:



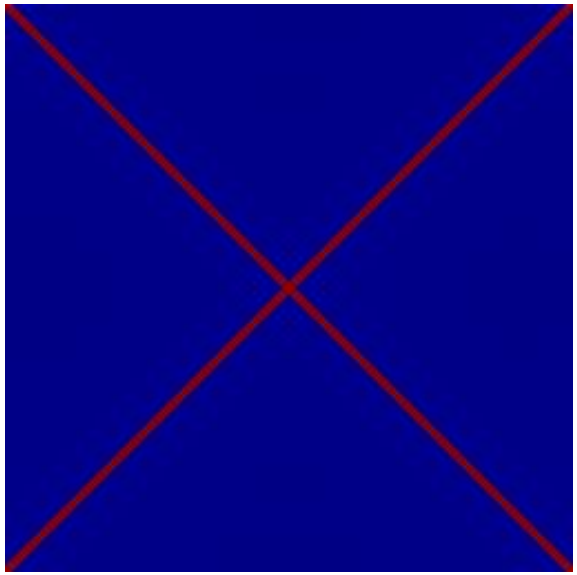
**CP Decomposition**
with 64 components



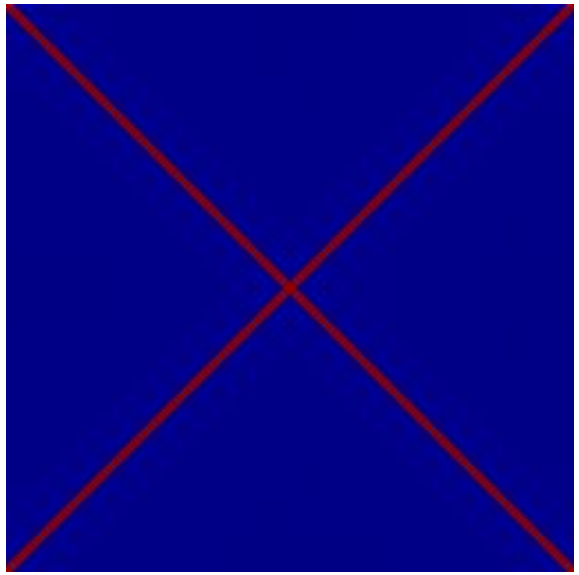
**TUCKER Decomposition**
with 64x64 core tensor

# Parameterization

But the second case is far more difficult:



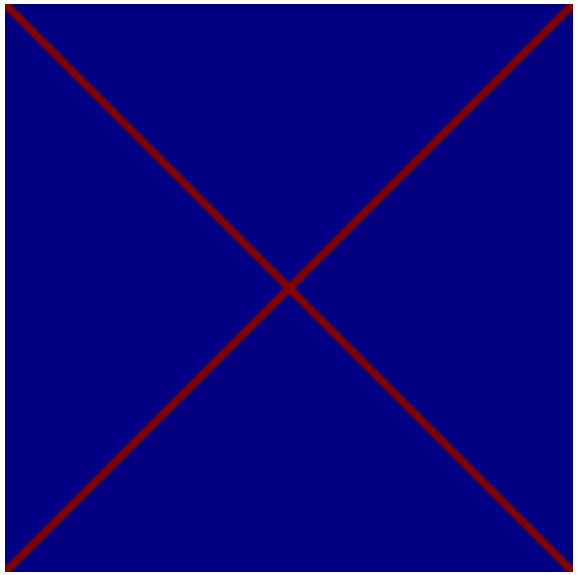
**CP Decomposition**
with 100 components



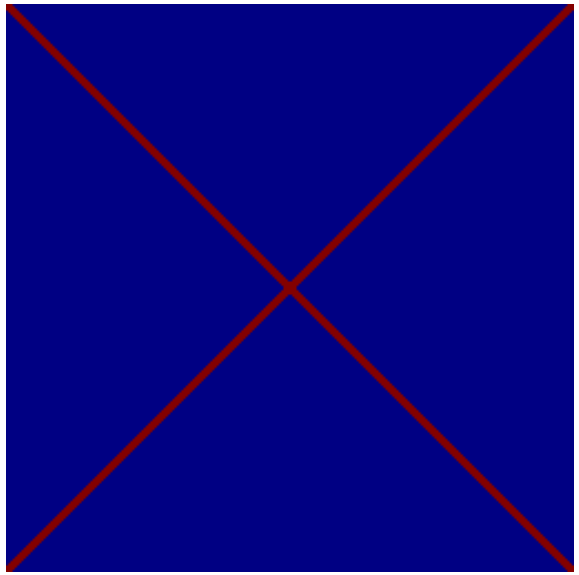
**TUCKER Decomposition**
with 100x100 core tensor

# Parameterization

But the second case is far more difficult:



**CP Decomposition**
with 128 components

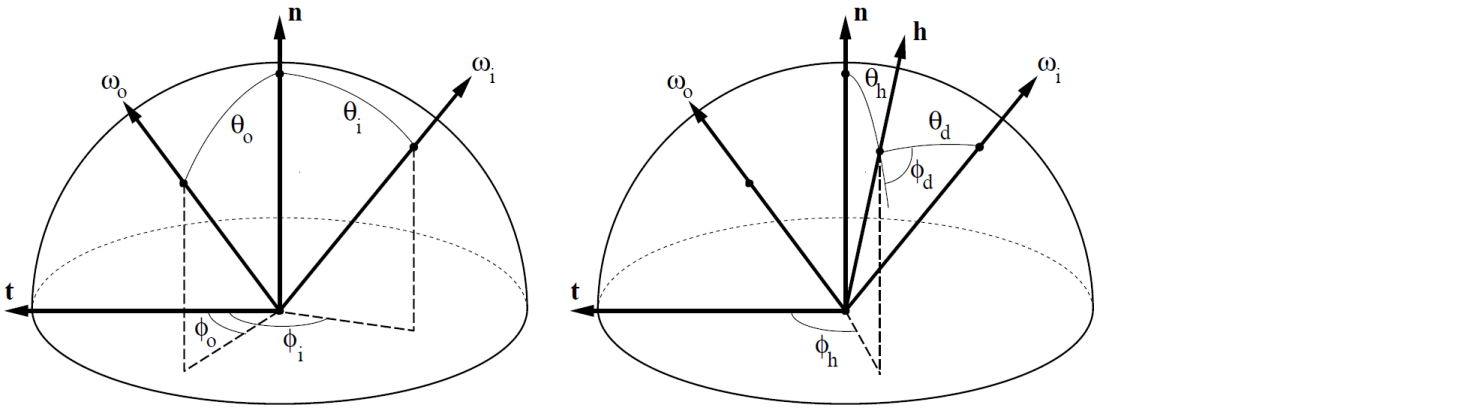


**TUCKER Decomposition**
with 128x128 core tensor

# Half-Diff Parameterization

Parameterization of BRDF via incoming and outgoing direction not well suited

- Better alternative via a halfway and a difference vector has been proposed in [Rusinkiewicz-1998]

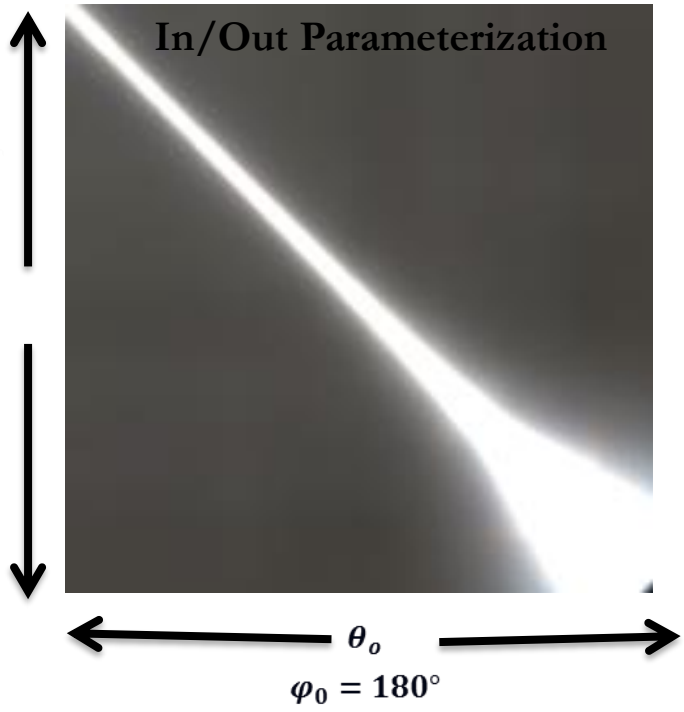


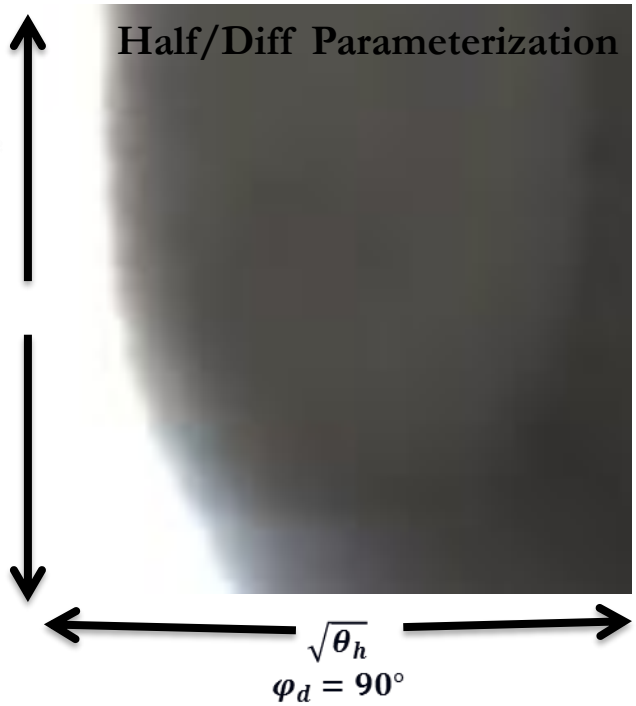

**In/Out Parameterization** **Half/Diff Parameterization**

Image from [Rusinkiewicz-1998]

# Half-Diff Parameterization for tensor-decomposition

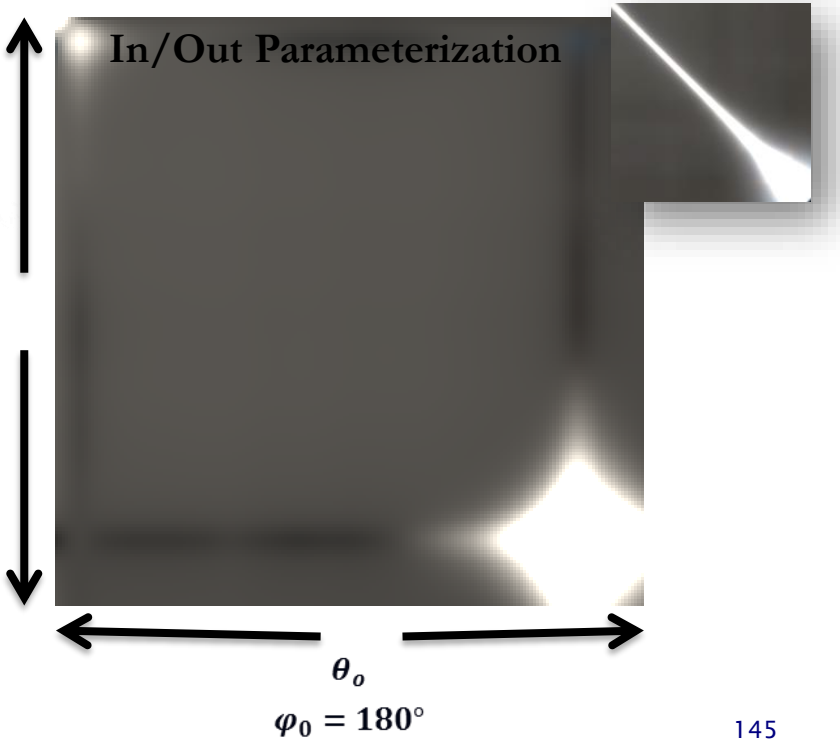
Comparison of two slices through the Mode-3 tensor of an isotropic BRDF

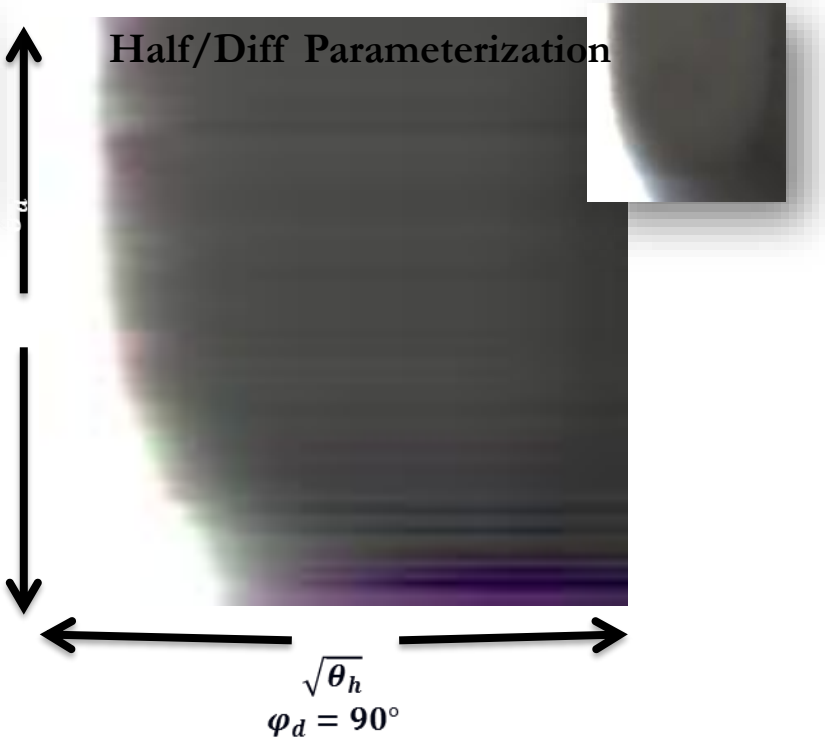


# Half-Diff Parameterization

CP approximation of the tensor with 6 components



The difference is also clearly visible in renderings:

Uncompressed BRDF

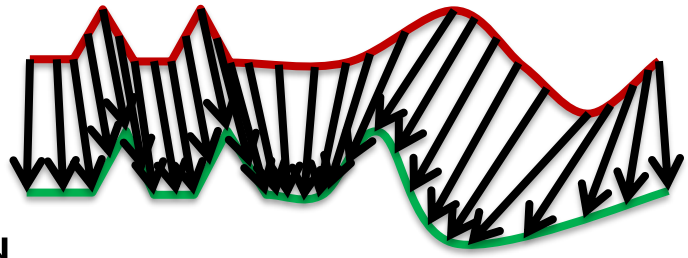
In/Out Parameterization

Half/Diff Parameterization

# Registration

Correlations can only be exploited, if corresponding features are aligned with each other

- The input data has to be registered correctly!



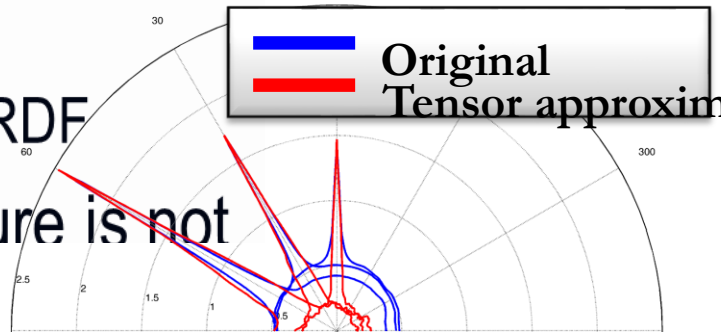
Registration of two functions via Dynamic Time Warping

Depending on the data-type different types of registration can be employed, e.g.

| | |
|---|---|
| **Geometry** | Rigid alignment, Non-Rigid alignment,<br>Reparameterization of the surface |
| **Motion Data** | Dynamic Time warping |
| **Images, Volumetric Data** | Rigid registration, Warping |
| **BTFs** | Alignment of local coordinate systems,<br>Good choice of reference plane,<br>Parallax correction via reference geometry |

# Error Measure

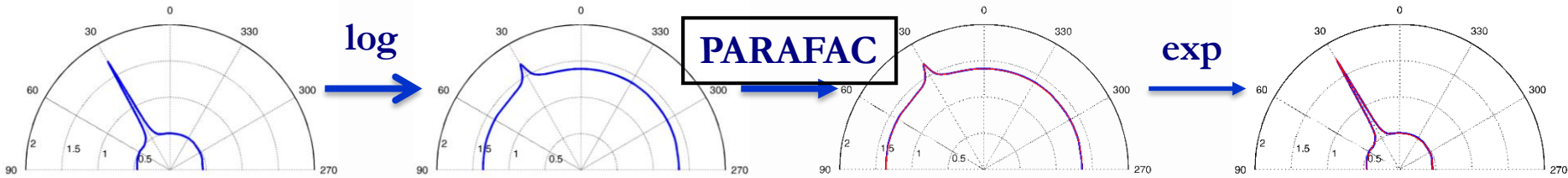
- Some datasets have a very high dynamic range
  - Example: BRDFs can exhibit a dynamic range of 10,000:1
- Errors in parts with small values can still be perceptually relevant
  - Example: diffuse component of a BRDF
- In these cases the $\ell^2$ error measure is not




Fourth root was applied to the

# Dynamic Range Reduction

- Reduce dynamic range by applying transformation to the data prior to tensor decomposition
  - E.g. $\mathbf{log}(x)$ was used for BRDFs in [Bilgili-2011]
    - Other functions like **roots** or **sigmoid functions** could also be used
  - Has to be inverted after decompression
  - Decomposition is no longer linear
    - Can be a problem in applications, where a linear decomposition is needed
      - For example, in [Sun-2007], the Tucker Decomposition is used to create a linear basis for BRDFs




**Fourth root was applied to the plots!**

# Relative Error via Per-Element Weights

- Employ a different error metric during the optimization
  - Only $\ell^2$ errors can be minimized efficiently via ALS
  - **Per-element** weights $w$ can be included into the approximation
    - Can be used to minimize relative errors:

($x$ original value, $\tilde{x}$ approximation)

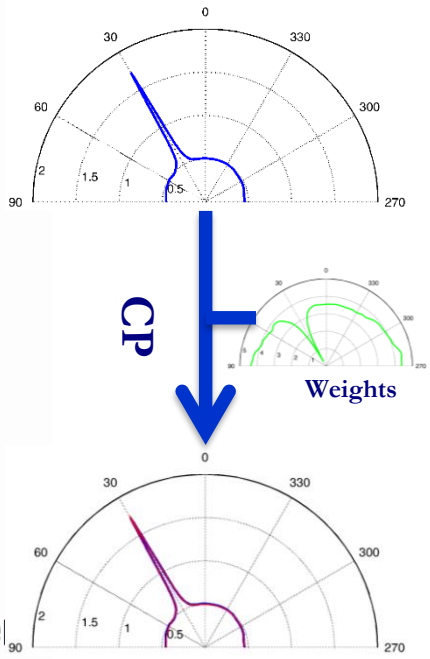
**Squared error relative to original value**

$$\frac{|x-\tilde{x}|^2}{|x|} = w|x-\tilde{x}|^2 \quad \text{with } w = \frac{1}{|x|}$$

**Square of the relative error**

$$\frac{|x-\tilde{x}|^2}{|x|^2} = w|x-\tilde{x}|^2 \quad \text{with } w = \frac{1}{|x|^2}$$

- Decomposition remains linear and no inversion is necessary after decompression
- Additional weights can be used to compensate for the irregular sampling, cosine $\theta_i$ fall-off, reliability of the input data etc.




**Fourth root was applied to the plots**

# Error Measure (comparison)

**Original**

**$\ell^2$ Error**

$$|x - \tilde{x}|^2$$

**Log error**

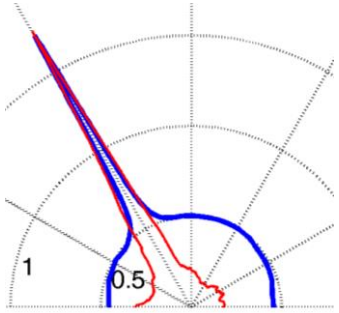
$$|\log(x) - \log(\tilde{x})|^2$$

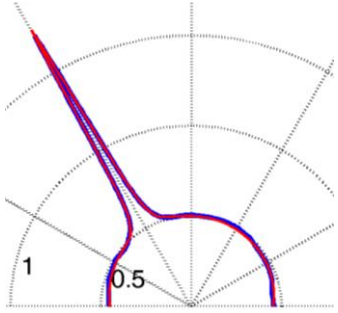
**Squared Error relative to original value**

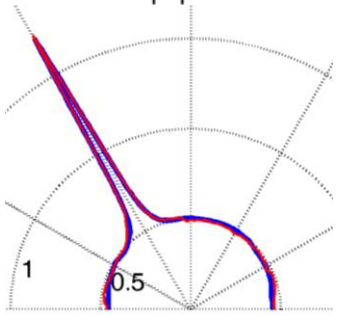
$$\frac{|x - \tilde{x}|^2}{|x|}$$

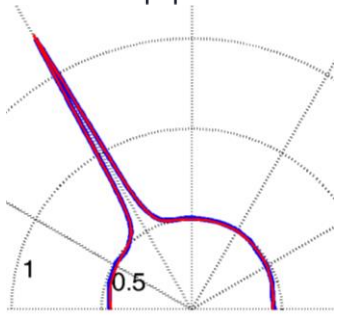
**Square of the relative error**

$$\frac{|x - \tilde{x}|^2}{|x|^2}$$




**Fourth root was applied to the plots!**

# BRDF Compression Results

**Uncompressed**

**Compressed**

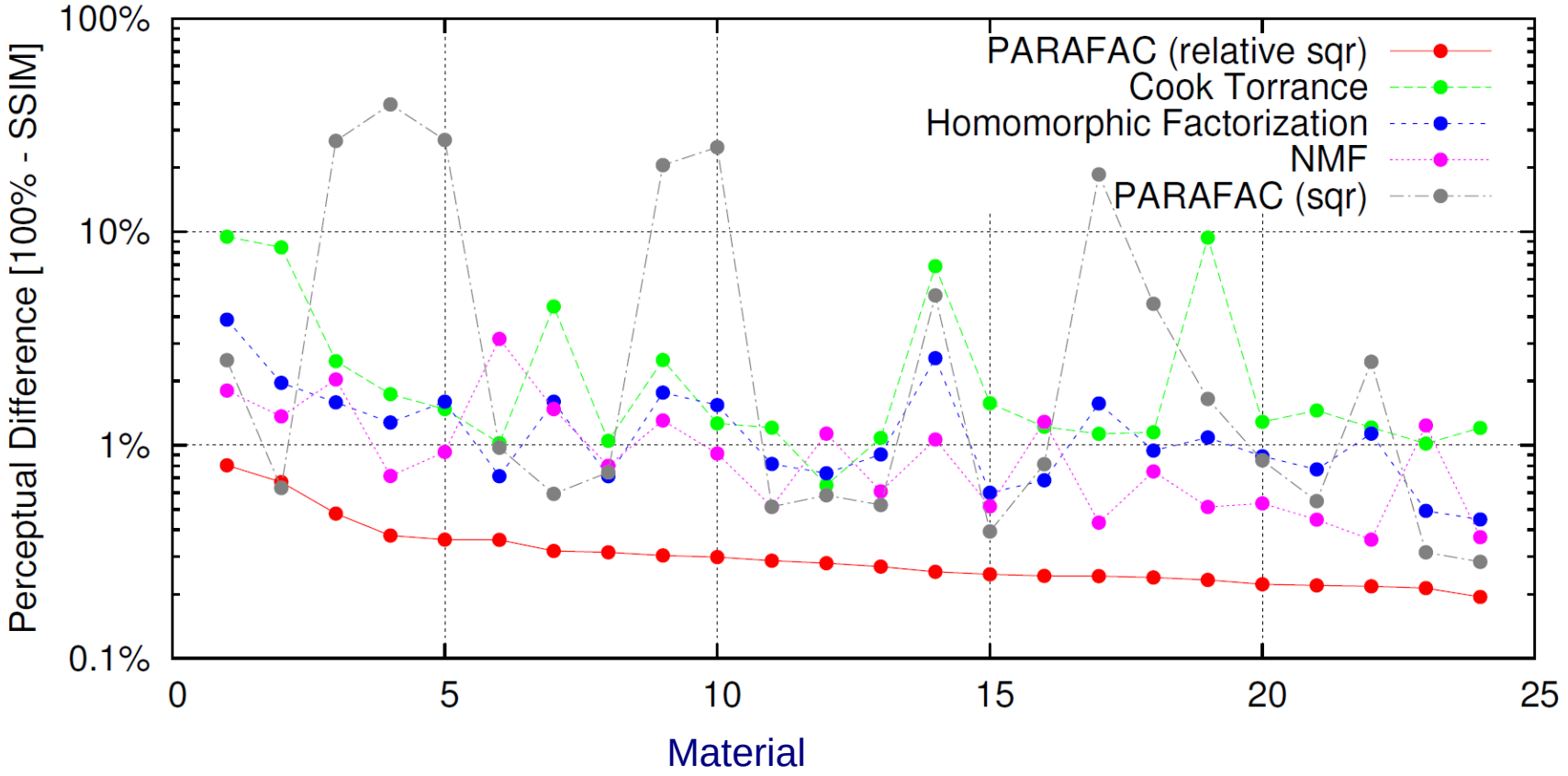
**CP Compression**

Components: **8**
Original: **33 MB**
Compressed: **23 KB**
Ratio: **≈1500:1**
E. Measure: $\frac{|x - \tilde{x}|^2}{|x|}$

Additional weights to compensate for irregular sampling and for $\cos\theta_i$ and $\cos\theta_o$

Results from [Ruiters-2010]

# BRDF Compression Results




Results from [Ruiters-2010]

# Which Decomposition to use?

## Tucker Decomposition

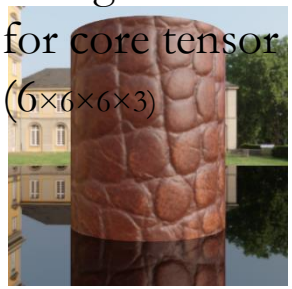
Potentially better compression ratios

- Only when the core tensor is small and not too sparse
  - Size of core tensor increases as the product of the reduced ranks
- Flexibility: user can choose the rank for each mode individually

Random access very expensive for large core-tensors

- Summation over all entries of the core tensor necessary.

23% of storage for core tensor (6×6×6×3)



99% of storage for core tensor (28×28×128×128)

$$\mathcal{T}_{i_1,\ldots,i_n} = (\boldsymbol{\mathcal{C}} \times_1 \boldsymbol{U}^{(1)} \times_2 \cdots \times_n \boldsymbol{U}^{(n)})_{i_1,\ldots,i_n} = \sum_{j_1} U^{(1)}_{i_1 j_1} \sum_{j_2} U^{(2)}_{i_2 j_2} \cdots \sum_{j_n} U^{(n)}_{i_n j_n} \mathcal{C}_{j_1,\ldots,j_n}$$

# Which Decomposition to use?

**CANDECOMP/PARAFAC Decomposition**

Sparse core tensor: diagonal structure

More columns in the factor matrices needed

Random access usually less expensive:

$$\mathcal{T}_{i_1,\ldots,i_n} = \left(\sum_{j=1}^{C} \sigma_j \circ \boldsymbol{v}_j^{(1)} \circ \cdots \circ \boldsymbol{v}_j^{(n)}\right)_{i_1,\ldots,i_n} = \sum_{j=1}^{C} \sigma_j v_{i_1,j}^{(1)} \cdots v_{i_n,j}^{(n)}$$

## Alternatives

Hierarchical Tensor Approximation

- Possibly faster decompression
- More compact  compression for data with multi-resolution decomposition

Clustered Tensor Approximation / Sparse Tensor Decomposition

- Reduction of decompression cost via clustering
- More compact when the underlying data can be clustered well

# Spatially varying BRDFs

Represent SVBRDF as a sum of separable functions (Parafac/Candecomp)

- Faithful representation possible
- Compact
- Suitable for real time rendering

Fit SVBRDF directly against the irregular samples

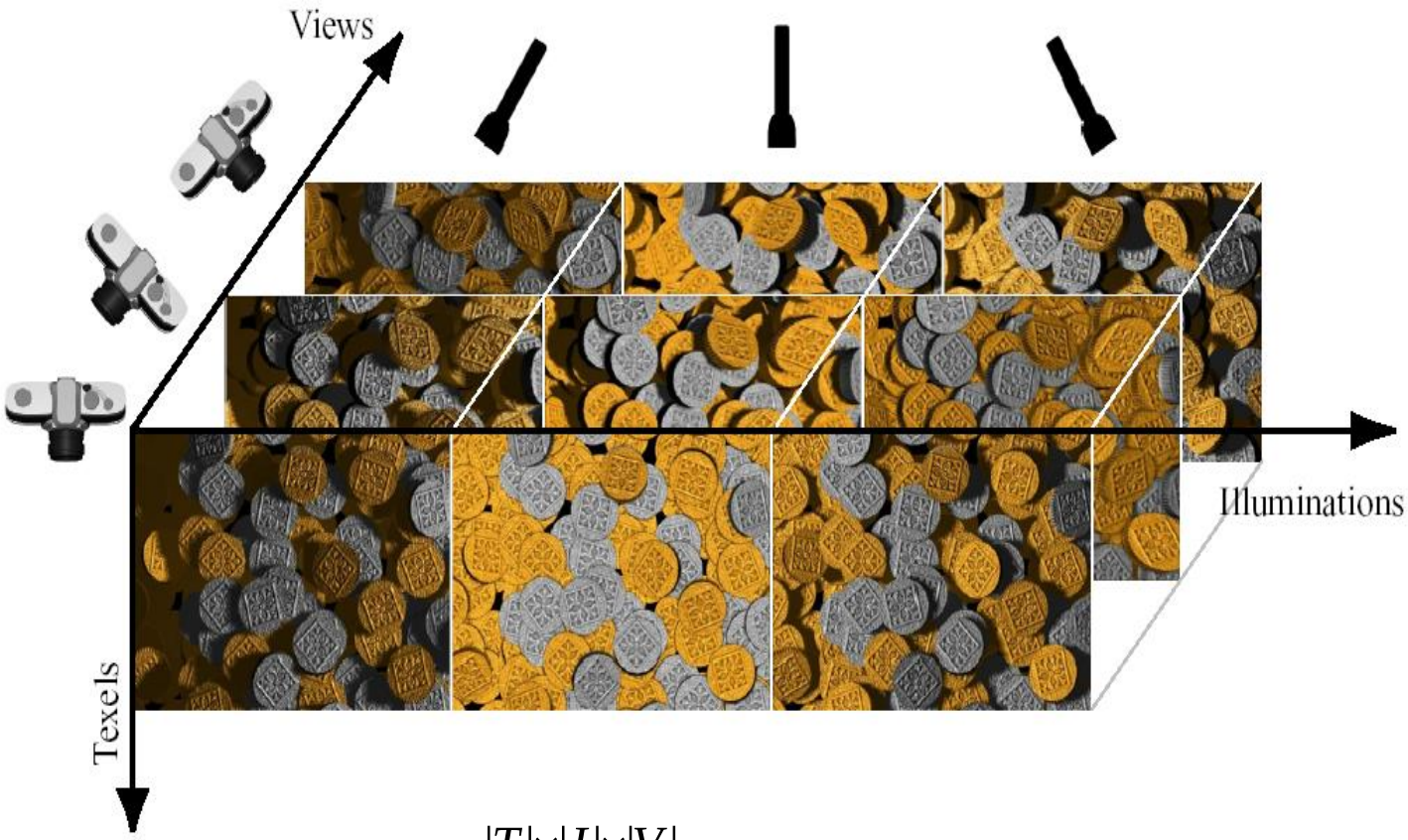
- Only work on the compressed representation
  - No resampling to regular grid
  - Full densely sampled dataset has never to be stored

Regularization

- Enforce angular smoothness
- Non-local spatial regularization
  - Exploit self-similarity of most materials

*Roland Ruiters, Christopher Schwartz, and Reinhard Klein, Data Driven Surface Reflectance from Sparse and Irregular Samples, Forum (Proc. of Eurographics) (May 2012), 31:2(315-324)*

# Multi-Linear Analysis of Images



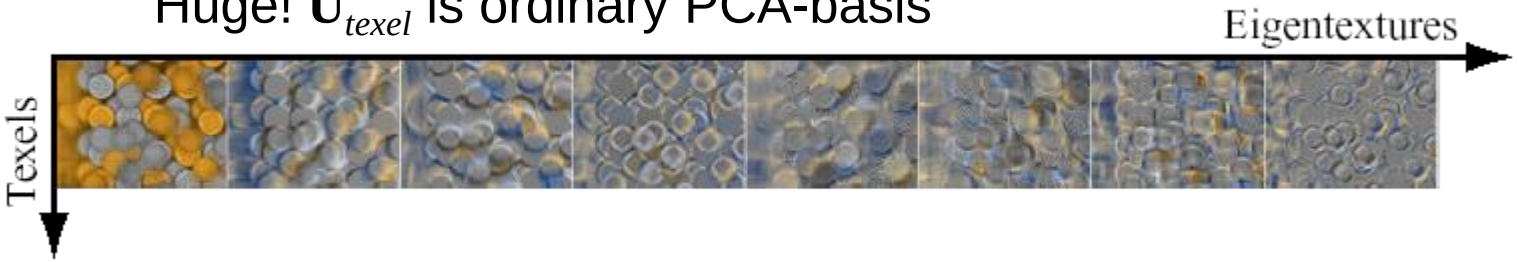

$$A \in |T| \times |I| \times |V|$$

# Multi-Linear Analysis of Images

N-mode SVD:

$$A \doteq Z \times_1 \mathbf{U}_{texel} \times_2 \mathbf{U}_{illum} \times_3 \mathbf{U}_{view}$$

Huge! $\mathbf{U}_{texel}$ is ordinary PCA-basis



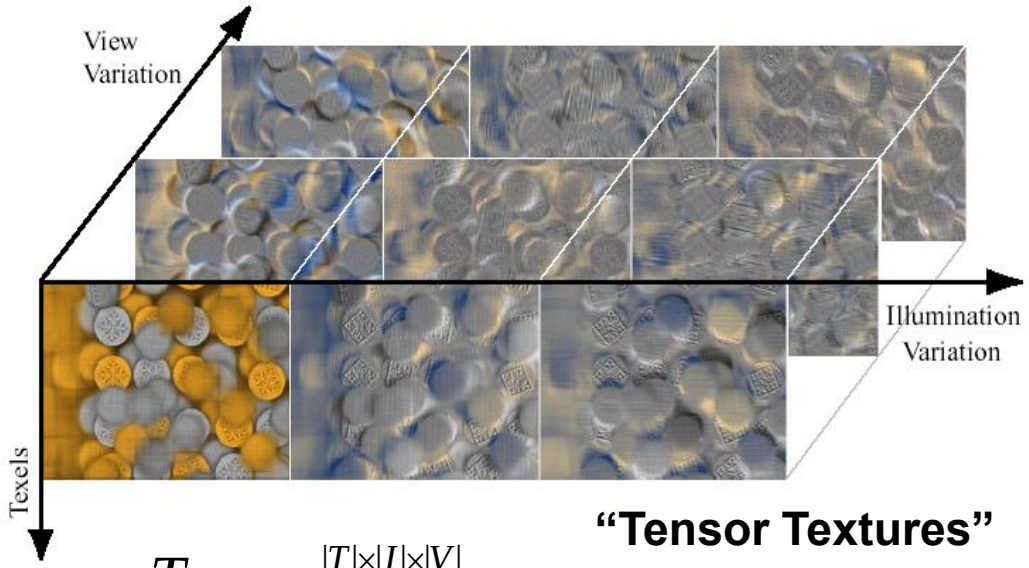

$\mathbf{U}_{illum}$, $\mathbf{U}_{view}$ encode texel and view (lighting) independent lighting (view) variation

# Multi-Linear Analysis

Orthogonalize only mode-2, mode-3 !

$$A = Z \times_1 \mathbf{U}_{texel} \times_2 \mathbf{U}_{illum} \times_3 \mathbf{U}_{view}$$

$$= T \times_2 \mathbf{U}_{illum} \times_3 \mathbf{U}_{view} \; ?$$




**“Tensor Textures”**

$$T \in |T| \times |I| \times |V|$$

# Multi-Linear Analysis

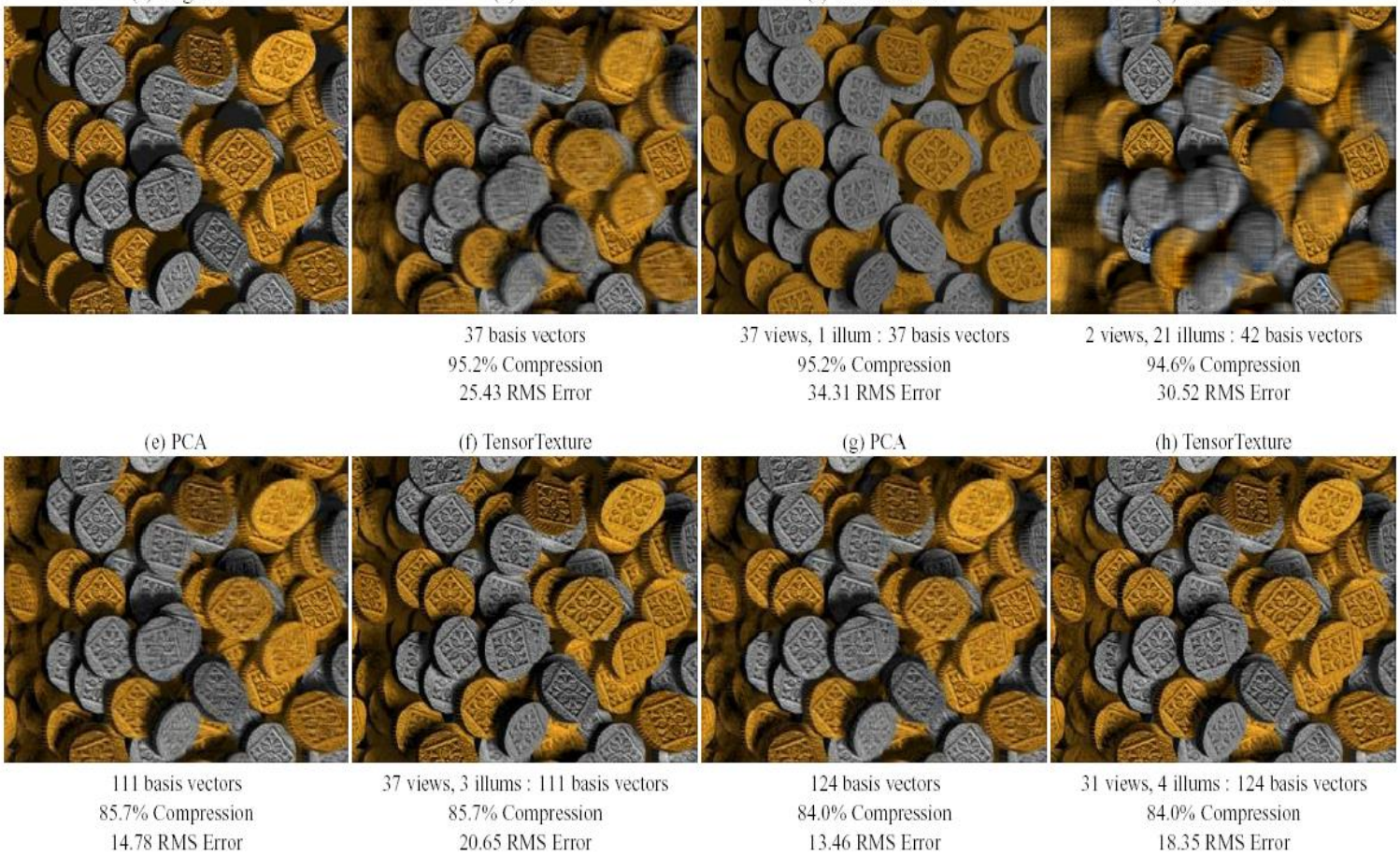
Advantages:

- SVD of $\mathbf{A}_{(2)}, \mathbf{A}_{(3)}$ can be computed fast

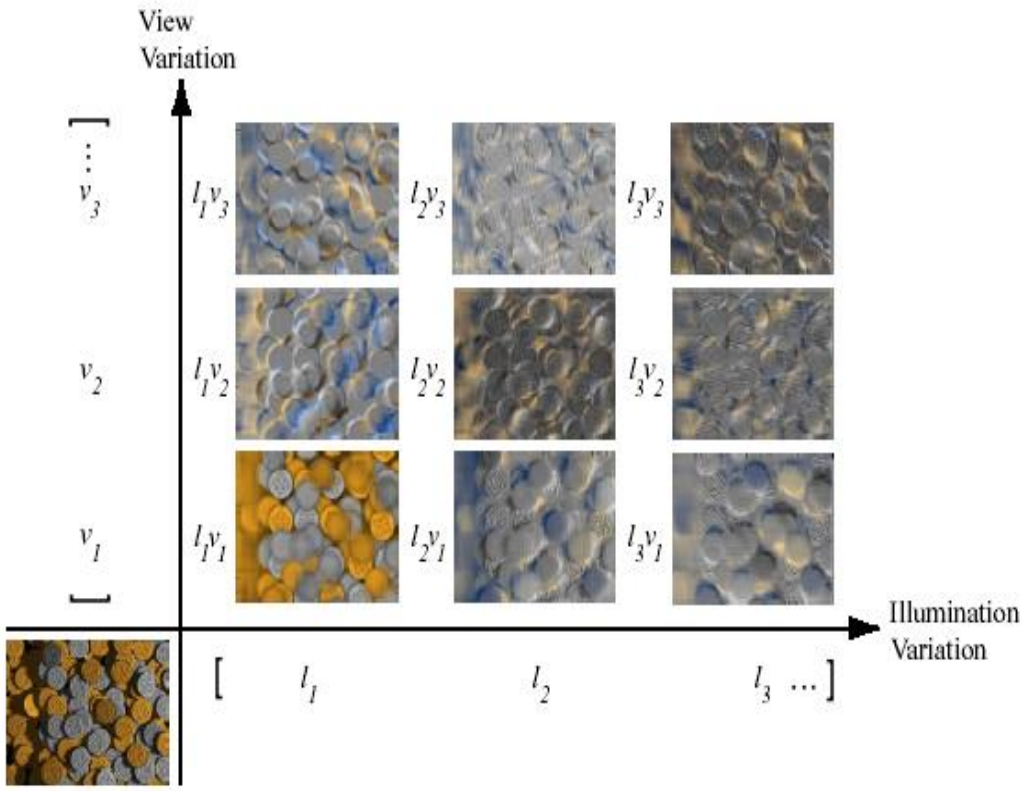
$$\mathbf{A}\mathbf{A}^T = \mathbf{U}\mathbf{S}^2\mathbf{U}^T,\ \mathbf{V}^T = \mathbf{S}^+\mathbf{U}^T\mathbf{A}$$

- $|I|+|V|$ instead of $|I||V|$ coefficients for each basis image
- Strategic dimensionality reduction by truncating $\mathbf{U}_{illum}$, $\mathbf{U}_{view}$ independently
  (=>refine T ' by iterative optimization)

# Multi-Linear Analysis



# Multi-Linear Rendering




Interpolation for novel directions

# Results


Dataset: 37 view, 21 illu dirs

Kept 37x11=407 tensortextures

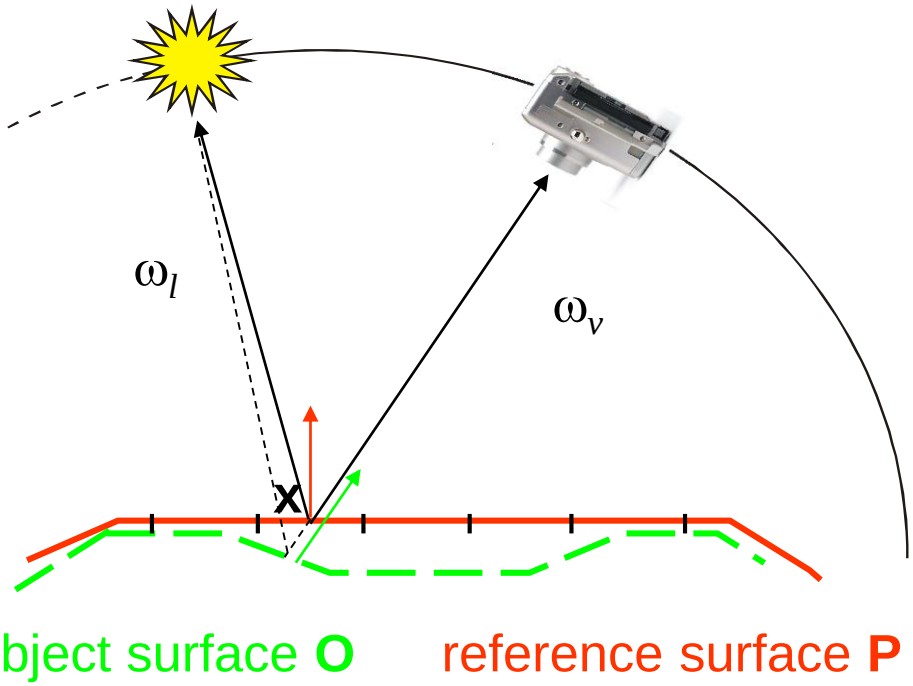
1.6 sec/image on 2GHz PIV

Preprocessing not mentioned

# Can we do even better?

## Problem Description

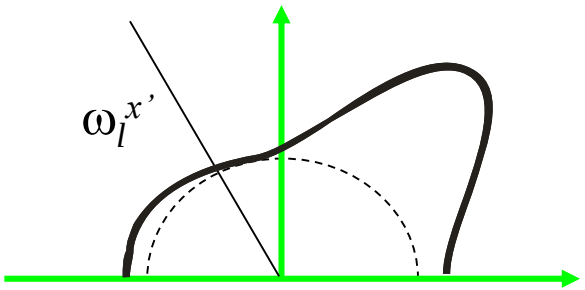
- What is the correct local coordinate frame to represent the reflection function at a surface point and how can it be found?
  - theoretically, the normal at a point x can be calculated from the geometry, but in general only proxy surface P is known
  - for anisotropic reflection the rotation around the normal matters



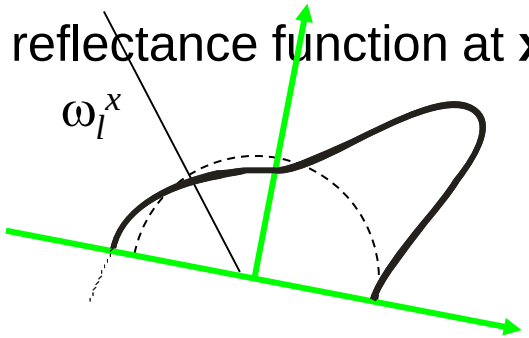
# Problem Description

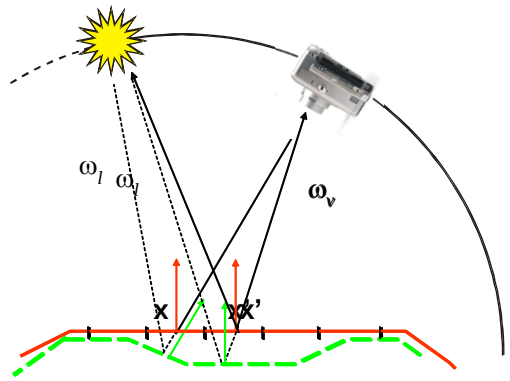
Normal deviations

- assume BRDF independent of spatial position
- reflectance function at **x**’

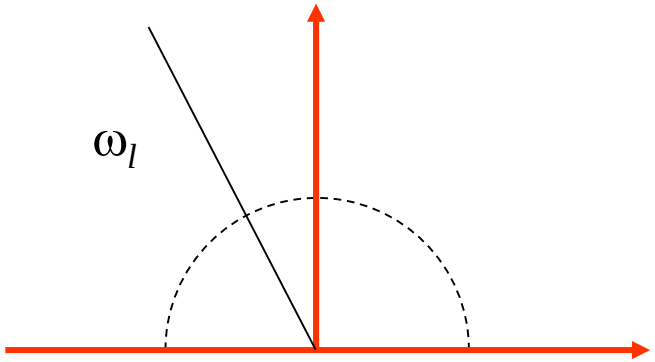



- reflectance function at **x**



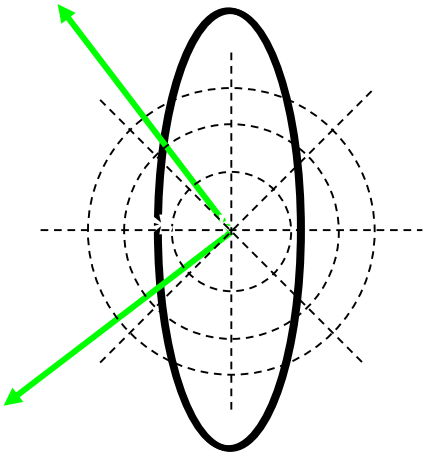

real object surface **O** reference surface **P**

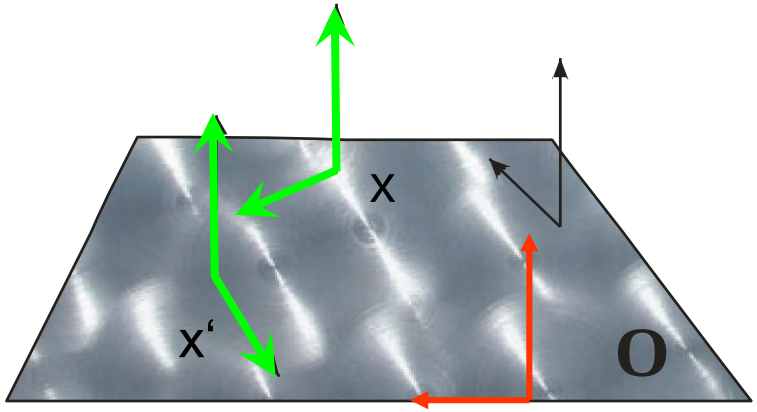


# Problem Description

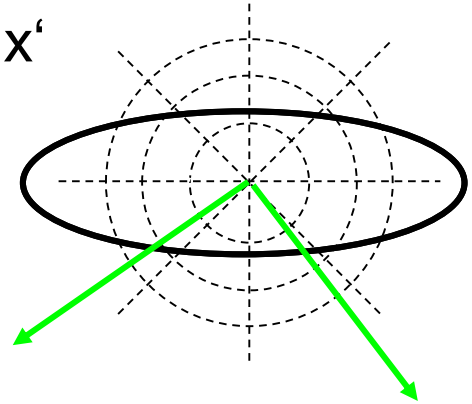
Deviations in orientation of local coordinate systems

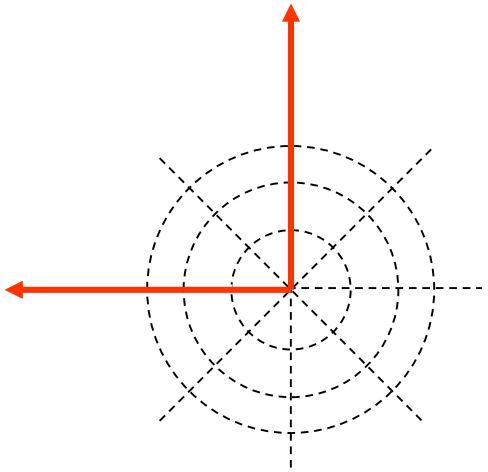
- x





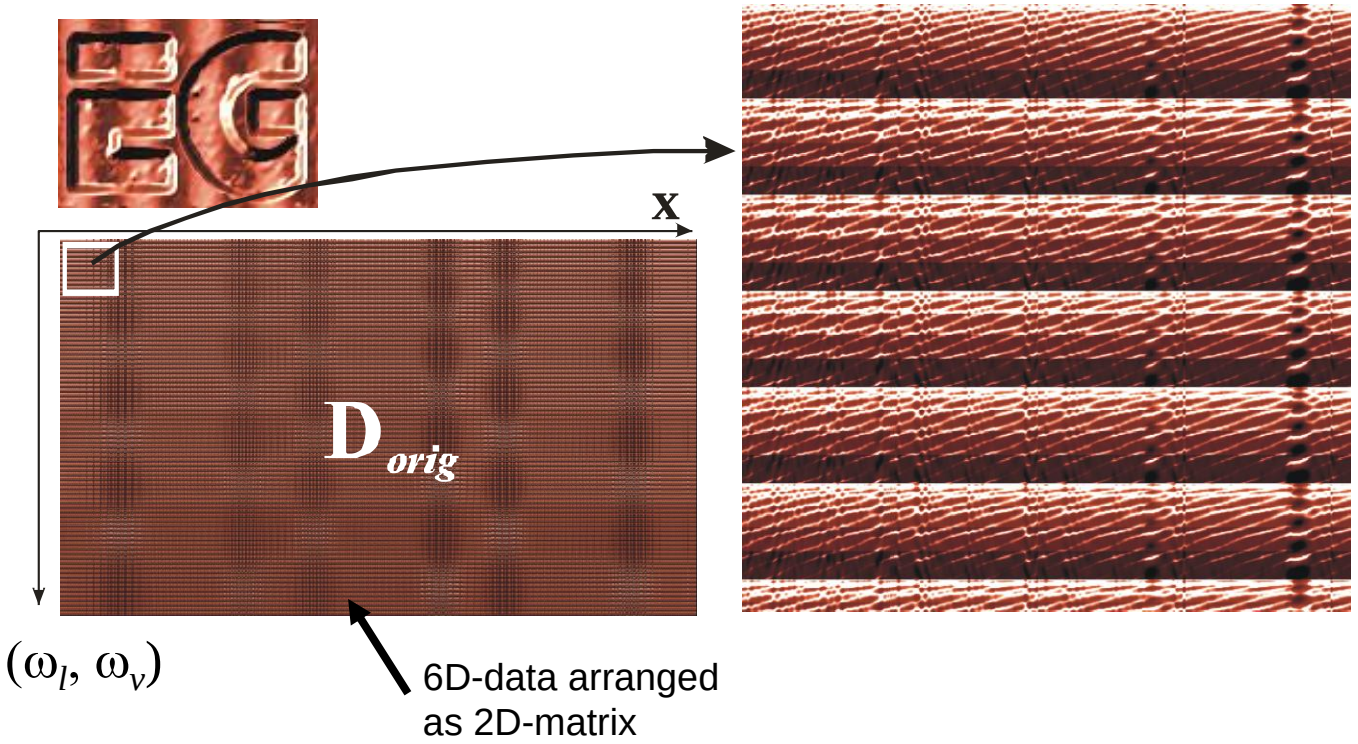

- x‘





# Problem Description

Synthetic reflectance field generated from heightfield, tangent map, ONE anisotropic BRDF using a planar reference surface

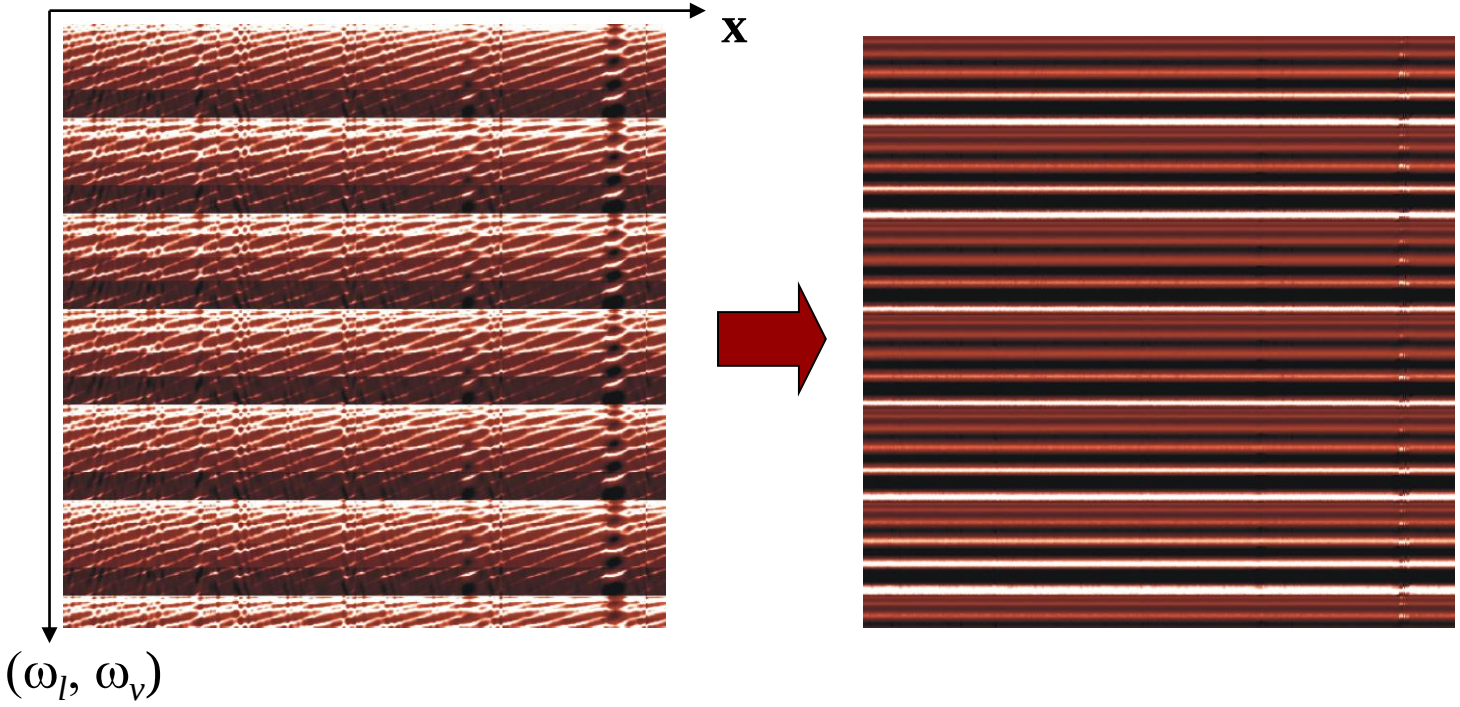



- Each surface point **x** appears to have a different BRDF!
- Could be corrected, if local coordinate systems known

# Our solution

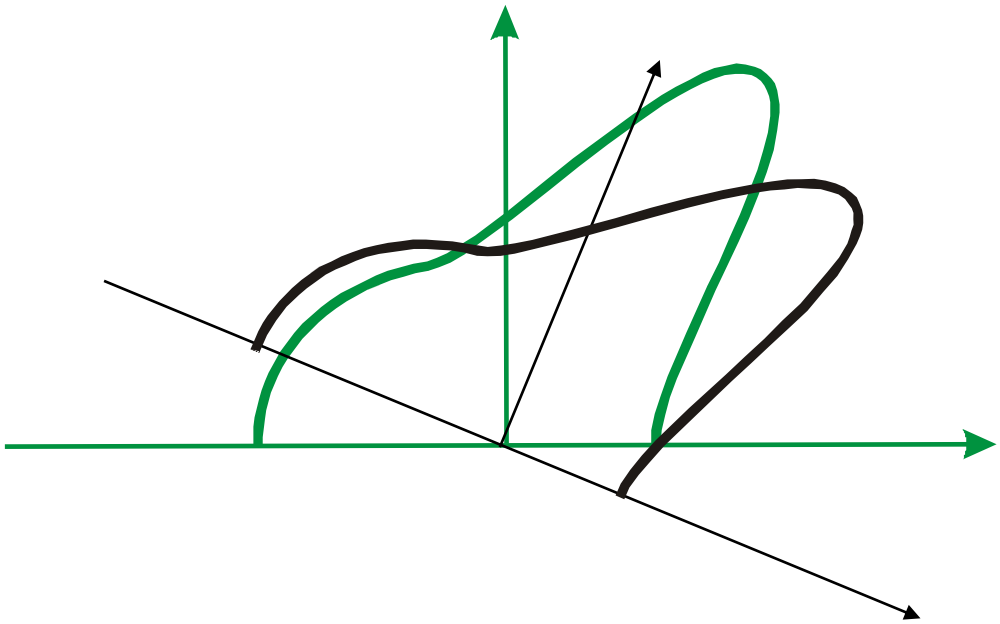
Global optimization of local coordinate systems by correlating reflectance functions

- reducing variance resulting from deviations between reference and real surface ➔ improved compression
- no assumptions about BRDF properties
- made efficient using FFTs on the rotation group SO(3)



# Method

Find rotation which maximizes correlation between reflectance functions:



Align the main features of reflectance functions

# Spherical correlation

**a, b** per-texel reflectance functions

Rotation $$\mathbf{R}(\Phi) = R_z(\phi) \cdot R_y(\theta) \cdot R_z(\psi)$$

Correlation

$$C_{\mathbf{a},\mathbf{b}}(\Phi) = \sum_{\omega_v \in \tilde{\Omega}^+} \sum_{\omega_l \in \tilde{\Omega}^+} \mathbf{a}(\omega_v, \omega_l)\mathbf{b}(\mathbf{R}^T(\Phi)\omega_v, \mathbf{R}^T(\Phi)\omega_l)$$

$$\bar{\Phi} = \arg\max_{\Phi} C_{\mathbf{a},\mathbf{b}}(\Phi)$$

# Correlating Reflectance Functions

Whole dataset:

- Correlate with mean

```
m = initialize mean
repeat
        for all reflectance functions a
                Φ_a = argmax C_{m,a}(Φ)
        end for
        m = mean of rotated reflectance
functions
until increase in correlation below thresh
```

- Combine with clustering for complex datasets with different materials

# Correlating Reflectance Functions

Computing

$$\bar{\Phi} = \arg\max_{\Phi} C_{\mathbf{a},\mathbf{b}}(\Phi)$$

in a brute force fashion requires evaluation of $C_{\mathbf{a,b}}(\Phi)$ for a dense grid $\{\Phi_{ijk} = (\theta_i, \phi_j, \psi_k)\}$ of rotations

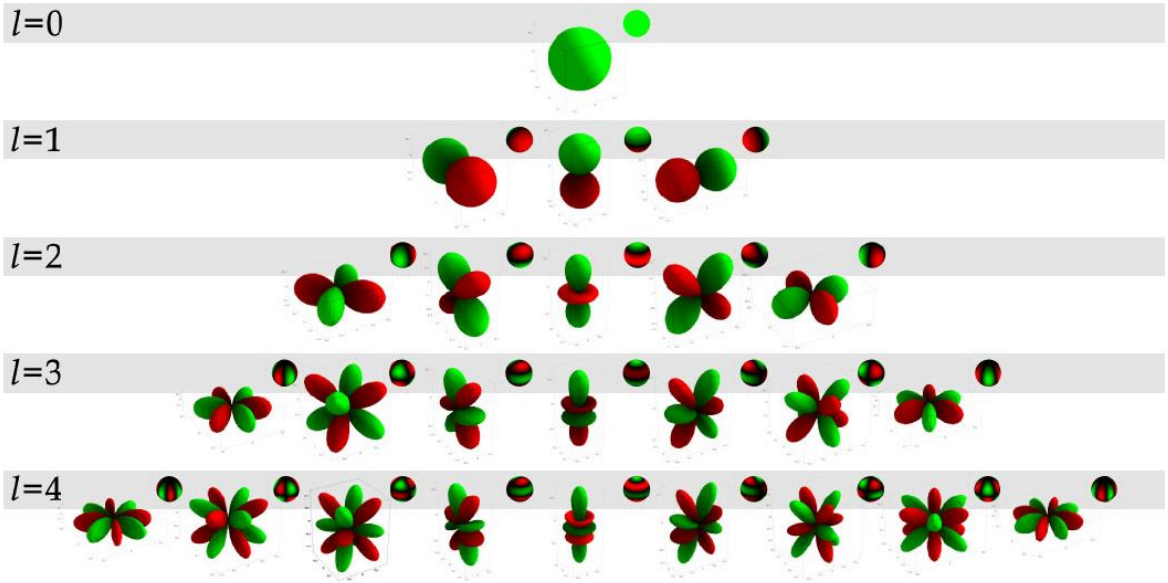
takes seconds…

Can we do better?

# Correlating reflectance Functions

Our solution

- project data into Spherical Harmonics Basis
- apply fast Fourier transform over rotation group SO(3)



# SO(3) and its Fourier Transform

Fourier decomposition over the sphere (band-limited)

$$\mathbf{a}(\omega) = \sum_{l=0}^{L} \sum_{|m|\leq l} \hat{a}_m^l Y_m^l(\omega)$$

$$\mathbf{b}(\omega) = \sum_{l=0}^{L} \sum_{|m|\leq l} \hat{b}_m^l Y_m^l(\omega)$$

Rotated basis functions can be decomposed

$$Y_m^l(\mathbf{R}^T(\Phi)\omega) = \sum_{|k|\leq l} D_{km}^l(\Phi) Y_k^l(\omega)$$

Wigner-D functions

# SO(3) and its Fourier Transform

Rotating a spherical function corresponds to „rotating“ its SH-coefficients

$$\mathbf{b}(\mathbf{R}^T(\Phi)\omega) = \sum_{l=0}^{L} \sum_{|k|\leq l} \left( \sum_{|m|\leq l} \hat{b}_m^l D_{km}^l(\Phi) \right) Y_k^l(\omega)$$

Correlation reduces to

$$C_{\mathbf{a},\mathbf{b}}^{S^2}(\Phi) = \sum_{l=0}^{L} \sum_{|k|\leq l} \sum_{|m|\leq l} \hat{a}_k^l \overline{\hat{b}_m^l D_{km}^l(\Phi)}$$

Wigner-Ds are SO(3) Fourier-base

# SO(3) and its Fourier Transform

Inverse SO(3) Fourier transform

$$C_{\mathbf{a},\mathbf{b}}^{S^2}(\Phi) = \sum_{l=0}^{L} \sum_{|k|\leq l} \sum_{|m|\leq l} \hat{a}_k^l \overline{\hat{b}_m^l D_{km}^l(\Phi)}$$

Direct evaluation has complexity $O(L^3)$

Evaluation for $L^3$ rotations has complexity $O(L^6)$

Fast Fourier Transform (FFT) on SO(3)

Evaluates $(2L+1)^3$ rotations in $O(L^3 \log^2 L)$ time!!

KOSTELEC P., ROCKMORE D.: *FFTs on the rotation group*. Tech. Rep. 03-11-060, Santa Fe Institute's Working Paper Series, 2003

# Generalization to 4D

Tensor product approach

$$\mathbf{a}(\omega_v, \omega_l) = \sum_{l_1, l_2}^{L} \sum_{m_1, m_2} \hat{a}_{m_1 m_2}^{l_1 l_2} Y_{m_2}^{l_2}(\omega_v) Y_{m_1}^{l_1}(\omega_l)$$

....

$$C_{\mathbf{a},\mathbf{b}}^{S^2 \times S^2}(\Phi) = \sum_{l_1 l_2}^{L} \sum_{m_1 m_1'} \sum_{m_2 m_2'} \hat{a}_{m_1 m_2}^{l_1 l_2} \overline{\hat{b}_{m_1' m_2'}^{l_1 l_2} D_{m_2 m_2'}^{l_2}(\Phi) D_{m_1 m_1'}^{l_1}(\Phi)}$$

Details in the paper…

# Timings

$$\bar{\Phi} = \arg\max_{\Phi} C_{\mathbf{a},\mathbf{b}}(\Phi)$$

| *L* | ***Direct (brute force)*** | ***Direct(SH)*** | ***FFT*** |
|---|---|---|---|
| 4 | 291 | 26 | 2 |
| 8 | >1500 | >3500 | 23 |
| 16 | N.N. | N.N. | 445 |

in ms (Athlon 3200+)

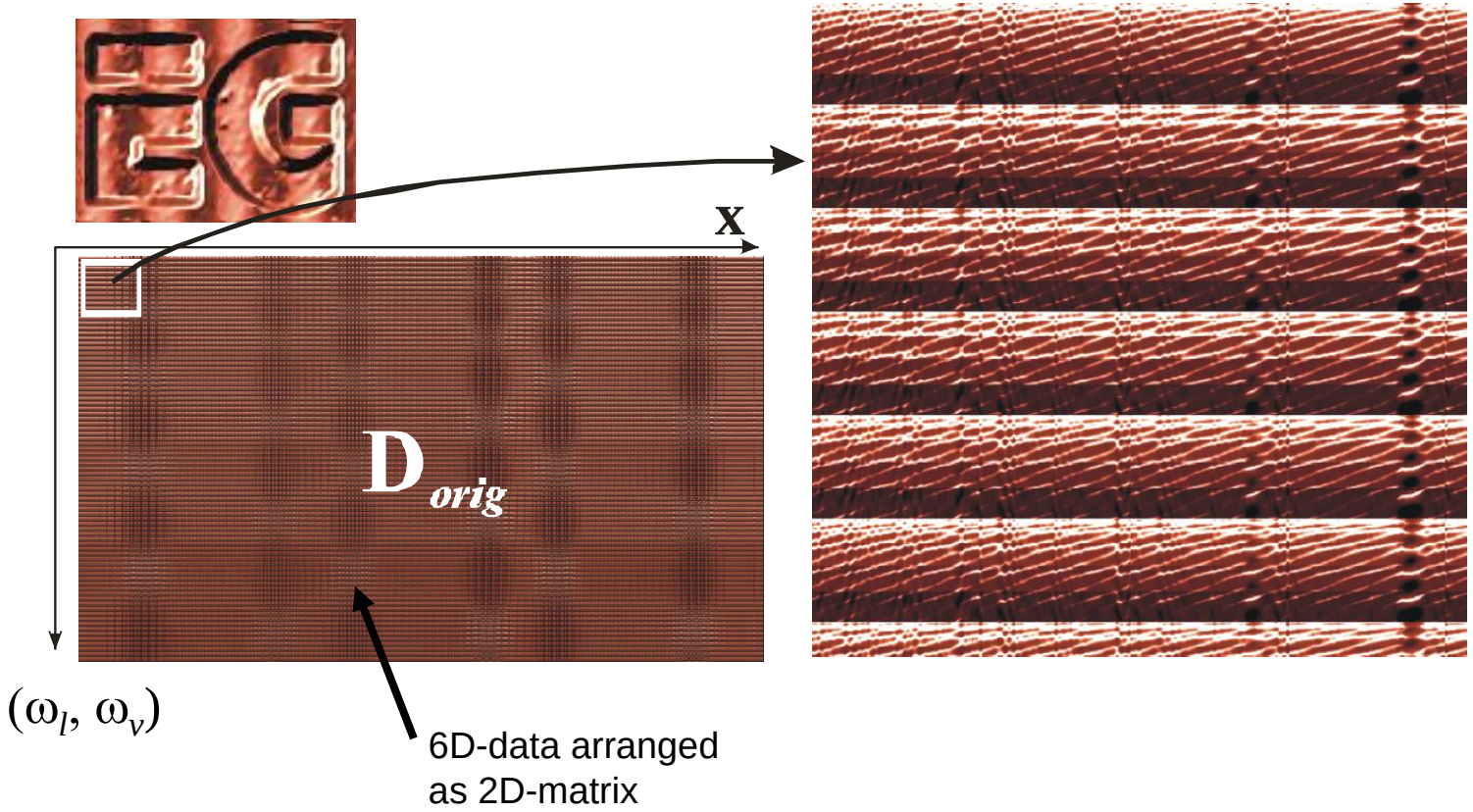
L=8

➔4913 rotations – still sparse

➔Use result as initialization for non-linear optimization step (Levenberg-Marquardt)

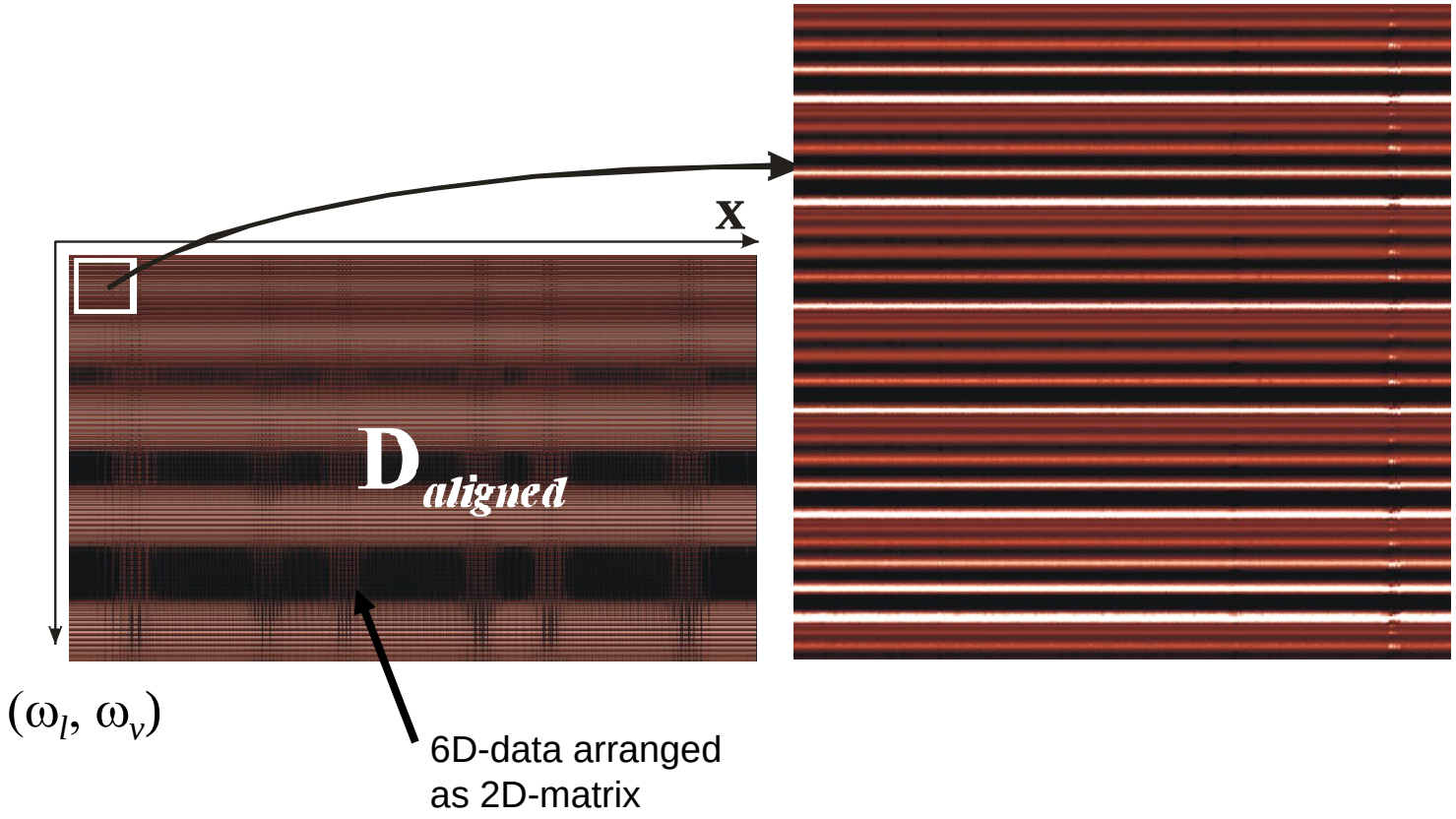
# Synthetic Example

## Before alignment



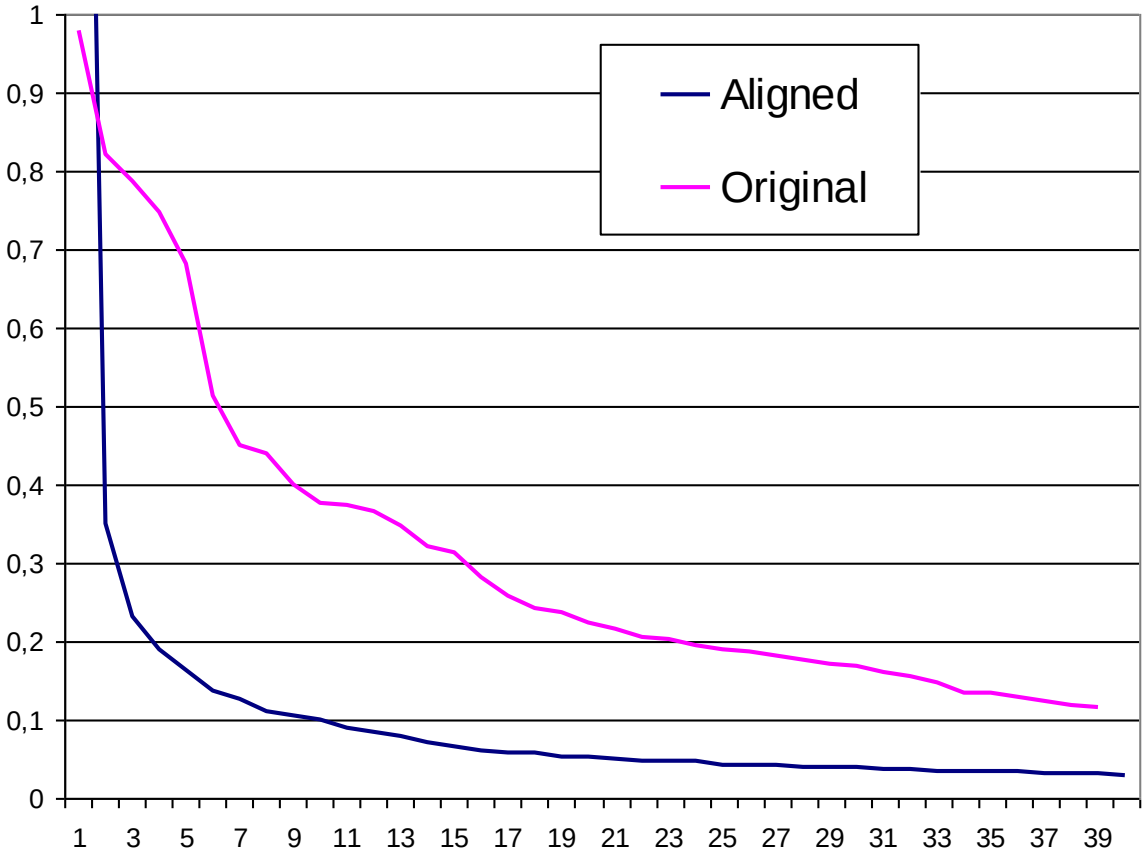
# Synthetic Example

After alignment

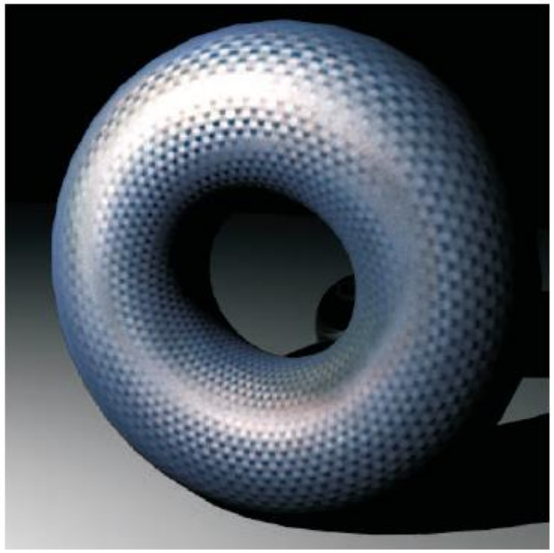


# Synthetic Example

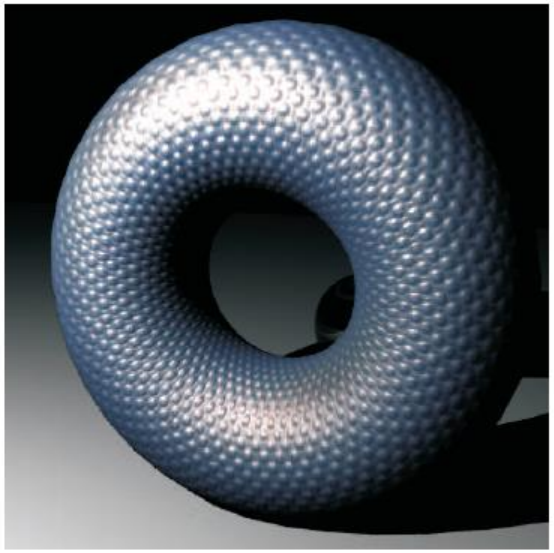
## Singular value plot



# Bidirectional Texture Function 1

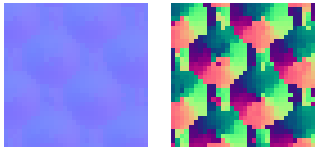


2 SVD components, 200 KB



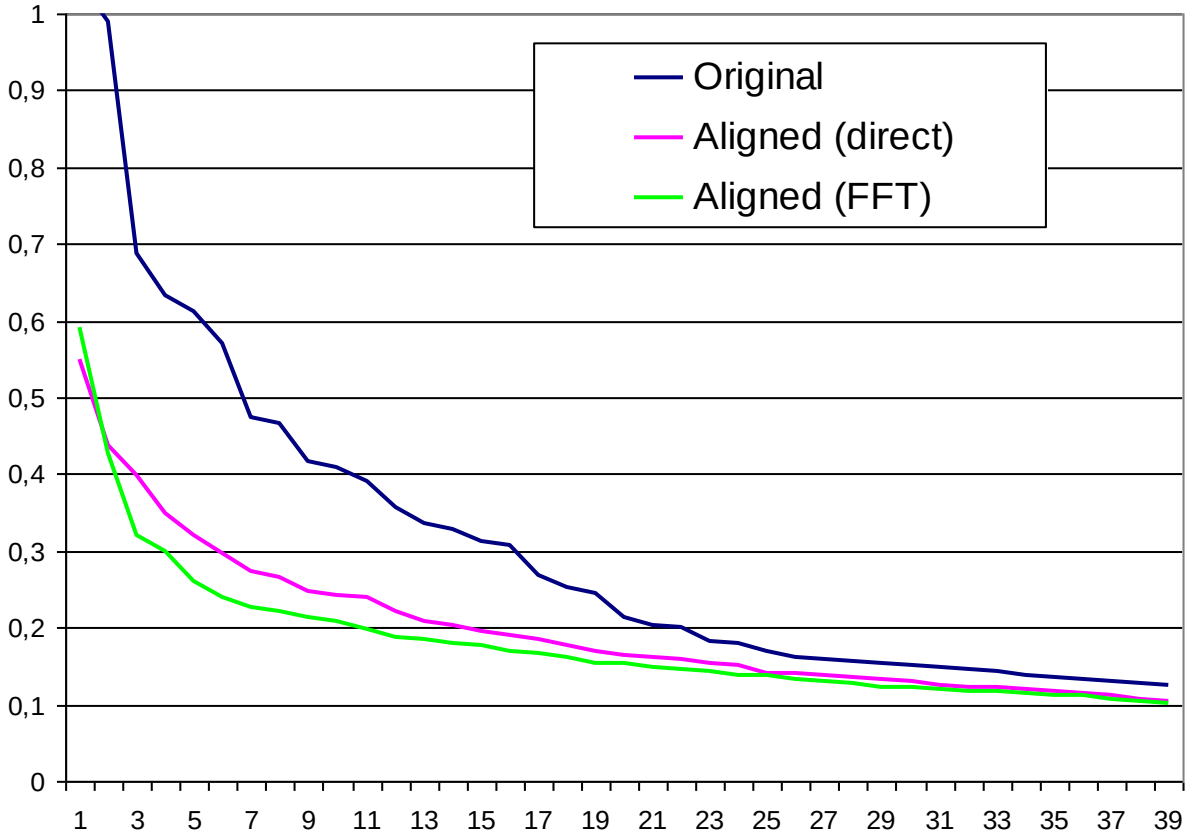
aligned, 2 SVD components, 200 KB

uncompressed 900 MB

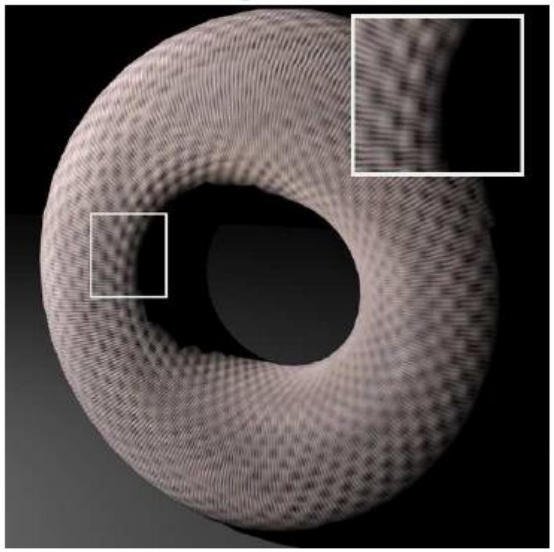


# Bidirectional Texture Function 1

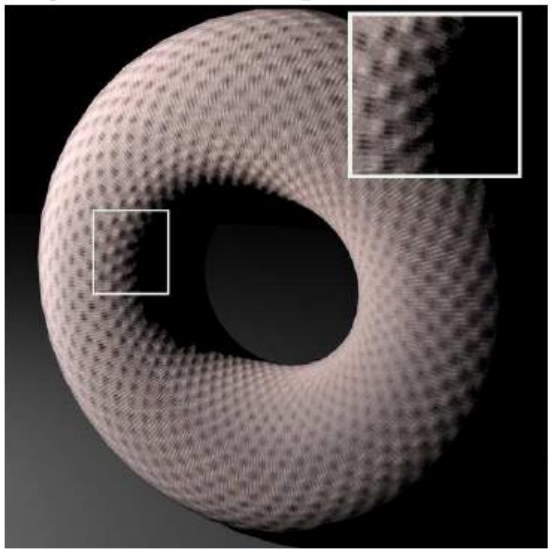
Singular value plot



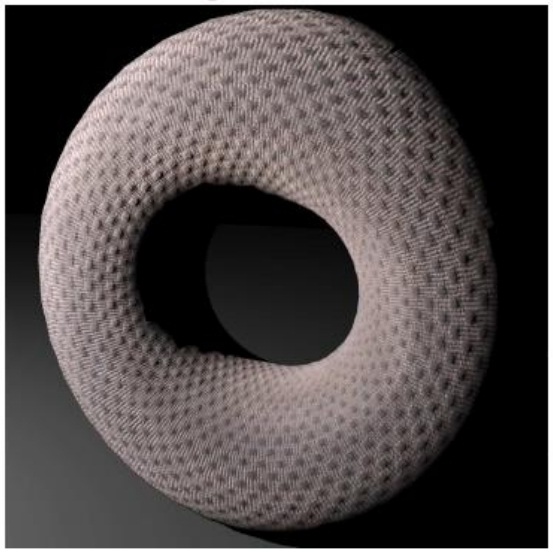
# Bidirectional Texture Function 2



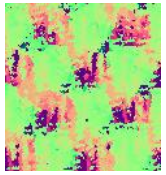
12 SVD components, 1 MB



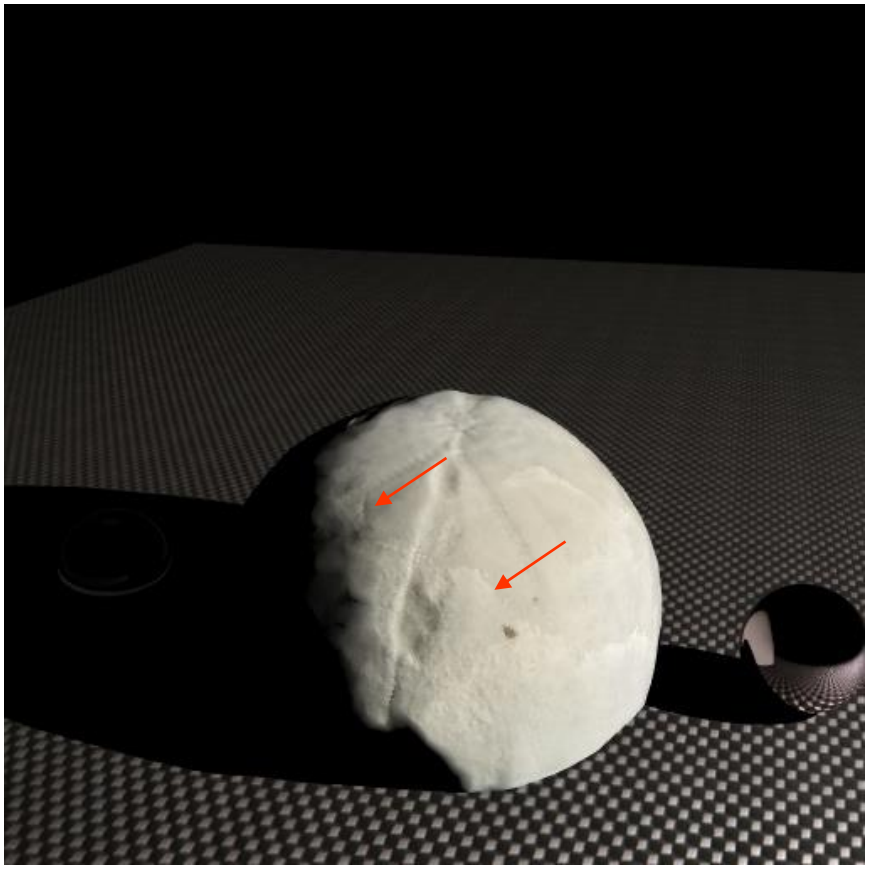
aligned, 12 SVD components, 1 MB



uncompressed 900 MB



# 6D Surface Reflectance Field

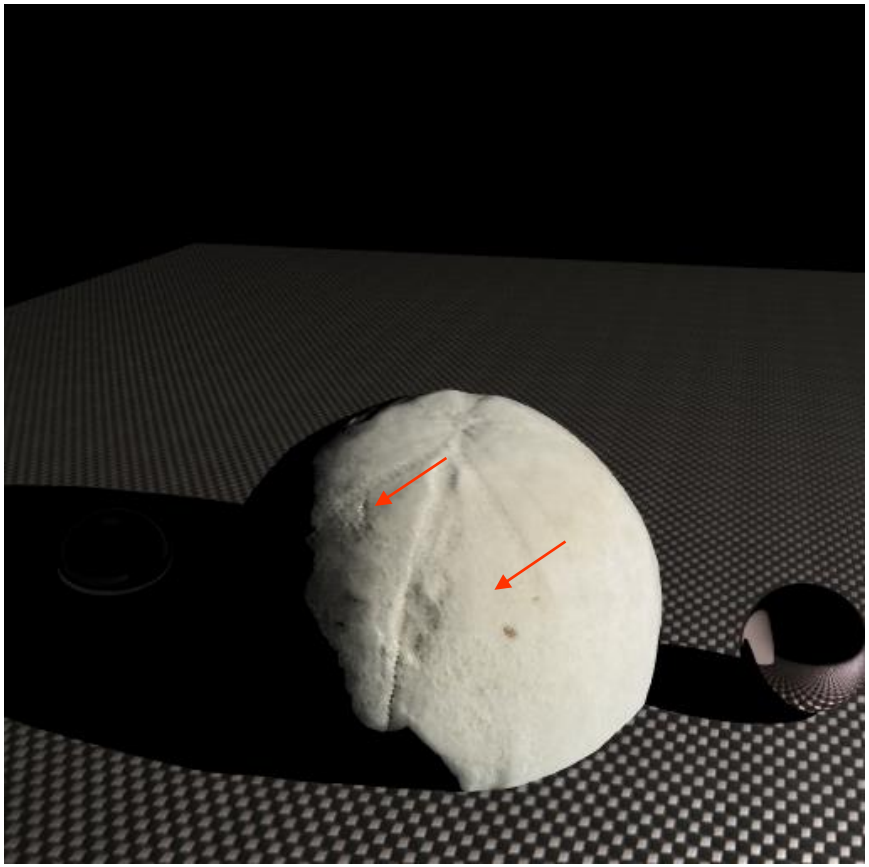


Fossile sea urchin

Surface details important for archeologists

LocalPCA compression (30 MB)

# 6D Surface Reflectance Field - Aligned



Fossile sea urchin

Surface details important for archeologists

LocalPCA compression (30 MB)

# 6D Surface Reflectance Field

**Image-Based Lighting (Uffizi)**

# 6D Surface Reflectance Field

**Image-Based Lighting (Uffizi) - Aligned**

# References


1. Design and Implementation of Practical Bidirectional Texture Function Measurement Devices focusing on the Developments at the University of Bonn, *Christopher Schwartz, Ralf Sarlette, Michael Weinmann, Martin Rump und Reinhard Klein*, In: Sensors (Apr. 2014), 14:5
2. Lightdrum—Portable light stage for accurate btf measurement on site. *Havran, Vlastimil, Jan Hošek, Šárka Němcová, Jiří Čáp, and Jiří Bittner*. *Sensors* 17, no. 3 (2017): 423.
3. Method and Apparatus for Digitizing the Appearance of A Real Material, *Kohlbrenner et al*. , US Patent US20160171748A1
4. "New scanning gonio-photometer for extended BRTF measurements. *Apian-Bennewitz, Peter.* In Reflection, Scattering, and Diffraction from Surfaces II, vol. 7792, p. 77920O. International Society for Optics and Photonics, 2010.
5. **pab Ltd Gonio-Photometer II, http://www.pab.eu/gonio-photometer/**
6. Realistic Graphics Lab, Gonio-photometer, *Wenzel Jakob*, http://rgl.epfl.ch/pages/lab/pgII (checked on 2020-07-01)